TU Delft

Delft University of Technology

Document Version
Final published version

Licence
CC BY

Citation (APA)
Kaseb, Z. (2026). *Quantum Computing and Artificial Intelligence for Power System Modeling and Control*. [Dissertation (TU Delft), Delft University of Technology]. https://doi.org/10.4233/uuid:883bb1e3-db98-4003-b667-3f5597f6e301

Important note
To cite this publication, please use the final published version (if applicable).
Please check the document version above.

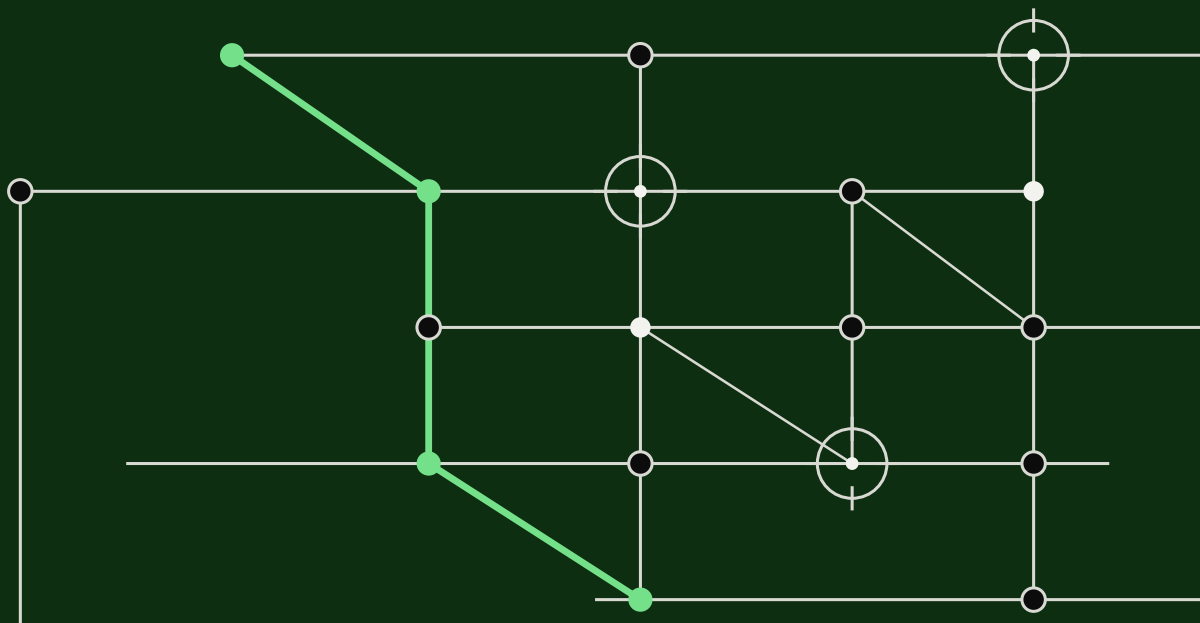
Copyright
In case the licence states “Dutch Copyright Act (Article 25fa)”, this publication was made available Green Open Access via the TU Delft Institutional Repository pursuant to Dutch Copyright Act (Article 25fa, the Taverne amendment). This provision does not affect copyright ownership.
Unless copyright is transferred by contract or statute, it remains with the copyright holder.

Sharing and reuse
Other than for strictly personal use, it is not permitted to download, forward or distribute the text or part of it, without the consent of the author(s) and/or copyright holder(s), unless the work is under an open content license such as Creative Commons.

Takedown policy
Please contact us and provide details if you believe this document breaches copyrights.
We will remove access to the work immediately and investigate your claim.

*This work is downloaded from Delft University of Technology.*

DELFT UNIVERSITY OF TECHNOLOGY 2026



# Quantum Computing and Artificial Intelligence

## for Power System Modeling and Control

Zeynab Kaseb

# Quantum Computing and Artificial Intelligence for Power System Modeling and Control

# Quantum Computing and Artificial Intelligence for Power System Modeling and Control

**Dissertation**

for the purpose of obtaining the degree of doctor
at Delft University of Technology
by the authority of the Rector Magnificus, prof. dr. ir. H. Bijl,
chair of the Board for Doctorates
to be defended publicly on
Thursday, 10 September 2026, at 12:30

by

**Zeynab KASEB**

This dissertation has been approved by the promotors.

Composition of the doctoral committee:

| | |
|---|---|
| Rector Magnificus, | *voorzitter* |
| Prof. dr. P. Palensky, | Delft University of Technology, *promotor* |
| Dr. rer. nat. M. Möller, | Delft University of Technology, *promotor* |
| Dr. P.P. Vergara Barrios, | Delft University of Technology, *copromotor* |
| *Independent members:* | |
| Prof. dr. ir. C. Vuik | Delft University of Technology, *committee member* |
| Prof. dr. ir. M. Popov | Delft University of Technology, *committee member* |
| Prof. dr. T. Keviczky | Delft University of Technology, *committee member* |
| Prof. dr. S. Chatzivasileiadis, | Technical University of Denmark, *committee member* |
| Dr. J.L. Cremer, | Delft University of Technology, *reserve member* |

This PhD thesis is part of the DATALESSs project 482.20.602, which is jointly financed by the Nederlandse Organisatie voor Wetenschappelijk Onderzoek (NWO) and the National Natural Science Foundation of China (NSFC).










*Keywords:* Advanced computational techniques; gate-based quantum computing; physics-informed neural networks; reinforcement learning; quantum annealing.

*Printed by:* Ipskamp

*Cover by:* Zeynab Kaseb

Copyright © 2026 by Z. Kaseb

ISBN 978-94-6518-375-6

An electronic copy of this dissertation is available at
https://repository.tudelft.nl/.

*To the teachers I met,*
*and the teachers I never realized I had.*

# Contents

**Summary** xi

**Preface** xiii

**1 Introduction** 1
- 1.1 Background and Motivation . . . 2
- 1.2 Research Objective and Questions . . . 4
  - 1.2.1 Physics-Informed Neural Networks for Power Flow . . . 4
  - 1.2.2 Quantum Neural Networks for Power Flow . . . 5
  - 1.2.3 Quantum Optimization Algorithms for (Optimal) Power Flow . . . 5
  - 1.2.4 Quantum Hardware-in-the-Loop . . . 5
  - 1.2.5 Reinforcement Learning for Newton-Raphson Initialization . . . 6
- 1.3 Contributions and Thesis Outline . . . 6
- References . . . 9

**2 Background** 15
- 2.1 Power System Modeling and Control . . . 16
  - 2.1.1 Power Flow Analysis . . . 17
  - 2.1.2 Optimal Power Flow . . . 17
- 2.2 Quantum Computing . . . 18
  - 2.2.1 Gate-Based Quantum Computing . . . 21
  - 2.2.2 Adiabatic Quantum Computing . . . 21
- 2.3 Artificial Intelligence . . . 22
  - 2.3.1 Deep Learning . . . 22
  - 2.3.2 Reinforcement Learning . . . 23
- References . . . 26

**3 Physics-Informed Neural Networks for Power Flow** 29
- 3.1 Motivation . . . 30
- 3.2 Proposed architecture: PINN4PF . . . 31
  - 3.2.1 Forward Pass . . . 31
  - 3.2.2 Loss Function . . . 33
- 3.3 Results . . . 36
  - 3.3.1 Model Setup . . . 36
  - 3.3.2 Model Performance . . . 37
  - 3.3.3 Model Sensitivity Analysis . . . 41
- 3.4 Discussion and Conclusions . . . 45
- References . . . 46

**4 Quantum Neural Networks for Power Flow 51**
4.1 Motivation . . . 52
4.2 Model Architecture . . . 53
4.3 Results . . . 55
4.3.1 Sensitivity Analysis for NN . . . 58
4.3.2 Convergence analysis of shots . . . 59
4.3.3 Impact of hardware noise . . . 60
4.3.4 Impact of the number of qubits . . . 60
4.3.5 PF Analysis . . . 61
4.4 Discussion and Conclusions . . . 62
References . . . 63

**5 Quantum Optimization Algorithms for (Optimal) Power Flow 67**
5.1 Motivation . . . 68
5.2 Combinatorial PF . . . 69
5.2.1 QUBO Model . . . 69
5.2.2 Ising Model . . . 71
5.3 Combinatorial OPF . . . 72
5.4 Algorithm Implementation . . . 75
5.4.1 Delta Update . . . 75
5.4.2 Higher-Order Reduction . . . 76
5.5 Solver Implementation . . . 77
5.5.1 QAOA . . . 77
5.5.2 Ising Machines . . . 77
5.6 Results . . . 78
5.6.1 Performance Comparison of GQC and AQC . . . 78
5.6.2 Full and Partitioned Formulations . . . 80
5.6.3 Well- and Ill-Conditioned Scenarios . . . 83
5.6.4 Scalability . . . 85
5.7 Discussion and Conclusions . . . 85
References . . . 87

**6 Quantum Hardware-in-the-Loop 89**
6.1 Motivation . . . 90
6.2 Quantum Hardware-in-the-Loop Framework . . . 91
6.2.1 Computational and Information Flow . . . 91
6.2.2 Hardware Configuration . . . 93
6.3 Results . . . 94
6.3.1 RES-Integrated Test System . . . 95
6.3.2 Computational Performance Assessment . . . 96
6.3.3 Error Assessment . . . 98
6.4 Discussion and Conclusions . . . 100
References . . . 103

**7 Reinforcement Learning for Newton-Raphson Initialization 105**
7.1 Motivation . . . 106

7.2 Results . . . 107
7.2.1 RL for accelerating NR convergence . . . 107
7.2.2 Scalability of RL for accelerating NR convergence . . . 110
7.2.3 Quantum-enhanced RL environment update mechanism . . . 112
7.2.4 Quantum Hardware Implementation . . . 114
7.3 Discussion and Conclusions . . . 116
References . . . 117

**8 Conclusions** **121**
8.1 Contributions to Research Questions . . . 122
8.2 Discussion and Recommendations . . . 124
References . . . 125

**Acknowledgements** **127**

**Curriculum Vitæ** **131**

**List of Publications** **133**

# Summary

This PhD thesis addresses a fundamental challenge in power system applications: how to reliably model and control modern power systems under increasing uncertainty and operational constraints. As modern power systems evolve toward highly distributed, renewable-rich infrastructures, classical numerical methods for power flow (PF) and optimal power flow (OPF) face intrinsic limitations. These limitations are not merely computational but structural: nonlinear solvers, such as Newton–Raphson (NR), exhibit sensitivity to initialization, numerical instability in ill-conditioned scenarios, and scaling behavior that is increasingly incompatible with the dimensionality requirements of future power system operation. Despite decades of algorithmic refinement, these challenges remain only partially resolved within classical computation.

The thesis systematically investigates advanced computational paradigms, i.e., artificial intelligence (AI) and quantum computing (QC), as complementary tools for overcoming these fundamental challenges. Rather than proposing incremental solver modifications, the thesis develops new conceptual formulations for PF and OPF that rethink how physical structure, optimization, and computation are represented. First, a physics-informed neural network (PINN) framework is introduced that partially encodes power balance equations directly into the learning process. This contribution advances existing PINN approaches by reducing dependence on complete network topology and line parameters while preserving physical interpretability. Second, the thesis explores quantum neural networks (QNNs) as a novel learning paradigm for PF analysis under data scarcity and noise, offering new insights into how quantum representations in the fault-tolerant quantum computing era can provide compact, potentially more expressive alternatives to classical deep learning.

Beyond learning-based methods, this PhD thesis provides a foundational contribution by reformulating PF and OPF as quadratic unconstrained binary optimization (QUBO) and Ising models, enabling their solution using quantum and quantum-inspired hardware. This reformulation constitutes a conceptual shift from continuous, nonlinear equation solving to discrete energy minimization, revealing new pathways to address scalability. The thesis further contributes to the first systematic comparison of gate-based and adiabatic quantum computing paradigms for PF and OPF across system sizes and operating conditions.

To bridge the gap between theoretical quantum algorithms and real-world power system requirements, this PhD thesis further introduces a quantum hardware-in-the-loop (QHIL) framework. This framework enables the evaluation of quantum solvers within closed-loop, real-time power system simulators. Therefore, critical system-level challenges, such as communication latency, embedding overhead, and real-time feasibility, are identified. These challenges should be addressed before

quantum computing can be meaningfully deployed in operational power systems.

Finally, recognizing the enduring relevance of classical solvers, this PhD thesis contributes a reinforcement learning (RL) framework that enhances NR convergence reliability through learned initialization strategies. By integrating a quantum-enhanced environment update mechanism, the thesis further explores how quantum optimization algorithms can accelerate learning and improve robustness without replacing well-established numerical methods.

In summary, this PhD thesis contributes new scientific knowledge by reframing PF and OPF through the lenses of scientific machine learning, quantum optimization algorithms, and hybrid computational architectures. It demonstrates that AI and QC are not merely faster solvers but fundamentally different tools that enable new representations, formulations, and system-level integrations. These contributions lay a conceptual and methodological foundation for next-generation power system modeling and control, with implications extending beyond power systems to other nonlinear engineering domains.

# Preface

George Box once said, ‘All models are wrong, some are useful’. So what can we do, then, on our path toward a net-zero energy future? One possible answer is to continuously challenge and redefine our assumptions about the models. Quantum computing and artificial intelligence are tools that allow us to reimagine these models, but how do we get there? This is the question I tried to address during my PhD.

*Zeynab Kaseb*
*Delft, February 2026*

# 1

# Introduction

## 1.1. Background and Motivation

Modern power systems are evolving from traditionally centralized, predictable infrastructures into highly distributed, dynamic, and data-rich cyber-physical-social energy systems. This evolution is primarily driven by the global commitment to achieving a net-zero energy future and the rapid growth of renewable energy sources (RES), including wind turbines, solar panels, electric vehicles, and energy storage systems, among others [1]. While this transition enables cleaner and more flexible power systems, it simultaneously introduces challenges in modeling and control of power systems that may host thousands of controllable elements interacting across multiple temporal and spatial scales [2].

At the heart of efficient power system operation lie two foundational analytical tasks: power flow (PF) analysis and optimal power flow (OPF). PF analysis determines the steady-state voltages and line flows and serves as the basis for planning, operation, and stability assessment [3]. PF analysis requires solving a set of nonlinear, nonconvex algebraic equations whose computational burden grows rapidly with system size and complexity [4]. Traditional solvers, such as the Newton-Raphson (NR) method, are reliable for well-conditioned operating conditions but suffer from scalability limitations, sensitivity to initial values, and convergence failures in heavily loaded or ill-conditioned operating conditions [5]. OPF extends PF by optimizing economic, stability, and/or technical objectives subject to network constraints [6]. Classical OPF solvers, including linear, quadratic, and nonlinear formulations, as well as Newton-based methods, have enabled decades of power system optimization. Yet, their performance deteriorates under the uncertainty, dimensionality, and non-linearity inherent in modern RES-rich power systems, often resulting in slow convergence, local minima, or computational intractability for real-time applications [7]. Ineffective PF analysis and OPF under these circumstances can lead to safety threats, including renewable energy generation curtailment and blackouts, as well as difficulties in accommodating distributed energy resources [8].

Several advanced algorithms have been developed in the literature to address the challenges faced by PF analysis, including scalability [9–13] and the management of ill-conditioned scenarios [14–19]. These studies primarily focus on advancing programming techniques and formulating mathematical models grounded in classical computational principles. Efforts to address the scalability concern concentrate on mitigating the computational challenges. For example, Lopez et al. [9] developed an extended dynamic programming approach based on the optimality principle of the recursive Hamilton-Jacobi-Bellman equations for distribution systems. The scalability of the proposed approach was demonstrated through benchmarking against commercial programming solvers. Goncalves et al. [10] formulated a mixed integer linear programming model for the operation planning problem of distribution systems. The proposed formulation uses linear expressions to ensure convergence to the optimal solution. Dai et al. [11] proposed a Gauss-Newton algorithm to improve scalability by reformulating the PF problem as a zero-residual least squares problem. Idema et al. [12] presented iterative linear solvers that offer enhanced scalability, making them suitable for the evolving

modern power systems. In [13], the authors introduced a sparse PF and integrated parallelization to accelerate computations in large power systems. Across these works, scalability is pursued within the classical computational paradigm, i.e., through linearization, reduced-order modeling, or parallelization. So, the underlying nonlinear, nonconvex formulation is reshaped. The resulting gains, therefore, tend to be specific to a given solver or formulation, which is what later motivates interest in non-classical alternatives.

In ill-conditioned scenarios, conventional PF solvers, such as NR, often experience numerical instability during convergence. Consequently, the literature offers alternatives to mitigate these challenges. For example, Tostado-Veliz et al. [14] presented a four-stage algorithm to address ill-conditioned scenarios. The algorithm is particularly useful for solving systems with reactive generator constraints. Similarly, Tripathy [17] applied a variation of NR that incorporates Gaussian elimination to ensure the utilization of the most recent information at each iteration. Liu et al. [18] took a different approach, where the PF model is transformed into a hypothetical dynamic system by introducing a differential equation for the loading parameter. This innovation enables a non-iterative method based on differential transformations to solve dynamic systems effectively. Iwamoto [19] devised a PF calculation method, characterized by its simplicity, absence of mathematical approximations, and minimal additional computational overhead when incorporated into NR. Common to these robustness techniques is that they harden the iterative solver, e.g., via alternative update rules, transformed formulations, or auxiliary corrections, while retaining its dependence on initialization.

Despite significant progress in PF analysis and OPF algorithms, there remains a critical need for scalable, computationally efficient, and numerically stable algorithms to meet the growing requirements of modern power systems [20, 21]. Consequently, the power systems community is increasingly interested in exploring novel approaches, including quantum computing (QC) solutions [22–24] and artificial intelligence (AI) approaches [25, 26], which have shown promising results in various applications and are, therefore, expected to revolutionize the field of PF and OPF in the near future [27, 28].

QC has recently emerged as a promising approach for addressing various types of problems. It can be categorized into two key paradigms: gate-based quantum computing (GQC) and adiabatic quantum computing (AQC). Optimism for a quantum speedup stems from the potential of quantum algorithms to take advantage of unique properties, e.g., superposition and entanglement, which allow computations with fewer operations than classical algorithms. An increasing number of studies, therefore, explore how QC can address the complexities of modern power systems. Most studies implement the GQC paradigm, which can simulate specific computations using quantum gates and discrete time steps [29, 30]. GQC offers flexibility in algorithm design and the potential for quantum speedup on specific problem instances, such as linear system solving. However, GQC algorithms require a large number of qubits and quantum gates even for small-scale problems, which makes them susceptible to the noise inherent in current quantum computers [31, 32]. A few studies focus on the AQC paradigm [33, 34]. AQC is similar to GQC in

terms of computational power, but it operates in an analog manner, i.e., without quantum gates or discrete time steps [35, 36]. Thus, AQC is particularly well-suited to the current noisy-intermediate-scale quantum (NISQ) era [37].

On the other hand, AI approaches, and more specifically artificial neural networks (NNs), are currently the most powerful numerical tools for providing accurate approximations of nonlinear problems (e.g., [25, 26, 28]). Several studies have demonstrated the superiority of AI approaches in PF analysis in terms of computational time by orders of magnitude (e.g., [38–40]). At the same time, the accuracy of the solutions is competitive with that of conventional iterative numerical methods (e.g., [32, 41]). NNs, therefore, can address the challenges mentioned above by utilizing the availability of massive measurements and/or augmented data, learning complex input-output relationships that are often difficult or even impossible for conventional iterative numerical methods to comprehend, and achieving the accuracy required for real-world applications [42]. Nevertheless, NNs are subject to overfitting, poor generalization, and scalability issues. They are doubtful to meet the physical constraints. Furthermore, their performance relies heavily on the size and quality of the training dataset, where not enough data is always available due to privacy concerns and the presence of missing data, among others [43].

## 1.2. Research Objective and Questions

The objective of this thesis is to investigate advanced computational techniques, including QC and AI, to enhance power system modeling and control to address the emerging complexities of modern power systems. Throughout, these techniques are positioned to complement rather than replace classical solvers, which are the established industry baseline. That said, each technique is evaluated against the specific limitation of classical solvers it is intended to address. The thesis is structured around five research questions, each addressed by one core chapter, as presented in the following subsections.

### 1.2.1. Physics-Informed Neural Networks for Power Flow

Traditional NNs can approximate PF mappings but often require extensive training data. Physics-Informed Neural Networks (PINNs) address this limitation by embedding power balance equations into the learning process, thereby maintaining physical consistency.

We develop a PINN that incorporates prior knowledge from power balance equations while reducing dependency on full system topology or complete line parameter data. A hidden function partially encodes line characteristics, an efficient end-to-end architecture maps power injections to voltages, and an adaptive activation function enhances representation flexibility. Our research question addressed in Chapter 3 is: **RQ1 - How can prior physical knowledge be efficiently embedded into neural networks to improve the accuracy, interpretability, and data efficiency for PF analysis?**.

### 1.2.2. Quantum Neural Networks for Power Flow

Power systems increasingly operate under challenges, such as missing measurements. Quantum neural networks (QNNs) realized on fault-tolerant hardware are of growing interest because their expressive power does not scale in the same way as classical deep NNs, potentially enabling more noise-resilient models for PF estimation.

We investigate the applicability of QNNs to improve learning performance when training data are limited, incomplete, or noisy. We further assess the potential benefits and limitations of the proposed architecture compared to classical NNs for PF analysis. Thus, our research question addressed in Chapter 4 is: **RQ2 - How can quantum neural networks provide advantages over classical deep learning for PF analysis under data scarcity and noise?**.

### 1.2.3. Quantum Optimization Algorithms for (Optimal) Power Flow

Many core power system problems, including PF and OPF, can be interpreted through the lens of nonlinear optimization. Quantum optimization algorithms, particularly those based on quadratic unconstrained binary optimization (QUBO) and Ising models, offer a promising alternative for tackling combinatorial problems that are challenging for classical solvers. As quantum hardware matures, such formulations enable faster decision-making in increasingly complex power grids with high renewable penetration.

We reformulate PF and OPF as combinatorial optimization problems using QUBO and Ising models, and design a complete algorithmic pipeline compatible with GQC and AQC. We also evaluate performance across power system sizes, operating conditions, and solver types to identify practical advantages, limitations, and scalability characteristics. Our research question addressed in Chapter 5 is: **RQ3 - How can PF and OPF be reformulated for solution on quantum and quantum-inspired hardware?**.

### 1.2.4. Quantum Hardware-in-the-Loop

For practical operation of future power grids, emerging quantum algorithms need to be directly tested on real power systems. Quantum Hardware-in-the-Loop (QHIL) frameworks embed quantum optimization modules within closed-loop, real-time digital simulations, while offering realistic evaluation of latency, embedding overhead, numerical stability, robustness, and overall operational feasibility. By doing so, QHIL reduces the gap between quantum computing research and practical power system applications.

We develop and demonstrate a QHIL framework that integrates quantum(-inspired) processors with real-time power system simulators. We also analyze the system's real-time behavior under dynamic grid conditions, quantify performance and stability, and identify practical bottlenecks, such as embedding overhead. Our research question addressed in Chapter 6 is: **RQ4 - What are the key system-level**

**challenges in deploying quantum and quantum-inspired PF and OPF solvers within real-time power system simulators?**.

### 1.2.5. Reinforcement Learning for Newton-Raphson Initialization

NR remains the dominant PF solver, yet its convergence can degrade in stressed or ill-conditioned systems. Reinforcement learning (RL) offers a promising mechanism for learning robust initialization strategies that improve convergence, complementing traditional numerical solvers rather than replacing them. Integrating quantum mechanisms could further expand exploration capabilities, potentially enabling faster and more diverse policy discovery in challenging operating conditions.

We design an RL framework that learns improved initialization strategies for the NR method to reduce convergence failures in ill-conditioned or heavily loaded systems where standard heuristics, such as flat start, are most prone to fail. We further extend this framework with a quantum-enhanced environment update mechanism to investigate whether quantum sampling can facilitate exploration and accelerate policy learning. Thus, our research question addressed in Chapter 7 is: **RQ5 - How can reinforcement learning (RL) enhance the convergence reliability of classical PF solvers, and can quantum mechanisms improve RL learning efficiency?**.

## 1.3. Contributions and Thesis Outline

The aforementioned challenges necessitate the development of new approaches that are both computationally efficient and numerically stable. Therefore, this PhD thesis explores a suite of advanced computational technologies, spanning AI approaches and QC solutions, to address the challenges of PF analysis and OPF in modern power systems. The diagram of this thesis is shown in Figure 1.1. Chapter 2 provides the necessary theoretical and technical background, i.e., power system modeling and control, QC, and AI, upon which the subsequent chapters build. Readers may refer to that chapter for method summaries.

The first approach investigated in Chapter 3 is a novel hidden function derived from the power balance equations, which partially encodes line characteristics into a modified loss function as prior physical knowledge. In contrast to existing approaches that require full system topology data and/or complete line parameters (e.g., [44]), the proposed approach reduces dependency on such information while preserving physical interpretability. We extend the existing body of knowledge on physics-informed neural networks for PF analysis (e.g., [45, 46]) by introducing an efficient, computationally scalable end-to-end architecture that learns the mapping from net active and reactive power injections to complex bus voltages. An adaptive activation function is also employed to enhance the representational flexibility of NNs compared to conventional static activation functions.

While NNs offer significant advantages for PF analysis, their practical implementation presents particular challenges. Firstly, they rely on large, high-quality

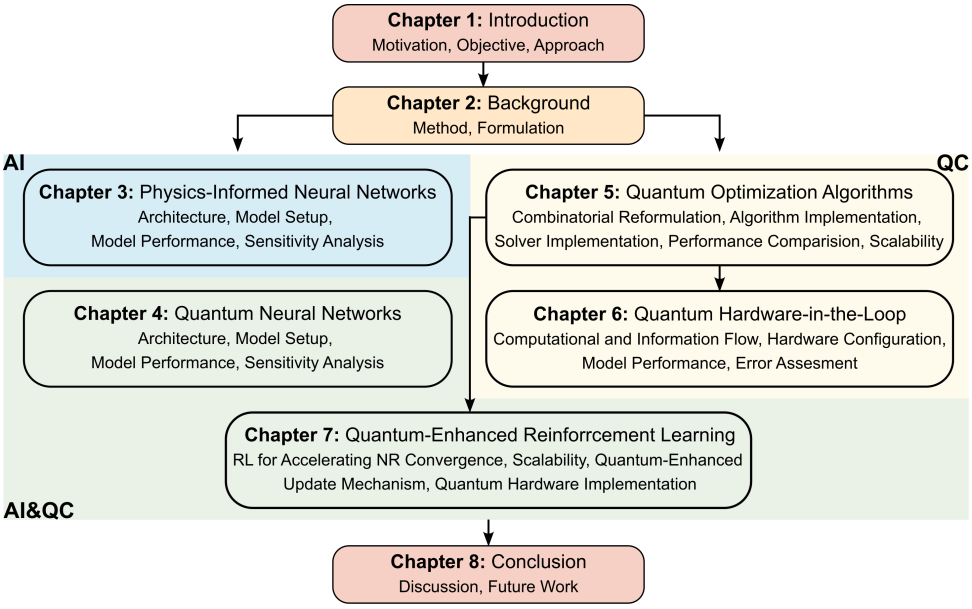



Figure 1.1: Connections between chapters of this PhD thesis.

datasets, which can be difficult to obtain due to privacy concerns and missing data. In addition, the computational complexity of the training process increases with the size and complexity of power systems. Finally, ensuring scalability, generalization ability, and robustness to noisy training data remain critical concerns for NNs. They are often case-specific and prone to overfitting to the training data, potentially compromising the reliability of NNs in safety-critical applications (e.g., [29, 47]).

A radically different machine learning approach that can help overcome the challenges faced by NNs is QNNs, meaning the enrichment of NNs with QC, which is increasingly gaining attention as it has the potential to address the complexities of modern power systems [36, 48]. Chapter 4 investigates QNNs that exploit quantum properties, such as superposition and entanglement, to enhance the training process but also address data scarcity concerns, which collectively make QNNs a potential candidate for learning from small and/or noisy datasets [49].

We further introduce quantum optimization algorithms, with a particular focus on quantum annealing and the Quantum Approximate Optimization Algorithm (QAOA) in Chapter 5. PF and OPF are reformulated as combinatorial optimization problems using QUBO and Ising models and solved using quantum(-inspired) hardware. Comprehensive experiments evaluate the performance of GQC versus AQC, full versus partitioned formulations for large networks, well- versus ill-conditioned operating points, and scalability up to a 1354-bus system. Figure 1.2 illustrates the scope of this research and situates the developed quantum algorithms within the broader context of solving practical problems with quantum hardware.

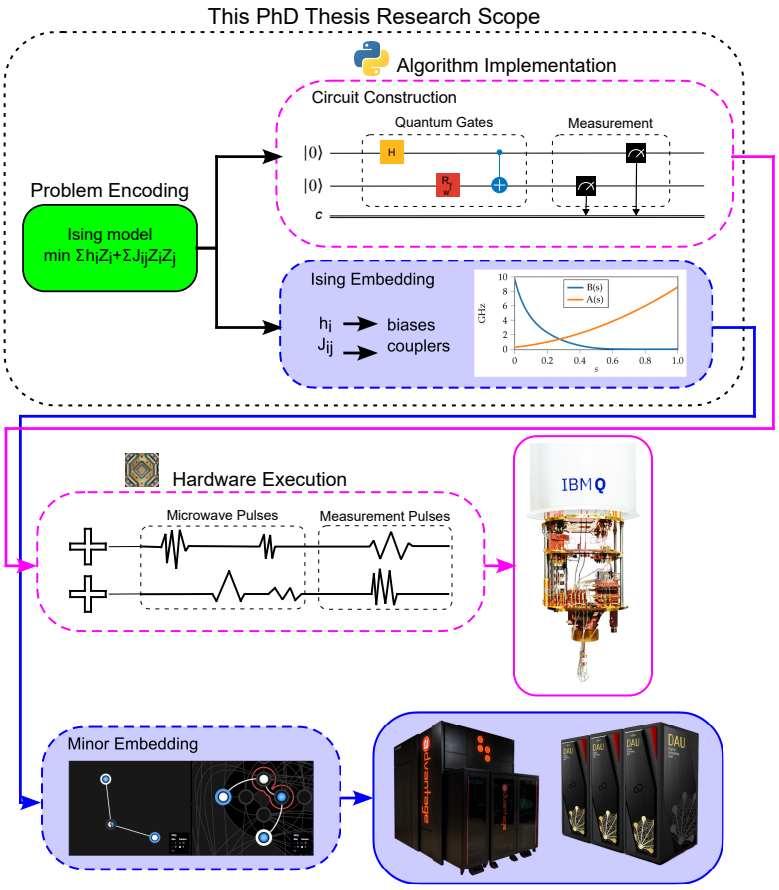



Figure 1.2: Overall scope of this PhD thesis within the broader workflow of solving combinatorial optimization problems using gate-based and adiabatic quantum hardware. The workflow comprises problem encoding, algorithm implementation, and hardware execution. Problem encoding refers to formulating the target minimization problem as an Ising model. Algorithm implementation includes quantum circuit design for gate-based quantum computing and Ising embedding for adiabatic quantum computing. Hardware execution involves applying microwave pulse sequences to gate-based quantum processors and realizing minor-embedded Ising models via physical couplers on adiabatic quantum hardware. The figure illustrates IBM gate-based quantum hardware for the former, and D-Wave quantum annealer and the Fujitsu digital annealer for the latter.

To validate the proposed quantum optimization algorithm in real-time simulations and uncover practical limitations and implementation challenges, we introduce a

QHIL framework in Chapter 6, providing the first demonstration of a real-time QHIL PF and OPF workflow. Hardware-in-the-loop (HIL) simulation couples physical hardware, such as quantum or quantum-inspired processors, with real-time power system simulators. This setup provides a controlled yet realistic environment for systematically evaluating algorithmic performance, stability, and responsiveness under dynamic grid conditions, including variable renewable generation and fluctuating loads (e.g., [50, 51]). For example, a quantum-in-the-loop framework that integrates GQC with real-time power system simulations is introduced in [52] to enhance real-time decision making by processing inputs from numerous distributed sensors and controllers.

Despite all these advancements, the NR method remains the workhorse of industry PF solvers due to its maturity and interpretability. Thus, rather than replacing NR, we develop an RL framework in Chapter 7 that enhances NR initialization to avoid convergence issues in ill-conditioned or heavily loaded power systems. Furthermore, we propose a quantum-enhanced RL environment update mechanism that enables partial quantum hardware embedding to improve learning efficiency. Chapter 8 concludes the thesis by summarizing the key contributions and outlining future research opportunities at the intersection of QC, AI, optimization, and power system modeling and control. We further discuss how the presented approaches are applicable to other domains and can be generalized to problems that involve numerical solvers.

# References

[1] M. S. Alam, F. S. Al-Ismail, A. Salem and M. A. Abido. 'High-level penetration of renewable energy sources into grid utility: Challenges and solutions'. In: *IEEE access* 8 (2020), pp. 190277–190299.

[2] L. Szabo, M. M. Girona, A. Jäger-Waldau, I. Kougias, A. Mezosi, F. Fahl and S. Szabo. 'Impacts of large-scale deployment of vertical bifacial photovoltaics on European electricity market dynamics'. In: *Nature Communications* 15 (1 Aug. 2024), p. 6681. DOI: 10.1038/s41467-024-50762-7.

[3] R. A. Jacob, S. Paul, S. Chowdhury, Y. R. Gel and J. Zhang. 'Real-time outage management in active distribution networks using reinforcement learning over graphs'. In: *Nature Communications* 15 (1 June 2024), p. 4766. DOI: 10.1038/s41467-024-49207-y.

[4] A. Song, Z. Dan, S. Zheng and Y. Zhou. 'An electricity-driven mobility circular economy with lifecycle carbon footprints for climate-adaptive carbon neutrality transformation'. In: *Nature Communications* 15 (1 July 2024), p. 5905. DOI: 10.1038/s41467-024-49868-9.

[5] S. A. Steinbach and M. J. Blaschke. 'How grid reinforcement costs differ by the income of electric vehicle users'. In: *Nature Communications* 15 (1 Nov. 2024), p. 9674. DOI: 10.1038/s41467-024-53644-0.

[6] M. Huneault and F. D. Galiana. 'A survey of the optimal power flow literature'. In: *IEEE Transactions on Power Systems* 6.2 (1991), pp. 762–770.

[7] N. Mbuli and W. Ngaha. 'A survey of big bang big crunch optimisation in power systems'. In: *Renewable and Sustainable Energy Reviews* 155 (2022), p. 111848.

[8] N. Sharma, A. Acharya, I. Jacob, S. Yamujala, V. Gupta and R. Bhakar. 'Major Blackouts of the Decade: Underlying Causes, Recommendations and Arising Challenges'. In: *2021 9th IEEE International Conference on Power Systems (ICPS)*. IEEE, Dec. 2021, pp. 1–6. DOI: 10.1109/ICPS52420.2021.9670166.

[9] J. C. Lopez, P. P. Vergara, C. Lyra, M. J. Rider and L. C. P. da Silva. 'Optimal Operation of Radial Distribution Systems Using Extended Dynamic Programming'. In: *IEEE Transactions on Power Systems* 33.2 (Mar. 2018), pp. 1352–1363. ISSN: 0885-8950. DOI: 10.1109/TPWRS.2017.2722399.

[10] R. R. Gonçalves, R. P. Alves, J. F. Franco and M. J. Rider. 'Operation Planning of Electrical Distribution Systems Using a Mixed Integer Linear Model'. In: *Journal of Control, Automation and Electrical Systems* 24.5 (Oct. 2013), pp. 668–679. ISSN: 2195-3880. DOI: 10.1007/s40313-013-0055-9.

[11] X. Dai, Y. Cai, Y. Jiang and V. Hagenmeyer. 'Rapid Scalable Distributed Power Flow with Open-Source Implementation'. In: *IFAC-PapersOnLine* 55.13 (2022), pp. 145–150. ISSN: 24058963. DOI: 10.1016/j.ifacol.2022.07.250.

[12] R. Idema, D. J. P. Lahaye, C. Vuik and L. van der Sluis. 'Scalable Newton-Krylov Solver for Very Large Power Flow Problems'. In: *IEEE Transactions on Power Systems* 27.1 (Feb. 2012), pp. 390–396. ISSN: 0885-8950. DOI: 10.1109/TPWRS.2011.2165860.

[13] L. Zeng, S. G. Alawneh and S. A. Arefifar. 'GPU-Based Sparse Power Flow Studies With Modified Newton's Method'. In: *IEEE Access* 9 (2021), pp. 153226–153239. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3127393.

[14] M. Tostado-Veliz, H. M. Hasanien, R. A. Turky, A. Alkuhayli, S. Kamel and F. Jurado. 'Mann-Iteration Process for Power Flow Calculation of Large-Scale Ill-Conditioned Systems: Theoretical Analysis and Numerical Results'. In: *IEEE Access* 9 (2021), pp. 132255–132266. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3114969.

[15] X. Yang and X. Zhou. 'Application of asymptotic numerical method with homotopy techniques to power flow problems'. In: *International Journal of Electrical Power & Energy Systems* 57 (May 2014), pp. 375–383. ISSN: 01420615. DOI: 10.1016/j.ijepes.2013.12.014.

[16] H. Sheng and H.-D. Chiang. 'CDFLOW: A Practical Tool for Tracing Stationary Behaviors of General Distribution Networks'. In: *IEEE Transactions on Power Systems* 29.3 (May 2014), pp. 1365–1371. ISSN: 0885-8950. DOI: 10.1109/TPWRS.2013.2289917.

[17] S. Tripathy, G. Prasad, O. Malik and G. Hope. 'Load-Flow Solutions for Ill-Conditioned Power Systems by a Newton-Like Method'. In: *IEEE Transactions on Power Apparatus and Systems* PAS-101.10 (Oct. 1982), pp. 3648–3657. ISSN: 0018-9510. DOI: 10.1109/TPAS.1982.317050.

[18] Y. Liu, K. Sun and J. Dong. 'A Dynamized Power Flow Method Based on Differential Transformation'. In: *IEEE Access* 8 (2020), pp. 182441–182450. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2020.3028060.

[19] S. Iwamoto and Y. Tamura. 'A Load Flow Calculation Method for Ill-Conditioned Power Systems'. In: *IEEE Transactions on Power Apparatus and Systems* PAS-100.4 (Apr. 1981), pp. 1736–1743. ISSN: 0018-9510. DOI: 10.1109/TPAS.1981.316511.

[20] K. Baker. 'Solutions of DC OPF are Never AC Feasible'. In: *arXiv preprint* (Nov. 2019). DOI: arXiv:1912.00319.

[21] J. S. Giraldo, O. D. Montoya, P. P. Vergara and F. Milano. 'A fixed-point current injection power flow for electric distribution systems using Laurent series'. In: *Electric Power Systems Research* 211 (Oct. 2022), p. 108326. ISSN: 03787796. DOI: 10.1016/j.epsr.2022.108326.

[22] H. Leipold and F. M. Spedalieri. 'Quantum annealing with special drivers for circuit fault diagnostics'. In: *Scientific Reports* 12.1 (July 2022), p. 11691. ISSN: 2045-2322. DOI: 10.1038/s41598-022-14804-8.

[23] H. Jing, Y. Wang and Y. Li. 'Data-driven quantum approximate optimization algorithm for power systems'. In: *Communications Engineering* 2 (2023). ISSN: 2731-3395. DOI: 10.1038/s44172-023-00061-8.

[24] F. F. C. Silva, P. M. S. Carvalho and L. A. F. M. Ferreira. 'A quantum computing approach for minimum loss problems in electrical distribution networks'. In: *Scientific Reports* 13.1 (July 2023), p. 10777. ISSN: 2045-2322. DOI: 10.1038/s41598-023-37293-9.

[25] T. Wolgast and A. Nieße. 'Learning the optimal power flow: Environment design matters'. In: *Energy and AI* 17 (2024), p. 100410. DOI: 10.1016/j.egyai.2024.100410.

[26] E. Cibaku, F. Gama and S. Park. 'Boosting efficiency in state estimation of power systems by leveraging attention mechanism'. In: *Energy and AI* 16 (2024), p. 100369. DOI: 10.1016/j.egyai.2024.100369.

[27] S. Golestan, M. Habibi, S. Mousazadeh Mousavi, J. Guerrero and J. Vasquez. 'Quantum computation in power systems: An overview of recent advances'. In: *Energy Reports* 9 (2023). ISSN: 23524847. DOI: 10.1016/j.egyr.2022.11.185.

[28] G. K. Dziugaite and D. M. Roy. 'Computing Nonvacuous Generalization Bounds for Deep (Stochastic) Neural Networks with Many More Parameters than Training Data'. In: *arXiv preprint* (Mar. 2017). DOI: arXiv:1703.11008.

[29] F. Feng, Y. Zhou and P. Zhang. 'Quantum Power Flow'. In: *IEEE Transactions on Power Systems* 36 (2021). ISSN: 0885-8950. DOI: 10.1109/TPWRS.2021.3077382.

[30] F. Feng, Y.-F. Zhou and P. Zhang. 'Noise-resilient quantum power flow'. In: *iEnergy* 2.1 (Mar. 2023), pp. 63–70. ISSN: 2771-9197. DOI: 10.23919/IEN.2023.0008.

[31] P. Pareek, A. Jayakumar, C. Coffrin and S. Misra. *Demystifying Quantum Power Flow: Unveiling the Limits of Practical Quantum Advantage*. Tech. rep. 2024. DOI: arXiv:2402.08617v2.

[32] **Z. Kaseb**, M. Möller, G. T. Balducci, P. Palensky and P. P. Vergara. 'Quantum neural networks for power flow analysis'. In: *Electric Power Systems Research* 235 (2024), p. 110677. DOI: 10.1016/j.epsr.2024.110677.

[33] P. Halffmann, P. Holzer, K. Plociennik and M. Trebing. 'A Quantum Computing Approach for the Unit Commitment Problem'. In: 2023, pp. 113–120. DOI: 10.1007/978-3-031-24907-5{_}14.

[34] T. Morstyn. 'Annealing-Based Quantum Computing for Combinatorial Optimal Power Flow'. In: *IEEE Transactions on Smart Grid* 14 (2023). ISSN: 1949-3053. DOI: 10.1109/TSG.2022.3200590.

[35] C. C. McGeoch. *Adiabatic Quantum Computation and Quantum Annealing Theory and Practice*. Springer Cham, 2014.

[36] K. Beer, D. Bondarenko, T. Farrelly, T. J. Osborne, R. Salzmann, D. Scheiermann and R. Wolf. 'Training deep quantum neural networks'. In: *Nature Communications* 11 (2020). ISSN: 2041-1723. DOI: 10.1038/s41467-020-14454-2.

[37] C. Cao, J. Xue, N. Shannon and R. Joynt. 'Speedup of the quantum adiabatic algorithm using delocalization catalysis'. In: *arXiv preprint* 3.1 (Jan. 2021). ISSN: 26431564. DOI: arXiv:2007.11212.

[38] D. Beinert, C. Holzhüter, J. M. Thomas and S. Vogt. 'Power flow forecasts at transmission grid nodes using Graph Neural Networks'. In: *Energy and AI* 14 (2023), p. 100262. DOI: 10.1016/j.egyai.2023.100262.

[39] Z. Wang, J.-H. Menke, F. Schäfer, M. Braun and A. Scheidler. 'Approximating multi-purpose AC Optimal Power Flow with reinforcement trained Artificial Neural Network'. In: *Energy and AI* 7 (2022), p. 100133. DOI: 10.1016/j.egyai.2021.100133.

[40] R. Ahshan, M. S. Abid and M. Al-Abri. 'Geospatial Mapping of Large-Scale Electric Power Grids: A Residual Graph Convolutional Network-Based Approach with Attention Mechanism'. In: *Energy and AI* 20 (2025), p. 100486. DOI: 10.1016/j.egyai.2025.100486.

[41] H.-L. Huang, X.-Y. Xu, C. Guo, G. Tian, S.-J. Wei, X. Sun, W.-S. Bao and G.-L. Long. 'Near-term quantum computing techniques: Variational quantum algorithms, error mitigation, circuit compilation, benchmarking and classical simulation'. In: *Science China Physics, Mechanics & Astronomy* 66 (2023). ISSN: 1674-7348. DOI: 10.1007/s11433-022-2057-y.

[42] R. A. d. Oliveira and M. H. Bollen. 'Deep learning for power quality'. In: *Electric Power Systems Research* 214 (Jan. 2023), p. 108887. DOI: 10.1016/j.epsr.2022.108887.

[43] L. von Rueden, S. Mayer, K. Beckh, B. Georgiev, S. Giesselbach, R. Heese, B. Kirsch, J. Pfrommer, A. Pick, R. Ramamurthy, M. Walczak, J. Garcke, C. Bauckhage and J. Schuecker. 'Informed Machine Learning – A Taxonomy and Survey of Integrating Knowledge into Learning Systems'. In: (Mar. 2019). DOI: 10.1109/TKDE.2021.3079836.

[44] Z. Wu, M. Zhang, S. Gao, Z.-G. Wu and X. Guan. 'Physics-Informed Reinforcement Learning for Real-Time Optimal Power Flow with Renewable Energy Resources'. In: *IEEE Transactions on Sustainable Energy* (2024), pp. 1–11. DOI: 10.1109/TSTE.2024.3452489.

[45] Q. Yang, G. Wang, A. Sadeghi, G. B. Giannakis and J. Sun. 'Two-Timescale Voltage Control in Distribution Grids Using Deep Reinforcement Learning'. In: *IEEE Transactions on Smart Grid* 11.3 (2020), pp. 2313–2323. DOI: 10.1109/TSG.2019.2951769.

[46] X. Hu, H. Hu, S. Verma and Z.-L. Zhang. 'Physics-Guided Deep Neural Networks for Power Flow Analysis'. In: *IEEE Transactions on Power Systems* 36.3 (May 2021), pp. 2082–2092. DOI: 10.1109/TPWRS.2020.3029557.

[47] L. von Rueden, S. Mayer, K. Beckh, B. Georgiev, S. Giesselbach, R. Heese, B. Kirsch, M. Walczak, J. Pfrommer, A. Pick, R. Ramamurthy, J. Garcke, C. Bauckhage and J. Schuecker. 'Informed Machine Learning - A Taxonomy and Survey of Integrating Prior Knowledge into Learning Systems'. In: *IEEE Transactions on Knowledge and Data Engineering* (2021). ISSN: 1041-4347. DOI: 10.1109/TKDE.2021.3079836.

[48] M. Schuld, I. Sinayskiy and F. Petruccione. 'The quest for a Quantum Neural Network'. In: *Quantum Information Processing* 13 (2014). ISSN: 1570-0755. DOI: 10.1007/s11128-014-0809-8.

[49] K. Beer. 'Quantum neural networks'. In: *Institutionelles Repositorium der Leibniz Universitat Hannover* (2022). DOI: 10.15488/11896.

[50] R. Wagle, G. Tricarico, A. F. S. Melo, V. Rosero-Morillo, A. Shukla, F. Gonzalez-Longatt *et al.* 'Real-time cyber-physical power system testbed for optimal power flow study using co-simulation framework'. In: *IEEE Access* 12 (2024), pp. 150914–150929.

[51] R. Wagle, P. Sharma, C. Sharma, M. Amin, J. L. Rueda and F. Gonzalez-Longatt. 'Optimal power flow-based reactive power control in smart distribution network using real-time cyber-physical co-simulation framework'. In: *IET Generation, Transmission & Distribution* 17.20 (2023), pp. 4489–4502.

[52] S. Chanda, M. Mohanpurkar and R. Hovsapian. 'Architecture for Quantum-in-the Loop Real-Time Simulations for Designing Resilient Smart Grids'. In: *2023 13th International Symposium on Advanced Topics in Electrical Engineering (ATEE)*. IEEE. 2023, pp. 1–6.

# 2

# Background

*This PhD thesis is interdisciplinary, drawing on concepts from power system modeling and control, quantum computing (QC), and artificial intelligence (AI). To support readers from diverse backgrounds, this chapter introduces the essential foundations on which the remainder of the thesis is built. The material is intentionally concise, providing only the minimum knowledge necessary to understand the methods and contributions developed in the subsequent chapters.*

## 2.1. Power System Modeling and Control

In a power system comprising $N$ buses, the relationship between the complex bus voltages and complex bus currents is described by Ohm's Law in matrix form, as $\mathbf{I} - \mathbf{YV} = 0$, where $\mathbf{I}$ and $\mathbf{V}$ represent the complex bus current and voltage vectors, respectively, and $\mathbf{Y}$ denotes the $N \times N$ complex bus admittance matrix, given by:

$$Y_{ik} = G_{ik} + jB_{ik}, \quad i,k \in \{1,\dots,N\},$$

where $G_{ik}$ and $B_{ik}$ represent the conductance and susceptance between buses $i$ and $k$, respectively. We can further represent the net power as $\mathbf{S} = \mathbf{VI}^* = \mathbf{V} \circ (\mathbf{YV})^*$, where $\mathbf{S} = \mathbf{P} + j\mathbf{Q}$ and '$\circ$' denotes element-wise vector multiplication. We can also express the complex net power injected in bus $i$ ($S_i$) as the difference between the total generation power and the total demand power, as:

$$S_i = S_i^G - S_i^D, \quad i \in \{1,\dots,N\}, \tag{2.1}$$

where $S_i^G$ and $S_i^D$ represent the total power generated and total power demand at bus $i$, respectively. Further expanding (2.1) into its real and imaginary components gives:

$$P_i + jQ_i = (P_i^G - P_i^D) + j(Q_i^G - Q_i^D), \quad i \in \{1,\dots,N\}.$$

Here, $P_i$ and $Q_i$ are the net injections of active and reactive power; $P_i^G$ and $Q_i^G$ are the total generated power; and $P_i^D$ and $Q_i^D$ are the total power demand at bus $i$. We can express $P_i$ and $Q_i$ in terms of voltage and admittance parameters, as:

$$P_i + jQ_i = V_i \sum_{k=1}^{N} Y_{ik} V_k, \quad i,k \in \{1,\dots,N\}. \tag{2.2}$$

Further expanding (2.2) into real and imaginary components yields:

$$P_i = \sum_{j=1}^{N} G_{ij}(\mu_i\mu_j + \omega_i\omega_j) + B_{ij}(\omega_i\mu_j - \mu_i\omega_j), \tag{2.3a}$$

$$Q_i = \sum_{j=1}^{N} G_{ij}(\omega_i\mu_j - \mu_i\omega_j) - B_{ij}(\mu_i\mu_j + \omega_i\omega_j), \tag{2.3b}$$

where the complex voltage at bus $i$ is expressed as $V_i = \mu_i + j\omega_i$, with $\mu_i$ and $\omega_i$ respectively representing its real and imaginary parts.

Each bus $i$ in a power system is defined by four primary variables: $P_i$, $Q_i$, voltage magnitude ($V_i$), and voltage phase angle ($\delta_i$). Based on the variables specified or controlled at each bus, buses in a power system are divided into three main types: *PV* bus, *PQ* bus, and *slack* bus, see Table 2.1. The *slack* bus serves as a reference point for computing the power flow (PF) and optimal power flow (OPF).

Table 2.1: Types of buses in a power system

| | ***slack* bus** | ***PV* bus** | ***PQ* bus** |
|---|---|---|---|
| Known variables | $V_i, \delta_i$ | $P_i, V_i$ | $P_i, Q_i$ |
| Unknown variables | $P_i, Q_i$ | $Q_i, \delta_i$ | $V_i, \delta_i$ |
| Number of buses in a network | 1 | $N_G$ | $N - N_G - 1$ |

### 2.1.1. Power Flow Analysis

PF analysis aims to compute the unknown parameters for different bus categories in a power system that satisfy power balance equations, which can be compactly expressed as a root-finding problem:

$$F(x): \quad \mathbf{S} - \mathbf{V} \circ (\mathbf{YV})^* = 0, \tag{2.4}$$

where $x$ is the state variables, which include unknown parameters, e.g., the complex voltages at *PQ* buses. More specifically, the steady-state power balance equations can be expressed as:

$$P_i = P_i^{\mathrm{G}} - P_i^{\mathrm{D}}, \quad \forall i \in \{1, \ldots, N\}, \tag{2.5a}$$

$$Q_i = Q_i^{\mathrm{G}} - Q_i^{\mathrm{D}}, \quad \forall i \in \{1, \ldots, N\}, \tag{2.5b}$$

which is nonlinear, non-convex, and without an analytical solution. Therefore, PF analysis is generally performed using iterative numerical methods, such as the Newton-Raphson (NR) and Gauss-Seidel methods. These methods start with initial values for unknown parameters $x^{(0)}$ at it $= 0$. The iterative procedure to update $x^{(\mathrm{it}+1)}$ can be written as:

$$x^{(\mathrm{it}+1)} = x^{(\mathrm{it})} + w^{(\mathrm{it})} \Delta x, \tag{2.6}$$

where $w^{(\mathrm{it})}$ is typically a relaxation factor that controls how much of the computed update $\Delta x$ is applied at each iteration.

NR is one of the most widely used iterative methods for PF analysis due to its quadratic convergence properties and robustness in well-conditioned cases. The method is based on a first-order Taylor series expansion of the PF equations around an operating point. The update step $\Delta x$ is computed, as:

$$\Delta x = -J^{-1}(x^{(\mathrm{it})}) F(x^{(\mathrm{it})}),$$

where $J(x) = \partial F(x)/\partial x$ is the Jacobian matrix evaluated at the current iteration $x^{(\mathrm{it})}$. At each iteration, $J(x)$ is computed and used to solve for $\Delta x$, which is then applied to update $x$, as calculated in (2.6). The process continues until convergence criteria, such as $||F(x)|| < \epsilon$, are met, where $\epsilon$ is a predefined tolerance.

### 2.1.2. Optimal Power Flow

The OPF problem seeks to determine the generator decision variables $P_i^G, Q_i^G$ such that the resulting system state variables, including bus voltages $\{V_i, \delta_i\}$ as well

as line currents, remain within prescribed operational limits, while satisfying the network power balance equations. The problem is formulated as:

$$\min \sum_{k \in \mathbb{G}}^{N_G} f_k\left(P_k^G\right), \tag{2.7}$$

subject to

$$P_i = P_i^G - P_i^D, \tag{2.8a}$$
$$Q_i = Q_i^G - Q_i^D, \tag{2.8b}$$
$$\underline{P_i^G} \leq P_i^G \leq \overline{P_i^G}, \tag{2.8c}$$
$$\underline{Q_i^G} \leq Q_i^G \leq \overline{Q_i^G}, \tag{2.8d}$$
$$\underline{V_i} \leq V_i \leq \overline{V_i}, \tag{2.8e}$$
$$\underline{\delta_i} \leq \delta_i \leq \overline{\delta_i}, \tag{2.8f}$$

where $f_k(.)$ is the generator's fuel cost function; $k \in \mathbb{G}$ a subset of buses connected to generators {1, 2, ..., $N_G$}; $N_G$ is the total number of generators; $\underline{P_i^G}$ and $\overline{P_i^G}$ are the minimum and maximum generated active power outputs at bus $i$; $\underline{Q_i^G}$ and $\overline{Q_i^G}$ are the minimum and maximum generated reactive power outputs at bus $i$; $\underline{V_i}$ and $\overline{V_i}$ are the minimum and maximum voltage magnitudes at bus $i$; $\underline{\delta_i}$ and $\overline{\delta_i}$ are the minimum and maximum voltage phase angles at bus $i$.

## 2.2. Quantum Computing

Quantum computing is an emerging computational paradigm that exploits the principles of quantum mechanics to process information in fundamentally different ways from classical digital computers. At the core of a quantum computer lies the *quantum bit*, or *qubit*, which serves as the quantum analogue of a classical bit. Whereas a classical bit can only take the definite value 0 or 1, a qubit can exist in a continuum of states represented by a linear combination of the computational basis states $|0\rangle$ and $|1\rangle$. A general single-qubit state is written as:

$$|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle,$$

where $\alpha_0, \alpha_1 \in \mathbb{C}$ are the *probability amplitudes* satisfying the normalization condition $|\alpha_0|^2 + |\alpha_1|^2 = 1$. Upon measurement in the computational basis, the qubit collapses to $|0\rangle$ with probability $|\alpha_0|^2$ and to $|1\rangle$ with probability $|\alpha_1|^2$. Unlike classical bits, the state of a qubit prior to measurement cannot be fully observed, as the amplitudes themselves are not directly measurable.

From a mathematical perspective, the states $|0\rangle$ and $|1\rangle$ can be represented as orthonormal vectors in a $2^1$-dimensional Hilbert space, as:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Therefore,

$$|\psi\rangle = \alpha_0 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \alpha_1 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha_0 \\ \alpha_1 \end{bmatrix}.$$

By extending the basis to $n$ qubits, the full state space becomes a $2^n$-dimensional complex vector space, as:

$$|\Psi\rangle = \sum_{b \in \{0,1\}^n} a_b \, |b\rangle = \sum_{k=0}^{2^n-1} a_k \, |k\rangle \,,$$

where the first expression uses binary strings to label the basis states, and the second expresses the same states using their corresponding natural number indices. Each $|k\rangle$ denotes an $n$-qubit basis vector in the $2^n$-dimensional Hilbert space. The complex amplitudes $a_k \in \mathbb{C}$ satisfy the normalization condition $\sum_k |a_k|^2 = 1$. This exponential scaling of the state space is one of the key reasons quantum computers can, in principle, outperform classical computers on certain computational problems [1].

Quantum information is manipulated using *quantum gates*, which are reversible linear transformations described mathematically by unitary matrices. A matrix $U \in \mathbb{C}^{2^n \times 2^n}$ is unitary if $U^\dagger U = I$, where $U^\dagger$ is the conjugate transpose of $U$. The unitarity constraint preserves normalization and ensures that quantum evolution is reversible. A direct implication of this reversibility is the no-cloning theorem, which states that an arbitrary quantum state cannot be duplicated [1]. This property has no classical analogue and is central to the security and behavior of quantum information processing.

Two properties that fundamentally distinguish quantum computation from classical computation are *superposition* and *entanglement*. Superposition allows a qubit or a collection of qubits to represent multiple states simultaneously. For example, the equally weighted state

$$|\Psi\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} |k\rangle$$

is a superposition of $N$ computational basis states. When a quantum operation $U$ is applied to this state, it acts on each component of the superposition,

$$U \, |\Psi\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} U \, |k\rangle \,.$$

This property, often referred to as *quantum parallelism*, enables certain quantum algorithms to process an exponentially large number of basis states simultaneously

by applying an operation to multiple basis states at once, although the measurement constraint limits direct access to all amplitudes.

Entanglement, on the other hand, refers to quantum correlations that have no classical analogue. Consider the well-known Bell state,

$$|\Psi\rangle = \frac{1}{\sqrt{2}}\left(|00\rangle + |11\rangle\right).$$

When this state is measured in the computational basis, i.e., when it is forced to collapse to one of the four possible basis states $|00\rangle$, $|01\rangle$, $|10\rangle$, or $|11\rangle$—the outcomes $|00\rangle$ and $|11\rangle$ each occur with probability $\left(\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2}$. The states $|01\rangle$ and $|10\rangle$ never appear. Importantly, measuring the first qubit immediately determines the state of the second qubit, regardless of the distance between them. This property reflects an inseparable correlation built into entangled quantum states.

*Measurement* is the way to retrieve information from a quantum computation. Since probability amplitudes cannot be accessed directly, one must infer them by repeatedly allowing the quantum state to collapse into a classical binary state and measuring it to reconstruct a probability distribution over all possible outcomes. For an $n$-qubit state $|\Psi\rangle$, the probability of observing the basis state $|i\rangle$ upon measurement is given by:

$$p_{|\Psi\rangle}(|i\rangle) = |\langle i|\Psi\rangle|^2 = |a_i|^2.$$

Once a measurement is performed and outcome $|i\rangle$ is obtained, the quantum state collapses to $|i\rangle$, i.e.,

$$|\Psi\rangle = \frac{a_i}{||a_i||^2}|i\rangle.$$

This collapse is irreversible and eliminates the superposition present before measurement, meaning that quantum computations must be designed carefully to extract useful information from the final measurement statistics.

Figure 2.1 illustrates a parameterized quantum circuit (PQC) that involves a pair of qubits interconnected with three quantum gates. The first qubit undergoes a Hadamard gate $H$ and the second qubit experiences a rotation around the y-axis $R_y(w^r)$. A controlled NOT gate ($CNOT$) entangles the second qubit with the first. The probabilistic expected value of the measurement, i.e., the expectation value of the probability distribution of the final quantum state, is then determined. The expectation value is represented by $\langle\psi|\psi\rangle$, which is subsequently subjected to post-processing to form the approximations.

QC models can be broadly categorized into two principal paradigms: (i) the *gate-based quantum computing* (GQC) model and (ii) the *adiabatic quantum computing* (AQC) model. Both models are computationally universal under idealized conditions, yet they differ substantially in how quantum states are manipulated [2]. Notice that current quantum technologies remain constrained by limited qubit counts, restricted connectivity, and susceptibility to noise, placing most practical devices in the noisy intermediate-scale quantum (NISQ) regime.

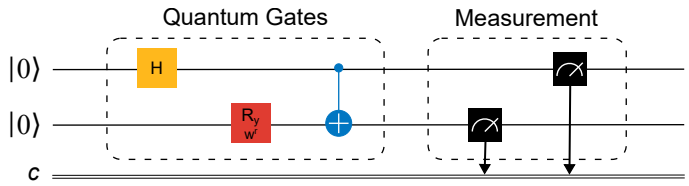



Figure 2.1: Schematic representation of a PQC involving a pair of qubits. It includes both a parameterized gate $R_y(w^r)$ with adjustable parameter $w^r$, and a non-parameterized gate $H$. The $CNOT$ gate entangles the second qubit with the first, executing a two-qubit operation. Finally, the measurement is performed to determine the expectation values of the involved qubits.

### 2.2.1. Gate-Based Quantum Computing

In the GQC model, computation is organized into *layers* or *depth*, where deeper circuits are generally more expressive but more vulnerable to hardware-induced errors (e.g., [3, 4]).

Despite its theoretical generality, executing deep quantum circuits on contemporary hardware poses practical challenges. NISQ processors are affected by decoherence, imperfect gates, crosstalk, and limited qubit connectivity, all of which constrain attainable circuit depth. As a result, many GQC studies rely on classical simulators or focus on small-scale demonstrations [5]. Experimental platforms include superconducting qubits (as used by IBM[1], Google[2], and Rigetti[3]), trapped-ion systems (IonQ[4], Quantinuum[5]), photonic architectures (PsiQuantum[6], Xanadu[7]), and neutral-atom platforms (QuEra[8]), each offering distinct trade-offs in terms of coherence time, gate fidelity, and connectivity.

### 2.2.2. Adiabatic Quantum Computing

AQC provides an alternative computation model in which the quantum state evolves *continuously* under a time-dependent Hamiltonian, rather than through discrete gate operations [2, 6]. The computation begins in the known ground state of an initial Hamiltonian and, if the evolution is sufficiently slow, the adiabatic theorem guarantees that the system remains in its instantaneous ground state, ultimately reaching the ground state of a problem Hamiltonian that encodes an optimization objective [2, 7].

Because AQC does not require deep gate sequences, it is more tolerant to certain forms of noise and therefore well-suited to the NISQ era [8]. Quantum annealing is a

[1] https://www.ibm.com/quantum
[2] https://quantumai.google
[3] https://www.rigetti.com
[4] https://ionq.com
[5] https://www.quantinuum.com
[6] https://psiquantum.com
[7] https://www.xanadu.ai
[8] https://quera.com

practical realization of AQC, targeting discrete optimization problems formulated in Ising or quadratic unconstrained binary optimization (QUBO) form. Quantum tunneling enables escaping local minima more effectively than classical simulated annealing, which can be advantageous for complex or rugged energy landscapes [9, 10].

The most prominent QA hardware platform is D-Wave's Advantage™ system[9], which contains thousands of qubits arranged in a structured but sparse connectivity graph. These quantum processing units operate at millikelvin temperatures to preserve coherence and enable tunneling dynamics. In this architecture, positive couplings $J_{ij} > 0$ favor opposite spin orientations, while negative couplings $J_{ij} < 0$ encourage alignment; the local field $h_i$ biases each spin toward $+1$ or $-1$ accordingly.

Alongside quantum annealers, quantum-inspired classical accelerators provide complementary capabilities. Fujitsu's Digital Annealer is a CMOS-based architecture that emulates annealing dynamics at room temperature, supports fully connected variable interactions, and scales to tens of thousands of binary variables with high numerical precision. Its recent evolution, the Quantum-Inspired Integrated Optimization (QIIO) platform[10], integrates advanced sampling strategies such as parallel tempering to improve exploration and avoid local minima [11].

## 2.3. Artificial Intelligence

AI includes computational techniques that enable machines to perform tasks that typically require human intelligence. Within AI, *machine learning* (ML) is a subfield focused on automatically inferring predictive or descriptive models from data, including paradigms such as supervised, unsupervised, and semi-supervised learning, as well as representation learning. *Deep learning* (DL) is a specialized branch of ML that employs multilayer neural networks (NNs) for hierarchical feature extraction and nonlinear function approximation. *Reinforcement learning* (RL) [12] is a subfield of ML that enables an agent to learn through trial and error and make sequential decisions by interacting with an environment.

### 2.3.1. Deep Learning

A feed-forward NN approximates output labels $\mathbf{y}$ given input features $\mathbf{x}$. In the context of the PF problem, $\mathbf{y} \in \{(\mathbf{V}_i, \delta_i) : i = 1, 2, \ldots, n\}$ corresponds to complex voltages while $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ corresponds to the net active and reactive power, see Figure 2.2. That is, the input and output layers consist of $n \times 2$ neurons each, where $n$ is the number of buses. The architecture employs a chain of functions $f(\mathbf{x}) = l_k \circ \ldots \circ l_1(\mathbf{x})$, sequentially processing $\mathbf{x}$ through $k$ hidden layers to obtain $\mathbf{f}(\cdot) \in \{(\hat{\mathbf{V}}_i, \hat{\delta}_i) : i = 1, 2, \ldots, n\}$. Here, $l_k(\mathbf{x}) = \sigma(W_k^T \cdot \mathbf{x} + b_k)$ is the $k$-th hidden layer. $W_k^T$ and $b_k$ are the weight matrix and bias vector for the

[9] https://www.dwavesys.com
[10] www.fujitsu.com/global/services/business-services/digital-annealer

respective hidden layer. Each hidden layer applies a linear transformation, i.e., $W_k^T \cdot \mathbf{x} + b_k$, followed by a nonlinear transformation, i.e., $\sigma(\cdot)$, to capture complex relationships between $\mathbf{x}$ and $\mathbf{y}$. $W_k$ and $b_k$ are trainable parameters optimized during the training process to minimize the difference between $\mathbf{f}(\cdot)$, approximated by the NN, and ground-truth output labels $\mathbf{y}$, obtained from the NR, based on a loss function of choice, e.g., the mean squared error (MSE):

$$\mathscr{L} = \frac{1}{N}\sum_{j=1}^{N}(\mathbf{y}_j - f(\mathbf{x}_j))^2, \tag{2.9}$$

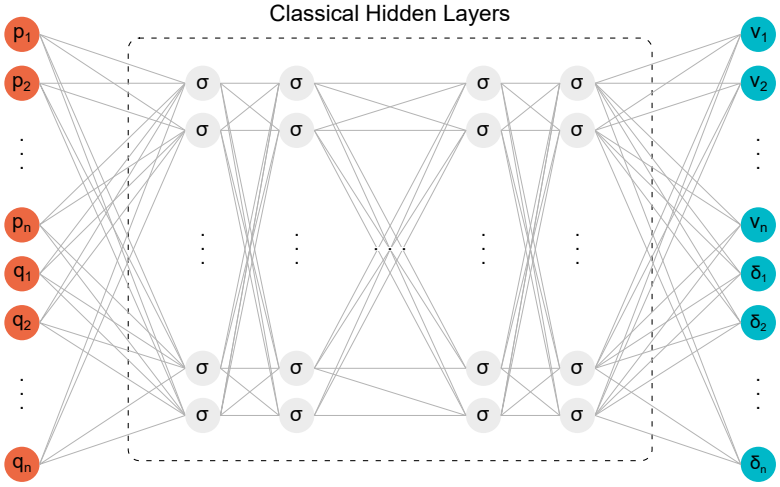



Figure 2.2: Schematic depiction of the NN designed for PF analysis. $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ and $\mathbf{y} \in \{(\mathbf{V}_i, \delta_i) : i = 1, 2, \ldots, n\}$ respectively represent the input features and output labels, where $n$ is the number of buses.

where $j$ is the index of the training data point, $N$ is the total number of training data points, $\mathbf{x}_j$ and $\mathbf{y}_j$ are the vector of input features and output labels obtained from the NR for the $j$-th training data point. $\mathbf{f}(.)$ is the vector of approximated output labels obtained from the NN.

### 2.3.2. Reinforcement Learning

Unlike supervised learning, which relies on labeled examples, RL learns through trial-and-error interaction, using feedback in the form of rewards to improve decision-making over time. RL problems are commonly formalized as Markov decision processes (MDPs), which define the dynamics, actions, and reward structure of the environment. Building on this foundation, modern RL algorithms such as Proximal Policy Optimization (PPO) provide scalable and robust tools for solving complex control and optimization tasks.

### Markov Decision Process (MDP)

MDP is the typical mathematical formulation for a standard RL framework, defined by the tuple [12]:

$$\mathcal{M} = \langle S, A, R, P, \gamma, d_0 \rangle,$$

where

- $S$ is the set of states representing all possible configurations of the environment,
- $A$ is the set of actions available to the agent,
- $R : S \times A \times S \to \mathbb{R}$ is the reward function that provides scalar feedback for taking action $a$ in state $s$,
- $P : S \times A \times S \to [0, 1]$ is the transition dynamics of the environment, i.e., for taking action $a$ in state $s$, $P(s' \mid s, a)$ gives the probability of transitioning to a specific next state $s'$. Collectively, $\{P(s' \mid s, a)\}_{s' \in S}$ forms a probability distribution over all possible next states, with $\sum_{s' \in S} P(s' \mid s, a) = 1$.
- $\gamma \in (0, 1)$ is the discount factor that balances the trade-off between rewards of immediate and future timesteps,
- $d_0 \in S$ represents the initial state distribution.

In an MDP, the transition dynamics distribution $P$ is Markovian, meaning that the next state given an action only depends on the current state. At each timestep during training, the RL agent observes the state of the environment, performs an action, and the environment transitions into the next state. The RL agent then receives a reward for taking that action, which guides learning to optimize a long-term objective that accelerates NR convergence. A schematic RL procedure is visualized in Figure 2.3.

Through interaction with the environment on a diverse set of system conditions, the RL agent learns a decision-making strategy, called a policy $\pi : S \to A, s \mapsto \pi(a|s)$, which defines a probability distribution over actions given a state.

An optimal policy generalizes well to unseen scenarios, thus enhancing the efficiency of PF analysis even in complex or highly stressed grids. The main objective is to find such a policy that maximizes the expected cumulative discounted reward over time:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[ \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right], \tag{2.10}$$

where $\tau = \{s_0, a_0, \ldots s_t, a_t, \ldots\}$ $\forall s \in S, a \in A$ is defined as a trajectory sampled from policy $\pi$. Note that throughout this text, a conventional notation for the reward at timestep $t$ is used, denoted as $r_t$, which is equivalent to $R(s_t, a_t)$ in (2.10). The optimal policy can then be found by maximizing the objective over the entire set of policies:

$$\pi^* = \arg\max_{\pi} J(\pi).$$

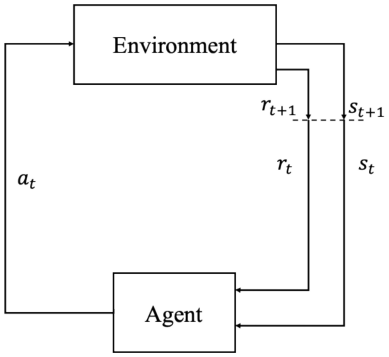



Figure 2.3: A visualized representation of an RL procedure. At each timestep $t$, the agent observes state $s_t$ and performs action $a_t$ in the environment. The environment then transitions to the next state $s_{t+1}$ according to the transition dynamics $P$ and receives the reward $r_{t+1}$.

To achieve an optimal policy, one can define state-value and state-action-value functions for a policy $\pi$, $V_\pi(s)$ and $Q_\pi(s, a)$, which estimate the expected future reward starting from a given state $s$ or state-action pair $(s, a)$, respectively:

$$V_\pi(s) = \mathbb{E}_{a\sim\pi, s'\sim P}\left[R(s, a) + \gamma V_\pi(s')\right], \tag{2.11}$$

$$Q_\pi(s, a) = \mathbb{E}_{s'\sim P}\left[R(s, a) + \gamma \mathbb{E}_{a'\sim\pi} Q_\pi(s', a')\right]. \tag{2.12}$$

(2.11) and (2.12) are referred to as the *Bellman Equations* for $V_\pi$ and $Q_\pi$, respectively. However, these equations assume that the transition dynamics $P$ are known, which oftentimes is not the case in RL. Therefore, most RL methods rely on an agent collecting experiences, i.e., trajectories $\tau \sim \pi$, by interacting with the environment.

Leveraging these experiences, the value functions $V_\pi$ and $Q_\pi$ can be learned with *value-based* methods. Alternatively, a policy can be learned directly using *policy-based* methods. Current state-of-the-art RL algorithms learn both the value function and a policy, a method referred to as *actor-critic* [13].

### Proximal Policy Optimization (PPO)

PPO is a widely used actor-critic algorithm in modern deep RL that aims to improve learning stability and efficiency by constraining policy updates while maximizing cumulative rewards [14]. Here, deep RL refers to approaches that use neural networks parametrized by a set of weights to approximate value functions or policies. PPO combines the strengths of policy-gradient methods with clipped surrogate objectives to prevent significant, destabilizing updates, thus offering a practical balance between exploration and exploitation. The PPO model consists of the following components:

**Policy Network** is a parameterized policy $\pi_\phi$, represented by a NN, where $\phi$ refers to the NN parameters. The policy network defines a probability distribution over actions given a state and is mathematically given as:

$$\text{Policy} = \pi_\phi(a_t|s_t),$$

where $s_t$ represents the state at timestep $t$, and the policy $\pi_\phi$ determines the action $a_t$ to be taken at that state.

In the context of PF analysis, the policy network takes the current state $s_t = [\mathbf{V}^t, \theta^t, k_t]$ as input and outputs either deterministic actions or parameters of a probability distribution (e.g., mean and standard deviation for Gaussian policies in continuous action spaces). This structure enables efficient handling of high-dimensional, continuous control tasks.

During training, PPO updates the policy network using stochastic gradient ascent on a clipped surrogate objective, preventing policy updates from being too large. The direction and magnitude of each update are guided by an advantage function that quantifies the improvement of an action relative to the average action under the current policy.

**Value Network** estimates the expected return from a given state under the current policy. It provides a baseline for computing the advantage function. In mathematical terms, the value function $V^\pi_{\phi'}(s)$ is defined as:

$$V^\pi_{\phi'}(s) = \mathbb{E}_\pi \left[ \sum_{t=0}^{T} \gamma^t r_t \mid s_0 = s \right],$$

where $\phi'$ are the parameters of the parametrized NN that approximates the value function and $T$ is the episode length.

This network is also typically implemented as a deep NN and trained using a value loss, such as mean squared error between predicted and actual returns. The value network plays a crucial role in reducing variance in policy updates, improving training stability, and facilitating better long-term decision-making.

**Learning Procedure** refers to the iterative interactions with the environment, as outlined in Algorithm 1, through which the PPO agent improves its policy. The policy and value networks are trained jointly using mini-batch gradient updates over trajectories collected during agent-environment interaction.

The PPO loss function includes both the clipped surrogate objective for the policy and a value loss term, optionally regularized by an entropy bonus to encourage exploration. This dual-network architecture enables the agent to both evaluate the quality of visited states (via the value network) and refine its behavior over time (via the policy network), making PPO a powerful framework for continuous control tasks such as PF initialization.

# References

[1] M. A. Nielsen and I. L. Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.

**Algorithm 1** Learning procedure of the RL agent at each timestep.

**Initialize** $\pi_\phi$ and $V^\pi_{\phi'}$
**while** training **do**
    Collect experience in the environment using $\pi_\phi$
    Compute advantage estimates $A_t$
    Update $\pi_\phi$
    Update $V^\pi_{\phi'}$
**end while**

[2] C. C. McGeoch. *Adiabatic Quantum Computation and Quantum Annealing Theory and Practice*. Springer Cham, 2014.

[3] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin and H. Weinfurter. 'Elementary gates for quantum computation'. In: *Physical Review A* 52 (5 Nov. 1995), pp. 3457–3467. ISSN: 1050-2947. DOI: 10.1103/PhysRevA.52.3457.

[4] A. W. Harrow, A. Hassidim and S. Lloyd. 'Quantum Algorithm for Linear Systems of Equations'. In: *Physical Review Letters* 103 (15 Oct. 2009), p. 150502. ISSN: 0031-9007. DOI: 10.1103/PhysRevLett.103.150502.

[5] J. Preskill. 'Quantum Computing in the NISQ era and beyond'. In: *Quantum* 2 (Aug. 2018), p. 79. ISSN: 2521-327X. DOI: 10.22331/q-2018-08-06-79.

[6] K. Beer, D. Bondarenko, T. Farrelly, T. J. Osborne, R. Salzmann, D. Scheiermann and R. Wolf. 'Training deep quantum neural networks'. In: *Nature Communications* 11 (2020). ISSN: 2041-1723. DOI: 10.1038/s41467-020-14454-2.

[7] P. I. Bunyk, E. M. Hoskinson, M. W. Johnson, E. Tolkacheva, F. Altomare, A. J. Berkley, R. Harris, J. P. Hilton, T. Lanting, A. J. Przybysz and J. Whittaker. 'Architectural Considerations in the Design of a Superconducting Quantum Annealing Processor'. In: *IEEE Transactions on Applied Superconductivity* 24 (4 Aug. 2014), pp. 1–10. DOI: 10.1109/TASC.2014.2318294.

[8] C. Cao, J. Xue, N. Shannon and R. Joynt. 'Speedup of the quantum adiabatic algorithm using delocalization catalysis'. In: *arXiv preprint* 3.1 (Jan. 2021). ISSN: 26431564. DOI: arXiv:2007.11212.

[9] H. Goto, K. Tatsumura and A. R. Dixon. 'Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems'. In: *Science Advances* 5 (4 Apr. 2019). DOI: 10.1126/sciadv.aav2372.

[10] A. Lucas. 'Ising formulations of many NP problems'. In: *Frontiers in Physics* 2 (2014). ISSN: 2296-424X. DOI: 10.3389/fphy.2014.00005.

[11] F. Yin, H. Tamura, Y. Furue, M. Konoshima, K. Kanda and Y. Watanabe. 'Extended Ising Machine with Additional Non-quadratic Cost Functions'. In: *Journal of the Physical Society of Japan* 92 (3 Mar. 2023). DOI: 10.7566/JPSJ.92.034002.

[12] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018.

[13] V. Konda and J. Tsitsiklis. 'Actor-Critic Algorithms'. In: *Advances in Neural Information Processing Systems*. Ed. by S. Solla, T. Leen and K. Müller. Vol. 12. MIT Press, 1999.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov. 'Proximal Policy Optimization Algorithms'. In: *arXiv preprint* (2017). DOI: arXiv:1707.06347.

# 3

# Physics-Informed Neural Networks for Power Flow

*This chapter introduces a physics-informed neural network for power flow (PINN4PF) that effectively captures the nonlinear dynamics of large-scale modern power systems. The proposed neural network (NN) architecture consists of two important advancements in the training pipeline: (i) a double-head feed-forward NN that aligns with power flow (PF) analysis, including an activation function that adjusts to the net active and reactive power injections patterns, and (ii) a physics-based loss function that partially incorporates power system topology information through a novel hidden function. The reader is referred to Sections 2.1 and 2.3.1 for background on PF and deep learning, upon which this chapter builds.*

Parts of this chapter have been published in International Journal of Electrical Power & Energy Systems **174** (2026) with the title "Physics-informed neural network with adaptive activation for power flow", doi: 10.1016/j.ijepes.2025.111525 [1].

## 3.1. Motivation

Deep learning approaches, and more specifically, artificial neural networks (NNs), are currently the most powerful set of numerical tools for providing accurate approximations of nonlinear problems (e.g., [2–4]). Several studies have demonstrated the superiority of deep learning approaches in power flow (PF) analysis in terms of computational time by orders of magnitude (e.g., [5–7]). At the same time, the accuracy of the solutions is competitive compared to the conventional iterative numerical methods (e.g., [8, 9]). NNs, therefore, can address the challenges mentioned above by leveraging the massive measurements and/or augmented data, learning complex input-output relationships that are often difficult or even impossible for conventional iterative numerical methods to comprehend, and achieving the accuracy required for real-world applications [10]. Nevertheless, NNs are subject to overfitting, the lack of generalization, and scalability issues. They are very unlikely to meet the physical constraints. Also, their performance relies heavily on the training dataset size and quality while not enough data is always available due to privacy reasons and the presence of missing data, among others [11].

Several studies in the literature have investigated the impact of various modifications on the efficacy of PF using deep learning approaches (e.g., [12, 13]). These modifications are categorized into three main stages: (i) pre-training (e.g., [14]); (ii) training (e.g., [15]); and (iii) post-training (e.g., [16]). A majority of these studies have primarily focused on the training stage. There are also instances where topology information, such as line physical properties (e.g., [17]) and graph topology (e.g., [18]), has been integrated into the training stage. The literature review also indicates the utilization of different prior knowledge for the PF and optimal power flow (OPF) problems, including equality and inequality constraints (e.g., [19]) and PF equations (e.g., [20]).

Measurement data has been used in [14] to enhance OPF using deep learning approaches. While this modification has demonstrated improvements in accuracy, its applicability is confined to the pre-training stage. Physical properties have been used on a few occasions in the past to enhance PF using deep learning approaches (e.g., [17, 21]), where the focus was mainly on the training stage. However, the high dependency on the physical properties of power systems affects the efficiency of this modification, certainly because physical properties are not always reliable due to different reasons, including aging and environmental conditions. In addition, integrating all the physical properties makes training processes computationally very expensive. The completion idea and filtering technique have also been employed in a few studies (e.g., [22]), where the focus is mainly on the post-training stage. Yet, similar to pre-training stage modifications, this approach has limitations as it is detached from the training stage.

Among various methods, modified loss functions have been widely used for PF (e.g., [23]) and OPF (e.g., [24]) and have shown significant promise in enhancing the performance and reliability of NNs. However, most existing contributions (e.g., [25]) rely on the full integration of system topology data and physical parameters into the learning process. Although this strategy improves physical consistency, it significantly increases computational complexity, thereby limiting scalability. In

addition, it assumes complete and accurate system information, which is often unrealistic in practice due to measurement noise and topology uncertainties. Consequently, developing a physics-based loss function that retains the benefits of physical consistency while reducing dependency on full system topology data and computational overhead remains an open research challenge in the field.

With this motivation, we introduce a novel hidden function derived from the power balance equations, which partially encodes line characteristics into a modified loss function as prior physical knowledge. In contrast to existing approaches that require full system topology data and complete line parameters (e.g., graph neural networks), the proposed approach reduces dependency on such information while preserving physical interpretability. Building upon this concept, we propose an end-to-end deep learning architecture, hereafter called physics-informed neural network for power flow (PINN4PF), which achieves up to two orders of magnitude improvement in performance compared to conventional NNs. This chapter focuses on the training stage and extends the existing body of knowledge on physics-informed NNs for PF (e.g., [15, 23]). PINN4PF learns the mapping from net active and reactive power injections to complex bus voltages. An adaptive activation function is also employed to enhance the representational flexibility of the NN compared to conventional static activation functions.

## 3.2. Proposed architecture: PINN4PF

The proposed physics-informed neural network for power flow (PINN4PF) includes two key modeling innovations in the *forward* and *loss* steps, respectively, denoted as A and B in Figure 3.1. These components together form a double-head architecture enhanced with an adaptive activation function and a physics-based loss function. Note that classical PF solvers, e.g., the Newton-Raphson (NR) solver, and PINN4PF can be interchangeably used to specify the state of the power system. The following subsections provide a detailed description of the scientific modeling and mathematical formulation of PINN4PF.

### 3.2.1. Forward Pass

A double-head feed-forward NN $f(\cdot) \in \{(f_0, f_1, f_2)\}$ is developed to approximate $\hat{\mathbf{y}} = \{(\hat{\mu}_i, \hat{\omega}_i) : i = 1, 2, \ldots, n\}$ at load buses using the dataset $\{\mathbf{x}, \mathbf{y}\}$. It has three types of layers, i.e., input, hidden, and output layers, as highlighted in Figure 3.1-A. The input and output layers correspond to the input features $\mathbf{x}$ and output labels $\mathbf{y}$, respectively. The neurons of the input layer contain active power $\mathbf{P}^D = [P_1, P_2, \ldots, P_n]$ followed by reactive power $\mathbf{Q}^D = [Q_1, Q_2, \ldots, Q_n]$ at all load buses, and hence, the input layer has $n \times 2$ neurons, where $n$ is the number of load buses. Following the input layer, there is a set of shared hidden layers $f_0(\cdot)$ acting as a feature extractor that projects $\mathbf{x}$ to a higher-dimensional space, where the two heads, $f_1(\cdot)$ and $f_2(\cdot)$, separately involve a few hidden layers to respectively approximate $\mu = [\mu_1, \mu_2, \ldots, \mu_n]$ and $\omega = [\omega_1, \omega_2, \ldots, \omega_n]$. Thus, the output layer of each head has $n$ neurons. The shared set of hidden layers and the two

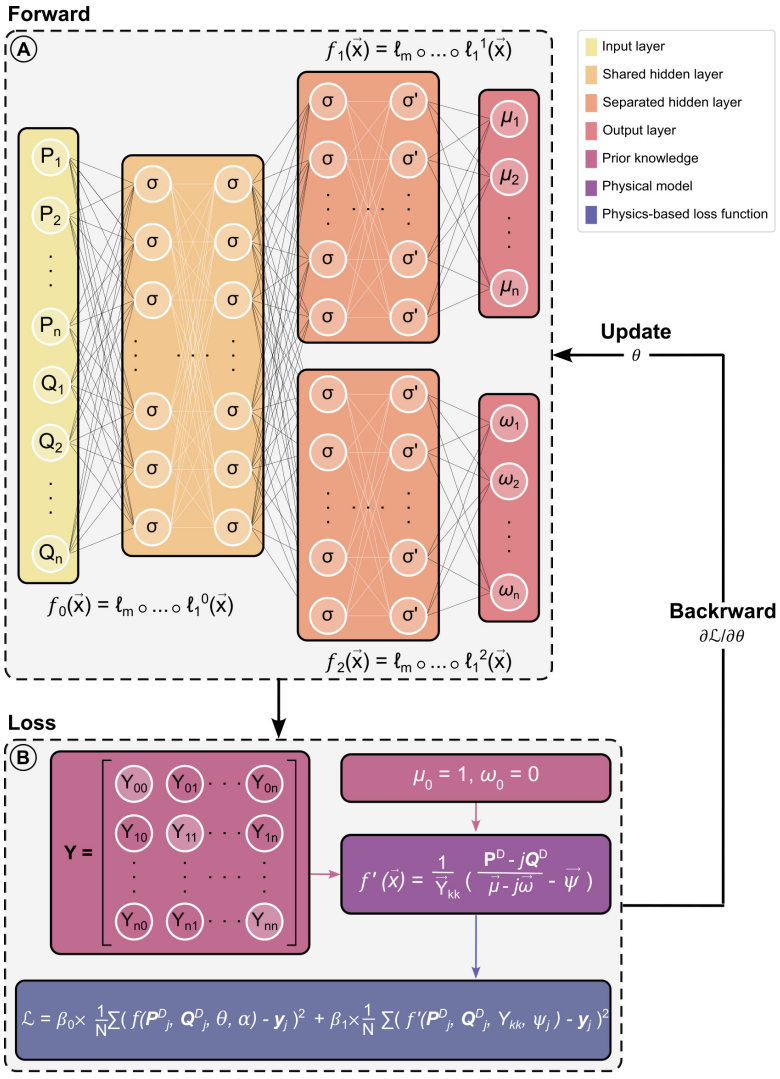



Figure 3.1: Schematic diagram of the proposed PINN4PF architecture incorporating two key advancements: (A) a double-head feed-forward NN, $f(\cdot) \in \{f_0, f_1, f_2\}$, utilizing an adaptive activation function $\sigma'(z) = \max(0, \alpha z)$ with a trainable slope $\alpha$; and (B) a physics-based loss function that combines supervised learning with physical constraints. The input layer receives a concatenated vector of $\mathbf{P}^D$ and $\mathbf{Q}^D$, resulting in $2n$ input neurons. Two separate network heads, $f_1(\cdot)$ and $f_2(\cdot)$, predict the $\mu$ and $\omega$, each with $n$ output neurons.

heads each are a chain of functions and can be represented as:

$$f_0(\mathbf{x}) = l_m^0 \circ \ldots \circ l_1^0(\mathbf{x}), \tag{3.1a}$$

$$f_1(\mathbf{x}) = l_m^1 \circ \ldots \circ l_1^1(\mathbf{x}), \tag{3.1b}$$

$$f_2(\mathbf{x}) = l_m^2 \circ \ldots \circ l_1^2(\mathbf{x}), \tag{3.1c}$$

which sequentially process $\mathbf{x}$ through $m$ hidden layers to obtain $\mathbf{f}_0(\cdot)$, $\mathbf{f}_1(\cdot)$, and $\mathbf{f}_2(\cdot)$, respectively. For the shared set of hidden layers and the two heads, the $k$-th hidden layer ($k = 1, 2, \ldots, m$) is given by:

$$l_k(\mathbf{x}) = \sigma\left(W_k^T \cdot \mathbf{x} + b_k\right). \tag{3.2}$$

Here, $W_k^T$ and $b_k$ are the weight matrix and bias vector for the corresponding hidden layer. Each hidden layer applies a linear transformation $W_k^T \cdot \mathbf{x} + b_k$, followed by a nonlinear transformation $\sigma(\cdot)$ to capture complex relationships between $\mathbf{x}$ and $\mathbf{y}$. Note that $W_k$ and $b_k$ are trainable parameters and are optimized during the training process.

The Rectified Linear Unit (ReLU) activation function $\sigma(\cdot)$ is applied to the shared set of hidden layers (3.1a). While an adaptive version of ReLU $\sigma'(\cdot)$ is developed to apply model-based nonlinear transformations to the two heads (3.1b) and (3.1c). That is, a trainable parameter $\alpha$ is introduced to scale the result of the linear transformation while applying nonlinearity. Hence, $\alpha$ serves as a guide to prevent overfitting and improve the generalization ability of the NN against unseen data. $\sigma(\cdot)$ and $\sigma'(\cdot)$ are represented as:

$$\sigma = \max(0, z), \tag{3.3a}$$

$$\sigma' = \max(0, \alpha z), \tag{3.3b}$$

where $z$ is equivalent to $W_k^T \cdot \mathbf{x} + b_k$. During each iteration of the training process, the gradients of the loss function concerning the trainable parameters $\nabla_{W_k, b_k, \alpha_k} L$ are computed for each hidden layer $k$ using the chain rule of differentiation. The resulting gradients are then backpropagated through the network to update and optimize the weights $W_k$, biases $b_k$, and the scaling parameter $\alpha_k$. This process enables the network to learn the optimal values of $\alpha_k$, along with $W_k$ and $b_k$, and hence, potentially leads to improved performance in approximating $\hat{\mathbf{y}}$. More detailed information about adaptive activation functions can be found in [26, 27].

### 3.2.2. Loss Function

Prior physical knowledge is integrated with the NN architecture to make it informed; see Figure 3.1-B. A part of the topology information of the power system, i.e., the diagonal elements of the admittance matrix $Y_{kk}$, is used to develop the physical model. In addition, the real and imaginary components of the complex voltage at the reference bus, i.e., $\mu_0 = 1, \omega_0 = 0$, are imported. Eventually, the physical model $f'(\cdot)$ is derived from the prior physical knowledge of the power system.

The derivation begins with Ohm's law, which relates voltage $V$, current $I$, and resistance $R$, that is, $V = I \times R$. By extending this equation to the complex domain, $\overline{V} = \overline{I} \times \overline{Z}$ relates the voltage phasor, current phasor, and impedance phasor. From this equation, $\overline{I}$ can be expressed in terms of the voltage phasor and admittance phasor $\overline{Y}$ (inverse of the impedance phasor), that is $\overline{I} = \overline{Y} \times \overline{V}$. The system of equations for all the buses can then be presented in matrix form as:

$$\left[I\right]_{n\times 1} = \left[Y\right]_{n\times n} \times \left[V\right]_{n\times 1}. \tag{3.4}$$

By rearranging (3.4), the current flowing into bus $k$ can be presented as a linear combination of the voltages at all other buses with weights given by the corresponding admittance matrix:

$$I_k = \sum_{i=1}^{n} Y_{ki} V_i. \tag{3.5}$$

Here, the power equation $S = V \times I^*$ provides further guides for subsequent derivations, that is $I = \frac{S^*}{V^*}$, where $S$ is complex apparent power. The current and voltage phasors are related as:

$$\frac{S_k^*}{V_k^*} = Y_{k1}V_1 + Y_{k2}V_2 + \ldots + Y_{kk}V_k + \ldots + Y_{kn}V_n, \tag{3.6}$$

wherein the right-hand side is an extended form of the right-hand expression in (3.5). Finally, the expression for calculating the voltage phasor at bus $k$ can be written as:

$$V_k = \frac{1}{Y_{kk}} \times \left( \frac{S_k^*}{V_k^*} - \sum_{i=1, i\neq k}^{n} Y_{ki} V_i \right). \tag{3.7}$$

In (3.7), $Y_{kk} = G_{kk} + jB_{kk}$ is known from the power system topology. Considering the input features and output labels needed to develop the NN for PF analysis, $V_k = \mu_k + j\omega_k$ and $V_k^* = \mu_k - j\omega_k$ are known from the output labels, and $S_k^* = P_k^D + jQ_k^D$ is known from the input features. The only unknown expression remaining in (3.7) is:

$$\psi_k = \sum_{i=1, i\neq k}^{n} Y_{ki} V_i, \tag{3.8}$$

which is called *hidden function* in this study. Note that $\psi$ is unique for each data point. Theoretically, $\psi$ is also unique for each bus. During the training process, the NN first approximates $\psi$ and then uses it to approximate $V = \mu + j\omega$ using:

$$f'(\mathbf{x}) = \frac{1}{Y_{kk}} \times \left( \frac{P^D - jQ^D}{\mu - j\omega} - \psi \right). \tag{3.9}$$

This unique relation improves the learning process by following the underlying physical laws of the power system. Instead of integrating the full admittance matrix $[Y]_{n\times n}$ into the physics-based loss function, only the diagonal elements $Y_{kk}$ are needed. This simplification reduces the wall-clock training time by up to three orders of magnitude compared to the full admittance matrix integration used in [28]. Note that this comparison refers to NN training with physics-based loss functions. While numerical solvers already exploit the sparsity of the admittance matrix, integrating the full matrix in NN training remains computationally expensive. By using only the diagonal elements, PINN4PF preserves key physical consistency while drastically reducing computational requirements. It is also important to note that the diagonal entries represent each bus's self-admittance, which dominates local voltage-current coupling and implicitly accounts for the aggregate effect of connected branches. This allows the model to retain essential physical constraints without requiring full system topology information.

Typically, the goal of the training process is to minimize the difference between $\hat{\mathbf{y}}$, approximated by the NN, and ground-truth output labels $\mathbf{y}$, obtained from NR, based on a loss function of choice. In this study, a modified loss function is developed that combines supervised and physics-based penalty terms. The supervised term enforces agreement with ground-truth data, while the physics-based term ensures local compliance with Ohm's and power balance laws. This hybrid formulation enhances learning stability and generalization, particularly under data-limited or uncertain operating conditions (e.g., [29]). The former term is the mean square of the difference between the output approximated by the NN $f(\cdot)$ and $\mathbf{y}$. whereas the latter is the mean square of the difference between the output obtained from the physical model $f'(\cdot)$ and $\mathbf{y}$, as:

$$\begin{aligned}\mathscr{L} = \beta_0 \times \underbrace{\frac{1}{N}\sum_{j=1}^{N}\Big(f(\mathbf{P}_j^D, \mathbf{Q}_j^D, \theta, \alpha) - \mathbf{y}_j\Big)^2}_{\text{Supervised penalty term}} \\ +\beta_1 \times \underbrace{\frac{1}{N}\sum_{j=1}^{N}\Big(f'(\mathbf{P}_j^D, \mathbf{Q}_j^D, Y_{kk}, \psi_j) - \mathbf{y}_j\Big)^2}_{\text{Physical penalty term}},\end{aligned} \tag{3.10}$$

where $N$ is the total number of data points $j$. $f(\cdot) \in \{(f_0, f_1, f_2)\}$ and $f'(\cdot)$ are the NN architecture (3.1a)–(3.1c) and the physical model (3.9), respectively. $\beta_0$ and $\beta_1$ represent the coefficients for the supervised and physical penalty terms, respectively, with the constraint that $\beta_0 + \beta_1 = 1$. Initially, $\beta_0 = 1$ and $\beta_1 = 0$. After 100 epochs, the value of $\beta_1$ increases at a specified rate, with its maximum value determined from the sensitivity analysis. $\theta = \{W, b\}$ is the trainable parameters of the NN and $\alpha$ is the trainable parameter for the activation function.

## 3.3. Results

The performance of PINN4PF is evaluated against two baselines: a linear regression model (LR) and a purely data-driven NN (MLP). Experiments are performed on 4-bus [30], 15-bus [31], 290-bus [32], and 2224-bus [33] test systems. The selected systems contain one reference bus with known voltages in complex number form, $\mu_0 + j\omega_0$, and unknown $P_0^D$ and $Q_0^D$, and load buses $i$ for which $P_i^D$ and $Q_i^D$ are known, while complex voltages $\mu_i + j\omega_i$ are unknown.

Input features and output labels of the datasets are represented as $\mathbf{x}$ and $\mathbf{y}$, respectively, where $\mathbf{x}$ includes $(\mathbf{P}_i^D, \mathbf{Q}_i^D)$ and $\mathbf{y}$ includes $(\mu_i, \omega_i)$ obtained from the NR solver for all load buses $i$. The Pandapower Python package [34] is used to perform NR and generate the datasets. While NR provides the physical reference against which errors are measured, LR, MLP, and PINN4PF are compared in terms of their ability to approximate NR results efficiently and accurately, which is a common practice in the literature (e.g., [35]). Note that Pandapower specifies the state variables of power systems, i.e., $[\delta\ V]^T$, that is, there is a need for converting the state variables to $\mu_i = V_i \cos\delta_i$ and $\omega_i = V_i \sin\delta_i$ to yield the output labels $\mathbf{y} = \{(\mu_i, \omega_i) : i = 1, 2, \ldots, n\}$.

### 3.3.1. Model Setup

A systematic approach is employed to generate the datasets. Considering $\mathbf{P}^D$ and $\mathbf{Q}^D$ known for a specific scenario of the test system, $S_i^D = \sqrt{(P_i^D)^2 + (Q_i^D)^2}$, and $pf_i = P_i^D / S_i^D$ are computed for each bus $i$. Here, $s_i^D$ is the mean, and a deviation of 30% from $S_i^D$ is the standard deviation to develop a normal distribution of size 5,000 for each bus $i$, i.e., $S_i \sim \mathcal{N}(S_i^D, 0.3)$. For $S_i \in \{S_{ij}^D : j = 1, 2, \ldots, 5000\}$, $P_{ij}^D = S_{ij}^D \times pf_i$ and $Q_{ij}^D = \sqrt{(S_{ij}^D)^2 - (P_{ij}^D)^2}$ are then computed for all buses $i$ and all samples $j$. This approach yields a pool of 5,000 scenarios from which the data points are randomly selected to form the dataset. The datasets contain different numbers of data points: 256, 512, 1024, and 2048 for the 4-bus, 15-bus, 290-bus, and 2224-bus test systems, respectively. The randomly selected data points out of 5,000 generated scenarios are sent to the classical PF solver, i.e., Pandapower, to perform PF analysis. Each dataset is then split into three subsets: 40% for training, 20% for validation, and the remaining 40% for testing.

Note that the number of data points is deliberately limited to highlight the ability of PINN4PF to learn effectively even with a relatively small training dataset, thanks to its physics-based loss function. While NNs typically require large datasets for effective training, PINN4PF reduces this dependency by embedding prior physical knowledge directly into the training process and leveraging adaptive activation functions. This approach is consistent with practices reported in the literature, where physics-informed learning has been employed as a strategy to reduce the amount of training data required (e.g., [36]).

The proposed end-to-end architecture is developed in Python. All components, including model definition, training loops, and evaluation routines, are implemented

within the PyTorch environment, leveraging its built-in automatic differentiation and GPU acceleration capabilities. The training process ends after 5,000 epochs for PINN4PF, MLP, and LR. The loss function used is (3.10), with $\beta_0 = 1$ and $\beta_1 = 0$ for MLP and LR, indicating a supervised penalty term. However, $\beta_0$ and $\beta_1$ are non-zero for PINN4PF, where $\beta_0 + \beta_1 = 1$, indicating a combination of supervised and physical penalty terms. The activation function is ReLU (3.3a) for MLP and the shared hidden layers of PINN4PF. The adaptive ReLU (3.3b) is used for the separated hidden layers of PINN4PF. The Adam optimization algorithm updates the trainable parameters during the training process for PINN4PF, MLP, and LR.

### 3.3.2. Model Performance

The performance of PINN4PF, MLP, and LR is systematically evaluated based on (i) generalization ability, (ii) robustness, (iii) impact of training dataset size on generalization ability, (iv) accuracy in approximating derived PF quantities, and (v) scalability. Experiments are done using the 15-bus test system. Additional experiments are also performed using 4-bus, 290-bus, and 2224-bus test systems for scalability.

#### Generalization ability

The mean and standard deviation of the mean squared error (MSE) for the test dataset obtained by PINN4PF and MLP are compared in Figure 3.2. PINN4PF is observed to achieve up to 85% and 65% lower maximum testing MSE for the voltage magnitude $(V^2)$ and voltage phase angle $(rad^2)$, respectively, compared to MLP.

#### Robustness

The robustness of PINN4PF, MLP, and LR is evaluated using the 15-bus test system under varying noise levels. Controlled uniform random noise is added to both the input features and output labels of the training dataset to simulate measurement uncertainties. This approach aligns with the use of bounded or non-Gaussian noise models commonly adopted in power system studies (e.g., [37]). The corrupted vectors are defined as $\mathbf{x}' = \mathbf{x} \pm \mathbf{r}_x$ and $\mathbf{y}' = \mathbf{y} \pm \mathbf{r}_y$, where $\mathbf{r}_x$ and $\mathbf{r}_y$ are vectors sampled from uniform distributions within the ranges $[0, 1]$ and $[0, 0.1]$, respectively. The larger perturbation range for the input features reflects higher uncertainty typically associated with active and reactive power measurements, while a smaller range is applied to the output labels to maintain physically meaningful voltage responses.

Figure 3.3 illustrates the changes in the MSE for the voltage magnitude $(V^2)$ based on the testing dataset with varying noise levels for MLP. The comparison is made relative to a constant curve, representing the MSE obtained for PINN4PF using the highest noise level, i.e., 10%. At the 0% noise level, MLP outperforms PINN4PF trained with 10% noisy data by approximately 18%. However, as the noise level increases, the performance of MLP deteriorates significantly, with MSE increasing up to six times. LR is excluded from the graph as its MSE at different

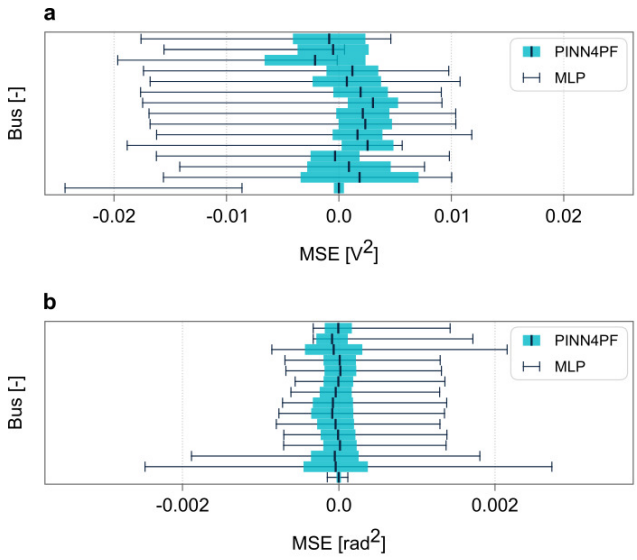



Figure 3.2: Mean and standard deviation of MSE for direct physical quantities: (a) $\mathbf{V}$ $(V^2)$ and (b) $\delta$ $(rad^2)$. The results are obtained by PINN4PF and MLP based on the testing dataset for the 15-bus test system.

noise levels is two orders of magnitude higher than that of PINN4PF. In addition, LR shows limited improvement with increasing dataset sizes and consistently underperforms compared to both PINN4PF and MLP.

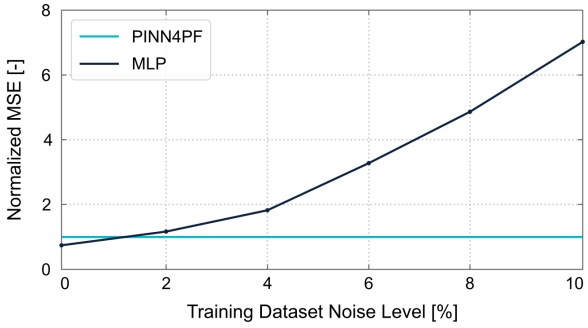



Figure 3.3: Illustration of the performance of PINN4PF and MLP based on the MSE obtained for $\mathbf{V}$ $(V^2)$ under varying noise levels in the training dataset for the 15-bus test system. The MSE values are normalized relative to that of PINN4PF at the 10% noise level.

### Training dataset size

The impact of the size of the training dataset on the performance of MLP is investigated using the 15-bus test system. Figure 3.4 illustrates the changes in the MSE for the voltage magnitude ($V^2$) based on the testing dataset with varying training dataset sizes. The comparison is made relative to a constant curve representing the MSE obtained for PINN4PF using 256 training data points. It is observed that MLP requires a training dataset twice as large as that of PINN4PF to achieve a still inferior performance. However, the performance of MLP improves with more training data, reaching a comparable level with a dataset four times larger than that of PINN4PF. This indicates that PINN4PF is more data-efficient. LR is not included in the graph as its MSE with different training dataset sizes is two orders of magnitude larger than that of PINN4PF. In addition, LR shows limited improvement with increasing training dataset sizes and consistently performs worse than PINN4PF and MLP. This indicates that the capacity of LR to capture complex relationships in PF analysis is significantly lower, and increasing the amount of training data is insufficient to bridge the performance gap.

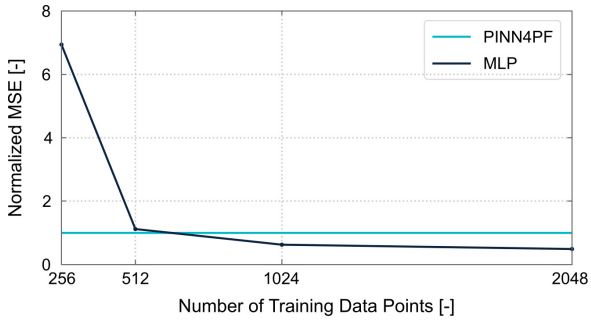



Figure 3.4: Illustration of the performance of PINN4PF and MLP based on the MSE obtained for $\mathbf{V}$ ($V^2$) for the 15-bus test system under different training dataset sizes. The MSE values are normalized relative to that of PINN4PF with 256 training data points.

### Accuracy of derived power flow quantities

For derived physical quantities, i.e., line current, line active power, and line reactive power, the mean and standard deviation of the testing MSE obtained by PINN4PF and MLP are computed and compared in Figure 3.5. It is also observed that PINN4PF achieves up to 81%, 63%, and 66% lower maximum testing MSE for the line current ($A^2$), line active power ($W^2$), and line reactive power ($VAR^2$), respectively, compared to MLP. Note that, while the testing MSE for direct quantities (voltage magnitude and phase) shows lower differences between PINN4PF and MLP, PINN4PF achieves substantially better accuracy for derived physical quantities. This improvement arises because the physics-based loss function enforces the underlying algebraic and system constraints that govern how these derived variables

are computed from the predicted voltages. Consequently, error propagation to derived quantities is reduced. Such behavior, i.e., enhanced internal consistency and improved generalization for derived variables under physics-informed learning, has been observed in recent studies in power systems and related domains (e.g., [38]).

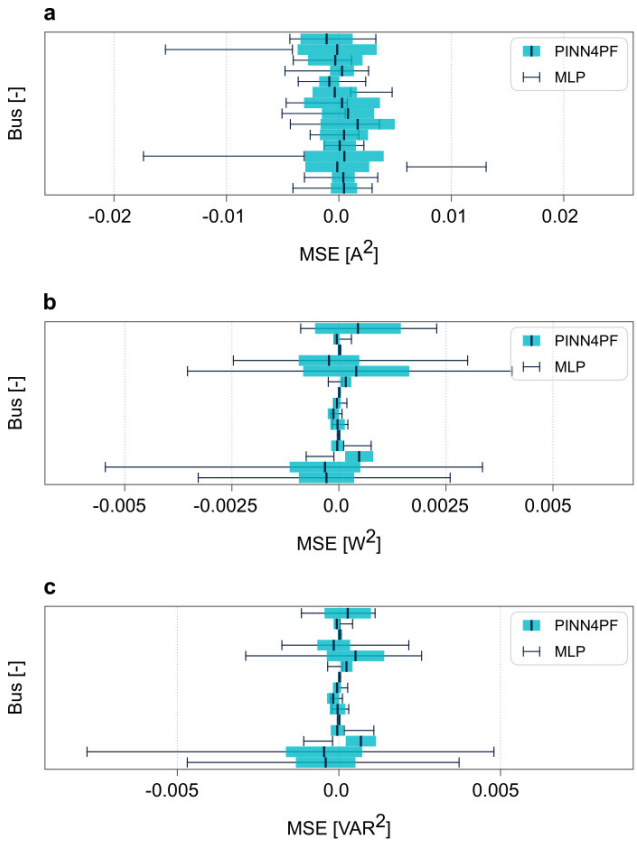



Figure 3.5: Mean and standard deviation of MSE for derived physical quantities: (a) $\mathbf{i}$ ($A^2$), (b) $\mathbf{P_l}$ ($W^2$), and (c) $\mathbf{Q_l}$ ($VAR^2$). The results are obtained by PINN4PF and MLP based on the testing dataset for the 15-bus test system.

## Scalability

The experiments are extended to compare the performance of PINN4PF, MLP, and LR across different test system sizes: 4-bus, 15-bus, 290-bus, and 2224-bus test systems. The maximum testing MSE for the direct physical quantities obtained for all test system sizes is presented in Table 3.1. Accordingly, PINN4PF significantly outperforms MLP and LR. To further highlight the performance of PINN4PF under

extreme conditions with limited training data, we consider a large-scale test system operating near its loadability limits, with a few bus voltages reaching approximately 0.85 ($p.u.$) and exceeding 1.05 ($p.u.$). The direct physical quantities for the 2224-bus test system are depicted in Figure 3.6. PINN4PF achieves a maximum MSE ($V^2$) 50% lower than MLP for this test system. This improvement is crucial for power system operations, particularly during unexpected events that cause deviations in $v$ from the nominal value.

Table 3.1: Performance comparison of power flow solvers based on MSE obtained for direct physical quantities.

| **Test System** | **Model** | **MSE** ($V^2$) | **MSE** ($rad^2$) |
|---|---|---|---|
| 4-bus | PINN4PF | $4.85 \times 10^{-4}$ | $3.81 \times 10^{-8}$ |
| | MLP | $5.70 \times 10^{-3}$ | $3.72 \times 10^{-7}$ |
| | LR | $2.52 \times 10^{-2}$ | $4.70 \times 10^{-6}$ |
| 15-bus | PINN4PF | $5.73 \times 10^{-6}$ | $1.06 \times 10^{-7}$ |
| | MLP | $3.96 \times 10^{-5}$ | $3.09 \times 10^{-7}$ |
| | LR | $6.32 \times 10^{-4}$ | $1.57 \times 10^{-5}$ |
| 290-bus | PINN4PF | $9.54 \times 10^{-10}$ | $1.65 \times 10^{-8}$ |
| | MLP | $3.03 \times 10^{-9}$ | $1.78 \times 10^{-8}$ |
| | LR | $1.03 \times 10^{-7}$ | $2.49 \times 10^{-8}$ |
| 2224-bus | PINN4PF | $1.03 \times 10^{-4}$ | $8.15 \times 10^{-5}$ |
| | MLP | $2.07 \times 10^{-4}$ | $7.66 \times 10^{-5}$ |
| | LR | $3.62 \times 10^{0}$ | $2.93 \times 10^{-4}$ |

The analysis also considers the performance of PINN4PF, MLP, and LR in approximating derived physical quantities. The maximum testing MSE for all test system sizes is presented in Table 3.2. The table demonstrates the superiority of PINN4PF over MLP and LR across all test system sizes. For the 2224-bus test system, for example, the maximum MSE ($A^2$) obtained by PINN4PF is respectively 92% and 98% lower than MLP and LR. In addition, as the size of the test system increases, the performance gap between PINN4PF, MLP, and LR becomes more pronounced. Figure 3.7 compares the performance of PINN4PF and MLP under extreme conditions in the 2224-bus test system for the derived physical quantity. It should be noted that while the testing MSE for the direct quantities, i.e., voltage magnitude and phase angle, shows marginal differences between PINN4PF and MLP, PINN4PF significantly outperforms MLP in terms of the derived physical quantities, such as line current.

### 3.3.3. Model Sensitivity Analysis

The architecture of PINN4PF and MLP, including the number of hidden layers and neurons per hidden layer, as well as hyperparameters, including the learning rate,

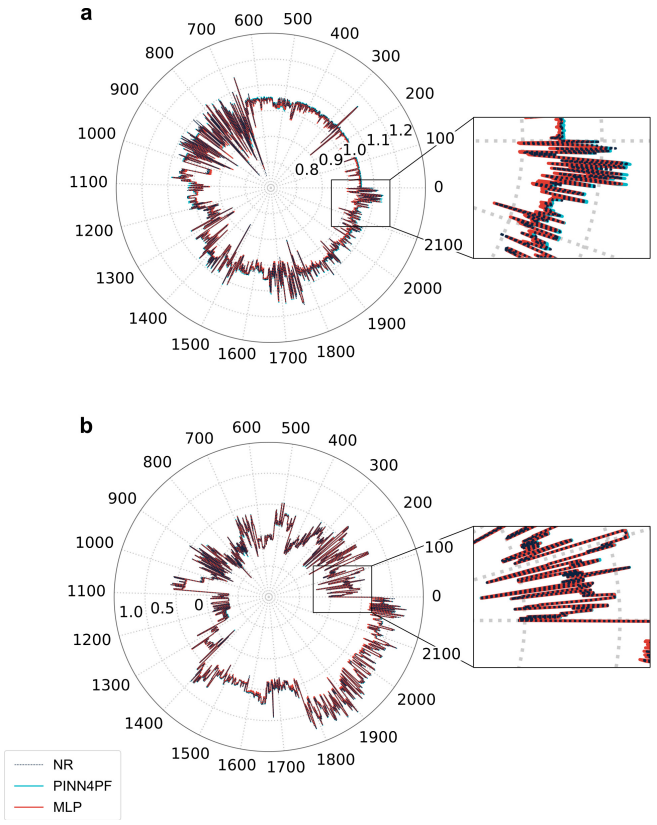



Figure 3.6: Comparison of (a) $\mathbf{V}$ and (b) $\delta$ obtained from NR, PINN4PF, and MLP for the 2224-bus test system. The bus indices are positioned along the outer circle, while the corresponding quantities are plotted along the radial direction.

the weight decay rate, and the percentage of dropout, are determined by sensitivity analysis to ensure a fair comparison[1]. The maximum weight for the share of the supervised penalty term ($\beta_0$) in (3.10) is also fine-tuned for PINN4PF. For MLP, $\beta_0 = 1$ and $\beta_1 = 0$. Sensitivity analysis is not performed for LR since it involves linear combinations and lacks hyperparameters. The optimal configurations are identified based on minimizing both training and testing MSE, while also ensuring that they remain close enough to indicate generalization ability. All experiments use the 15-bus test system and a noisy dataset comprising 512 data points.

[1] An exhaustive search across 2000 unique hyperparameter combinations is conducted using the Optuna Python package [39].

Table 3.2: Performance comparison of power flow solvers based on MSE obtained for derived physical quantities.

| Test System | Model | MSE ($A^2$) | MSE ($W^2$) | MSE ($VAR^2$) |
|---|---|---|---|---|
| 4-bus | PINN4PF | $2.43 \times 10^{-6}$ | $4.25 \times 10^{-4}$ | $1.06 \times 10^{-4}$ |
| | MLP | $2.80 \times 10^{-5}$ | $8.27 \times 10^{-3}$ | $2.06 \times 10^{-3}$ |
| | LR | $4.04 \times 10^{-5}$ | $1.34 \times 10^{-2}$ | $3.36 \times 10^{-2}$ |
| 15-bus | PINN4PF | $4.35 \times 10^{-6}$ | $2.09 \times 10^{-7}$ | $2.17 \times 10^{-7}$ |
| | MLP | $2.39 \times 10^{-5}$ | $5.77 \times 10^{-7}$ | $6.42 \times 10^{-7}$ |
| | LR | $1.10 \times 10^{-2}$ | $8.70 \times 10^{-4}$ | $1.74 \times 10^{-3}$ |
| 290-bus | PINN4PF | $3.84 \times 10^{-7}$ | $2.01 \times 10^{-13}$ | $4.77 \times 10^{-14}$ |
| | MLP | $2.85 \times 10^{-6}$ | $1.10 \times 10^{-12}$ | $2.44 \times 10^{-13}$ |
| | LR | $1.36 \times 10^{-5}$ | $8.20 \times 10^{-12}$ | $1.84 \times 10^{-12}$ |
| 2224-bus | PINN4PF | $1.15 \times 10^{-6}$ | $1.13 \times 10^{-8}$ | $9.45 \times 10^{-7}$ |
| | MLP | $3.69 \times 10^{-5}$ | $7.22 \times 10^{-8}$ | $5.12 \times 10^{-5}$ |
| | LR | $7.85 \times 10^{-1}$ | $3.55 \times 10^{-1}$ | $2.44 \times 10^{2}$ |

### Sensitivity analysis for MLP

We explore eight configurations, intentionally avoiding deeper networks with more than eight hidden layers due to the limited size of both the dataset and the test system. Each configuration comprises input and output layers defined by $n \times 2$ neurons in the first and last hidden layers, respectively, while intermediate layers are structured with $n \times 4/3$ neurons. We observe that when the number of hidden layers is fewer than seven, a reduction in layer count correlates with a decrease in generalization ability. We examine learning rates ranging from $1 \times 10^{-1}$ to $1 \times 10^{-10}$ and weight decay rates within the same span. Dropout rates are tested between 0% and 2%. Sensitivity analysis results indicate that the selected configuration for MLP involves seven hidden layers and is trained with a learning rate of $2.3 \times 10^{-4}$, a weight decay of $1.8 \times 10^{-5}$, a dropout percentage of 0.2%, and a batch size of 16. For this configuration, the training and testing MSE achieved are $3.74 \times 10^{-5}(V^2)$ and $3.96 \times 10^{-5}(V^2)$, respectively.

### Sensitivity analysis for PINN4PF

We systematically assess the influence of the number of hidden layers, ranging from one to four, within the shared set of hidden layers and the individual heads. We evaluate distinct configurations where the shared hidden layers contain $n \times 2$ neurons, and each head comprises $n$ neurons. Learning and weight decay rates are varied from $1 \times 10^{-1}$ to $1 \times 10^{-10}$, and dropout rates are varied from 0% to 2%. Sensitivity analysis reveals that the selected configuration for PINN4PF consists of two shared hidden layers alongside four hidden layers per head. In addition, PINN4PF is trained with a learning rate of $1.3 \times 10^{-4}$, a weight decay of

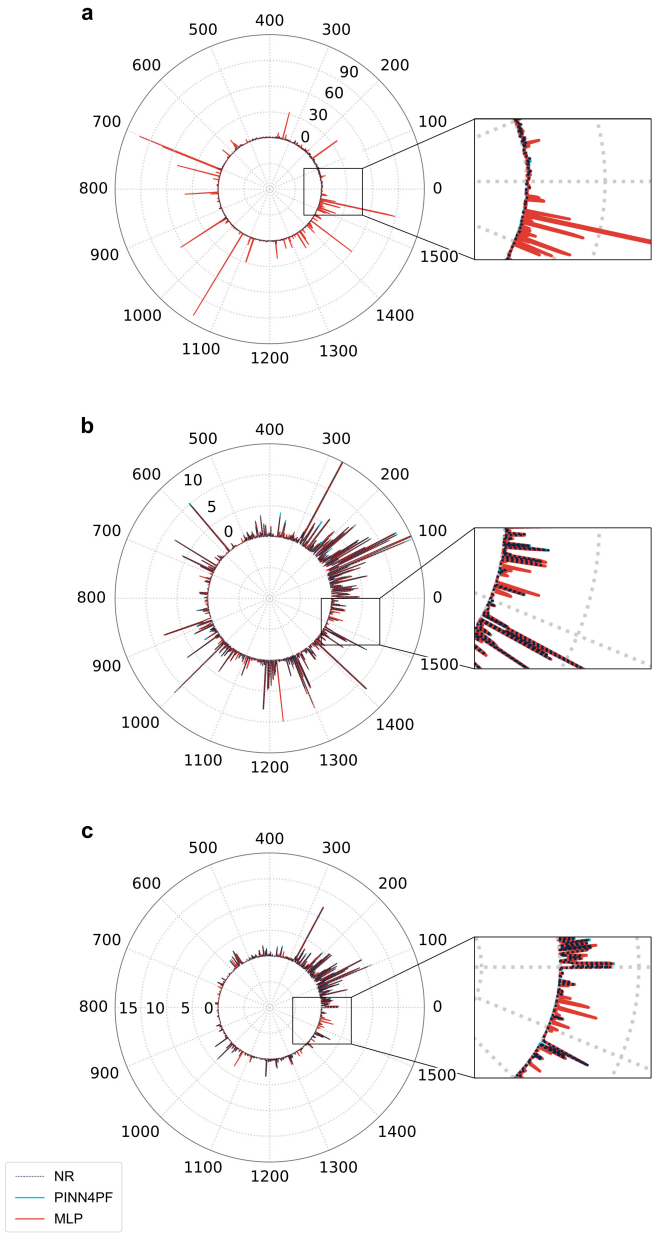



Figure 3.7: Comparison of (a) $\mathbf{i}$ ($A^2$), (b) $\mathbf{P_l}$ ($W^2$), and (c) $\mathbf{Q_l}$ ($VAR^2$) obtained from NR, PINN4PF, and MLP for the 2224-bus test system. The bus indices are positioned along the outer circle, while the corresponding quantities are plotted along the radial direction.

$1.1 \times 10^{-5}$, a dropout rate of $0.1\%$, and a batch size of $16$. The maximum weight for the share of the supervised penalty term is $\beta_1 = 0.71$. For this configuration, the achieved training and testing MSE are $5.64 \times 10^{-6}(V^2)$ and $5.73 \times 10^{-6}(V^2)$, respectively.

### Impact of modifications on the performance of PINN4PF

We evaluate the following configurations: (i) a double-head feed-forward NN with ReLU (3.3a) and supervised penalty term (ReLU&Supervised), (ii) a double-head feed-forward NN with adaptive ReLU (3.3b) and supervised penalty term (AdaptiveReLU&Supervised), (iii) a double-head feed-forward NN with ReLU (3.3a) and physical penalty term (ReLU&Physical), and (iv) a double-head feed-forward NN with adaptive ReLU (3.3b) and physical penalty term (AdaptiveReLU&Physical) that is the selected configuration for PINN4PF. Table 3.3 presents a comparative analysis of their performance in terms of direct physical quantities. The results show that combining the physics-based loss function with the adaptive activation function yields superior performance for PINN4PF compared to the other combinations.

Table 3.3: Performance comparison of different configurations of PINN4PF based on MSE obtained for the 15-bus test system.

| **Model** | **MSE** $(V^2)$ | **MSE** $(rad^2)$ |
|---|---|---|
| ReLU&Supervised | $4.05 \times 10^{-5}$ | $3.24 \times 10^{-7}$ |
| AdaptiveReLU&Supervised | $1.42 \times 10^{-5}$ | $3.73 \times 10^{-7}$ |
| ReLU&Physical | $6.31 \times 10^{-6}$ | $4.70 \times 10^{-7}$ |
| AdaptiveReLU&Physical | $5.73 \times 10^{-6}$ | $1.06 \times 10^{-7}$ |

## 3.4. Discussion and Conclusions

The physics-based loss function acts as a training penalty to enforce power-balance consistency and enhance the accuracy of bus voltage predictions, whereas actual power system losses are computed exactly from the predicted/calculated voltages using standard electrical formulations. Although only the diagonal elements of the admittance matrix are used to develop the loss function, including a physical penalty term imposes additional calculations during training, which makes PINN4PF computationally more expensive than supervised models and highlights a trade-off between accuracy and computational cost. The optimization algorithm suggests that, for larger test systems, the share of the supervised penalty term ($\beta_0$) in the loss function should be significantly smaller than that of the physical penalty term ($\beta_1$), indicating the increasing importance of physical constraints. Furthermore, PINN4PF demonstrates strong performance under fixed network topology. It requires retraining if the system topology changes, as the current architecture does not support topological generalization. Future research can therefore explore the

potential of transfer learning to adapt pre-trained models to new topologies with minimal additional training.

In modern power systems, missing or noisy measurements and incomplete topology data are common, often causing traditional PF solvers to fail. PINN4PF addresses these challenges by integrating partial topology information, specifically, the diagonal elements of the admittance matrix, alongside combining supervised and physics-based penalty terms. This design enables PINN4PF to learn complex system dynamics from limited data and enhances robustness against noise or incompleteness in both training samples and topology information. Future work can further assess PINN4PF's resilience to noise in the admittance matrix and, relatedly, examine whether the model can recover solutions in ill-conditioned cases where the NR method fails to converge—a regime in which a learned mapping is not bound by the conditioning of the NR Jacobian. Finally, the datasets are simulation-based rather than derived from field measurements, which aligns with standard practice in power system studies (e.g., [35, 40]), given the limited availability of large-scale, labeled operational data. Future work can focus on validating the model using real-world power system data.

## References

[1] **Z. Kaseb**, S. Orfanoudakis, P. P. Vergara and P. Palensky. 'Physics-informed neural network with adaptive activation for power flow'. In: *International Journal of Electrical Power & Energy Systems* 174 (2026), p. 111525. DOI: 10.1016/j.ijepes.2025.111525.

[2] G. K. Dziugaite and D. M. Roy. 'Computing Nonvacuous Generalization Bounds for Deep (Stochastic) Neural Networks with Many More Parameters than Training Data'. In: *arXiv preprint* (Mar. 2017). DOI: arXiv:1703.11008.

[3] T. Wolgast and A. Nieße. 'Learning the optimal power flow: Environment design matters'. In: *Energy and AI* 17 (2024), p. 100410. DOI: 10.1016/j.egyai.2024.100410.

[4] E. Cibaku, F. Gama and S. Park. 'Boosting efficiency in state estimation of power systems by leveraging attention mechanism'. In: *Energy and AI* 16 (2024), p. 100369. DOI: 10.1016/j.egyai.2024.100369.

[5] D. Beinert, C. Holzhüter, J. M. Thomas and S. Vogt. 'Power flow forecasts at transmission grid nodes using Graph Neural Networks'. In: *Energy and AI* 14 (2023), p. 100262. DOI: 10.1016/j.egyai.2023.100262.

[6] Z. Wang, J.-H. Menke, F. Schäfer, M. Braun and A. Scheidler. 'Approximating multi-purpose AC Optimal Power Flow with reinforcement trained Artificial Neural Network'. In: *Energy and AI* 7 (2022), p. 100133. DOI: 10.1016/j.egyai.2021.100133.

[7] R. Ahshan, M. S. Abid and M. Al-Abri. 'Geospatial Mapping of Large-Scale Electric Power Grids: A Residual Graph Convolutional Network-Based Approach with Attention Mechanism'. In: *Energy and AI* 20 (2025), p. 100486. DOI: 10.1016/j.egyai.2025.100486.

[8] H.-L. Huang, X.-Y. Xu, C. Guo, G. Tian, S.-J. Wei, X. Sun, W.-S. Bao and G.-L. Long. 'Near-term quantum computing techniques: Variational quantum algorithms, error mitigation, circuit compilation, benchmarking and classical simulation'. In: *Science China Physics, Mechanics & Astronomy* 66 (2023). ISSN: 1674-7348. DOI: 10.1007/s11433-022-2057-y.

[9] **Z. Kaseb**, M. Möller, G. T. Balducci, P. Palensky and P. P. Vergara. 'Quantum neural networks for power flow analysis'. In: *Electric Power Systems Research* 235 (2024), p. 110677. DOI: 10.1016/j.epsr.2024.110677.

[10] R. A. d. Oliveira and M. H. Bollen. 'Deep learning for power quality'. In: *Electric Power Systems Research* 214 (Jan. 2023), p. 108887. DOI: 10.1016/j.epsr.2022.108887.

[11] L. von Rueden, S. Mayer, K. Beckh, B. Georgiev, S. Giesselbach, R. Heese, B. Kirsch, J. Pfrommer, A. Pick, R. Ramamurthy, M. Walczak, J. Garcke, C. Bauckhage and J. Schuecker. 'Informed Machine Learning – A Taxonomy and Survey of Integrating Knowledge into Learning Systems'. In: (Mar. 2019). DOI: 10.1109/TKDE.2021.3079836.

[12] H. Li, L. Liu, S. Yu, Q. He, Q. Wu, J. Zhang and Q. Lu. 'Graph attention convolution network for power flow calculation considering grid uncertainty'. In: *International Journal of Electrical Power & Energy Systems* 165 (2025), p. 110513. DOI: 10.1016/j.ijepes.2025.110513.

[13] D. Liang, G. Li, X. Liu, L. Zeng, H.-D. Chiang and S. Wang. 'Bayesian state estimation for partially observable distribution networks via power flow-informed neural networks'. In: *International Journal of Electrical Power & Energy Systems* 170 (2025), p. 110886. DOI: 10.1016/j.ijepes.2025.110886.

[14] X. Lei, Z. Yang, J. Yu, J. Zhao, Q. Gao and H. Yu. 'Data-Driven Optimal Power Flow: A Physics-Informed Machine Learning Approach'. In: *IEEE Transactions on Power Systems* 36.1 (Jan. 2021), pp. 346–354. DOI: 10.1109/TPWRS.2020.3001919.

[15] Q. Yang, G. Wang, A. Sadeghi, G. B. Giannakis and J. Sun. 'Two-Timescale Voltage Control in Distribution Grids Using Deep Reinforcement Learning'. In: *IEEE Transactions on Smart Grid* 11.3 (2020), pp. 2313–2323. DOI: 10.1109/TSG.2019.2951769.

[16] P. L. Donti, D. Rolnick and J. Z. Kolter. 'DC3: A learning method for optimization with hard constraints'. In: *arXiv preprint* (Apr. 2021). DOI: arXiv:2104.12225.

[17] A. B. Jeddi and A. Shafieezadeh. 'A Physics-Informed Graph Attention-based Approach for Power Flow Analysis'. In: *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, Dec. 2021, pp. 1634–1640. DOI: 10.1109/ICMLA52953.2021.00261.

[18] S. De Jongh, F. Gielnik, F. Mueller, L. Schmit, M. Suriyah and T. Leibfried. 'Physics-Informed Geometric Deep Learning for Inference Tasks in Power Systems'. In: *22nd Power Systems Computation Conference*. Porto, Portugal, 2022.

[19] M. Yang, G. Qiu, Y. Wu, J. Liu, N. Dai, Y. Shui, K. Liu and L. Ding. 'Physics-Guided Graph Neural Networks for Real-time AC/DC Power Flow Analysis'. In: *arXiv preprint* (Apr. 2023). DOI: arXiv:2305.00216.

[20] A. Kody, S. Chevalier, S. Chatzivasileiadis and D. Molzahn. 'Modeling the AC Power Flow Equations with Optimally Compact Neural Networks: Application to Unit Commitment'. In: *22nd Power Systems Computation Conference*. 2022.

[21] W. Liao, D. Yang, Q. Liu, Y. Jia, C. Wang and Z. Yang. 'Data-driven Reactive Power Optimization of Distribution Networks via Graph Attention Networks'. In: *Journal of Modern Power Systems and Clean Energy* 12.3 (2024), pp. 874–885. DOI: 10.35833/MPCE.2023.000546.

[22] Y. Li, C. Zhao and C. Liu. 'Model-informed generative adversarial network for learning optimal power flow'. In: *IISE Transactions* (Nov. 2023), pp. 1–14. DOI: 10.1080/24725854.2023.2286507.

[23] X. Hu, H. Hu, S. Verma and Z.-L. Zhang. 'Physics-Guided Deep Neural Networks for Power Flow Analysis'. In: *IEEE Transactions on Power Systems* 36.3 (May 2021), pp. 2082–2092. DOI: 10.1109/TPWRS.2020.3029557.

[24] Z. Hu, H. Zhang, R. Yang, Y. Chen and H. Wu. 'Optimal power flow based on physical-model-integrated neural network with worth-learning data generation'. In: *CSEE Journal of Power and Energy Systems* (2025), pp. 1–10. DOI: 10.17775/CSEEJPES.2023.09170.

[25] Z. Wu, M. Zhang, S. Gao, Z.-G. Wu and X. Guan. 'Physics-Informed Reinforcement Learning for Real-Time Optimal Power Flow with Renewable Energy Resources'. In: *IEEE Transactions on Sustainable Energy* (2024), pp. 1–11. DOI: 10.1109/TSTE.2024.3452489.

[26] **Z. Kaseb**, Y. Xiang, P. Palensky and P. P. Vergara. 'Adaptive activation functions for deep learning-based power flow analysis'. In: *IEEE PES ISGT EUROPE*. 2023.

[27] A. D. Jagtap, K. Kawaguchi and G. E. Karniadakis. 'Adaptive activation functions accelerate convergence in deep and physics-informed neural networks'. In: *Journal of Computational Physics* 404 (Mar. 2020), p. 109136. DOI: 10.1016/j.jcp.2019.109136.

[28] S. Yan, F. Vazinram, **Z. Kaseb**, L. Spoor, J. Stiasny, B. Mamudi, A. H. Ardakani, U. Orji, P. P. Vergara, Y. Xiang and J. Guo. 'Data driven approach towards more efficient Newton-Raphson power flow calculation for distribution grids'. In: *ICT.OPEN*. 2025.

[29] Q.-H. Ngo, B. L. Nguyen, T. V. Vu, J. Zhang and T. Ngo. 'Physics-informed graphical neural network for power system state estimation'. In: *Applied Energy* 358 (Mar. 2024), p. 122602. DOI: 10.1016/j.apenergy.2023.122602.

[30] J. J. Grainger and W. D. Stevenson Jr. *Power System Analysis*. McGraw-Hill, Inc., 1994.

[31] H. Farhangi and G. Joos. 'Microgrid Benchmarks'. In: *Microgrid Planning and Design*. Wiley, May 2019, pp. 25–36. DOI: 10.1002/9781119453550.ch2.

[32] G. Kerber. 'Aufnahmefähigkeit von Niederspannungsverteilnetzen für die Einspeisung aus Photovoltaikkleinanlagen'. PhD thesis. TECHNISCHE UNIVERSITÄT MÜNCHEN, 2011.

[33] W. A. Bukhsh and K. McKinnon. 'Network Data of Real Transmission Networks'. In: (2013).

[34] L. Thurner, A. Scheidler, F. Schäfer, J.-H. Menke, J. Dollichon, F. Meier, S. Meinecke and M. Braun. 'pandapower—an open-source python tool for convenient modeling, analysis, and optimization of electric power systems'. In: *IEEE Transactions on Power Systems* 33.6 (2018), pp. 6510–6521. DOI: 10.1109/TPWRS.2018.2829021.

[35] N. Lin, S. Orfanoudakis, N. O. Cardenas, J. S. Giraldo and P. P. Vergara. 'PowerFlowNet: Power flow approximation using message passing Graph Neural Networks'. In: *International Journal of Electrical Power & Energy Systems* 160 (Sept. 2024), p. 110112. DOI: 10.1016/j.ijepes.2024.110112.

[36] A. Safonova, G. Ghazaryan, S. Stiller, M. Main-Knorn, C. Nendel and M. Ryo. 'Ten deep learning techniques to address small data problems with remote sensing'. In: *International Journal of Applied Earth Observation and Geoinformation* 125 (Dec. 2023), p. 103569. DOI: 10.1016/j.jag.2023.103569.

[37] S. Cubonovic, D. Cetenovic and A. Rankovic. 'Impact of the non-Gaussian measurement noise on the performance of state-of-the-art state estimators for distribution systems'. In: *Serbian Journal of Electrical Engineering* 21 (1 2024), pp. 113–133. DOI: 10.2298/SJEE2401113C.

[38] S. Falas, M. Asprou, C. Konstantinou and M. K. Michael. 'Physics-Informed Neural Networks for Accelerating Power System State Estimation'. In: IEEE, Oct. 2023, pp. 1–5. DOI: 10.1109/ISGTEUROPE56780.2023.10408467.

[39] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama. 'Optuna: A Next-Generation Hyperparameter Optimization Framework'. In: *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019.

[40] R. Nellikkath and S. Chatzivasileiadis. 'Physics-Informed Neural Networks for Minimising Worst-Case Violations in DC Optimal Power Flow'. In: *2021 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*. 2021, pp. 419–424. DOI: 10.1109/SmartGridComm51999.2021.9632308.

# 4

# Quantum Neural Networks for Power Flow

*We propose the potential application of quantum and hybrid quantum-classical neural networks in the power flow (PF) problem. Experiments are conducted using two datasets based on 4-bus and 33-bus test systems. A systematic performance comparison is also conducted among quantum, hybrid quantum-classical, and classical neural networks. The comparison is based on (i) generalization ability, (ii) robustness, (iii) training dataset size needed, (iv) training error, and (v) training process stability. The reader is referred to Sections 2.1 and 2.2 for background on PF and gate-based quantum computing (AQC), upon which this chapter builds.*

Parts of this chapter have been published in Electric Power Systems Research **235** (2024) with the title "Quantum neural networks for power flow analysis", doi: 10.1016/j.epsr.2024.110677 [1].

## 4.1. Motivation

Neural networks (NNs) have been widely used for the power flow (PF) problem due to their capacity to address complexities present in modern power systems [2]. By learning from historical data, NNs reveal nonlinear and complex relationships between inputs and outputs even in cases for which traditional iterative numerical methods fail to converge, e.g., for ill-conditioned scenarios. This capability is especially critical in accommodating varying load demands and integrating distributed energy resources in large-scale modern power systems [3, 4].

However, while NNs offer significant advantages for the PF problem, their practical implementation presents certain challenges. Firstly, they rely on large, high-quality datasets, which can be hard to obtain due to various reasons, such as privacy concerns and high proportions of missing data. In addition, the computational complexity of the training process increases with the size and complexity of power systems. For instance, the increased depth of NNs necessitates complicated hyperparameter tuning, which demands substantial computational resources. Finally, ensuring scalability, generalization ability, and robustness to noisy training data remains a critical concern for classical NNs. They are often case-specific and prone to overfitting to training data, which can potentially compromise the reliability of classical NNs in safety-critical applications (e.g., [5, 6]).

A radically different machine learning approach that can help overcome the challenges faced by classical NNs is quantum neural networks (QNNs), which means the enrichment of NNs with quantum computing. Quantum computing has the potential to address the complexities of modern power systems. The scope of the studies is grouped into two main quantum computing paradigms: (i) gate-based quantum computing (GQC) and (ii) adiabatic quantum computing model (AQC) paradigms. A majority of the studies, summarized in Table 4.1, focus on the GQC approach while a few studies implemented the AQC approach. The reader is referred to Section 2.2 for background on GQC and AQC upon which this chapter builds. The Harrow-Hassidim-Lloyd (HHL) algorithm has been widely used in different applications, including the PF problem and state estimation (e.g., [5, 7]). The HHL algorithm theoretically offers up to exponential speedup in solving systems of linear equations compared to state-of-the-art classical solvers. However, it requires a large number of quantum gates, even for small-scale problems, and hence, its performance is adversely affected by the noise inherent in current quantum computers [8]. Moreover, it does not allow to extract the full solution vector $\mathbf{z}$ efficiently but only a scalar key performance indicator of the form $\mathbf{z}^T M \mathbf{z}$, where $M$ is some sparse matrix. In [9], a variational quantum linear solver (VQLS) has been developed, which uses fewer quantum gates and, therefore, enables resilient PF using noisy quantum computers. However, there is no theoretical justification, yet, that VQLS might offer any computational speedup. In addition to addressing the solution of linear systems of equations, several studies have focused on quadratic unconstrained binary optimization (QUBO) model for specific applications, such as Optimal Power Flow (OPF) [10]. Note that QUBO model can be solved by GQC and AQC approaches, as presented in Table 4.1.

Motivated by the potential of emerging quantum devices, this chapter specifically

looks at GQC and QNNs. QNNs are a topic of active research in the broader field of quantum machine learning, which comprise layers of quantum gates. They are able to explore high-dimensional feature spaces with a limited number of quantum gates, which potentially leads to superior performance in practical applications [11]. In addition, the inherent randomness of quantum phenomena allows for the capture of complex relationships between inputs and output with reduced reliance on large datasets [12]. These advantages not only enhance the training process but also address data scarcity concerns and make QNNs a potential candidate for learning from small, incomplete, and/or noisy datasets [13].

Table 4.1: Literature on the use of quantum computing for PF.

| Paradigm | Algorithm* | Application† | Test System Size‡ | Hardware |
|---|---|---|---|---|
| GQC | HHL [14] | DCPF | 3-bus | IBM |
| | VQC [15] | TSA | 11-bus/NPCC | IBM |
| | HHL [5] | FDPF | 5-bus | - |
| | HHL [16] | ACPF | 3-bus/5-bus | IBM |
| | VQLS [9] | ACPF | 5-bus | IBM |
| | HHL [7] | SE | 2-bus/4-bus | - |
| | QAOA [17] | UC | 9-DER | IBM |
| | QAOA [18] | MS | 24-bus | - |
| | HHL [19] | DCOPF | 3-bus/14-bus | - |
| AQC | VQC [20] | FD | 30-bus | D-Wave |
| | QA [21] | UC | 2-TU | D-Wave |
| | QA [10] | ACOPF | 55-bus | D-Wave |

*HHL=Harrow-Hassidim-Lloyd algorithm, VQC=variational quantum circuits, VQLS=variational quantum linear solver, QAOA=quantum approximate optimization algorithm, QA=quantum annealing.
†DCPF=direct current power flow, TSA=transient stability assessment, FDPF=fast-decoupled power flow, ACPF=alternating current power flow, SE=state estimation, UC=unit commitment, MS=maximum sections of power delivery and data traffic, DCOPF=direct current optimal power flow, FD=fault detection, ACOPF=alternating current optimal power flow.
‡NPCC=Northeast Power Coordinating Council test system [22], DER=distributed energy resource, TU=thermal unit.

## 4.2. Model Architecture

The reader is referred to Section 2.3.1 for background on NN architecture. Parameterized quantum circuits (PQC) function as the core units rather than classical hidden layers for QNNs. Similar to NNs, QNNs can be fine-tuned through classical optimization processes to find the relationship between $\mathbf{x}$ as input features and $\mathbf{y}$ as output labels. $\mathbf{x}$ is initially encoded into a quantum state of multiple qubits, a phase known as feature map. Subsequently, the data is processed using PQCs and form an ansatz. The measurement of the PQCs is fed into a loss function. Optimization of QNNs can happen through gradient-based methods, facilitated by

the analytical techniques for measuring the gradient of the expectation of PQCs, as outlined in [23]. The representation of QNNs is given by:

$$|\psi^{\text{in}}\rangle = U(\mathbf{x})|0\ldots0\rangle, \quad |\psi^{\text{out}}\rangle = \omega(\mathbf{w}^r)|\psi^{\text{in}}\rangle, \tag{4.1}$$

where the input state $|\psi^{\text{in}}\rangle$ is created by transforming $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ into valid quantum states using the feature map $U(.)$ applied to the vacuum state $|0\ldots0\rangle$. The ansatz $\omega(.)$ is applied to $|\psi^{\text{in}}\rangle$. There are no parameters for the feature map that need to be tuned by the optimizer. In contrast, the vector of adjustable parameters of the ansatz $\mathbf{w}^r$ is fine-tuned through the training process using a dataset consisting of $N$ training pairs $\{\mathbf{x}, \mathbf{y}\}$. The resulting output state $|\psi^{\text{out}}\rangle$ cannot be read out directly but needs to be deduced by the measurement to obtain $\hat{\mathbf{y}} \in \{(\hat{\mathbf{V}}_i, \hat{\delta}_i) : i = 1, 2, \ldots, n\}$. We explore two distinct implementations of QNNs: a pure QNN and a hybrid quantum-classical neural network (QCNN), described in the following subsections.

Figure 4.1 shows an illustration of pure QNNs, which includes three fundamental components: a feature map, an ansatz, and the measurement. The feature map $U(\cdot)|0\ldots0\rangle$ transforms $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ into a vector of quantum states $|\psi^{\text{in}}\rangle$. The dimension of this vector corresponds to the number of qubits in the PQC, i.e., $n \times 2$, where $n$ is the number of buses. The ansatz $\omega_{(.)}|\psi^{\text{in}}\rangle$ takes $|\psi^{\text{in}}\rangle$ as input and applies a combination of quantum gates. Finally, the expectation value is determined, which is then subjected to additional post-processing to derive $\hat{\mathbf{y}} \in \{(\hat{\mathbf{V}}_i, \hat{\delta}_i) : i = 1, 2, \ldots, n\}$.

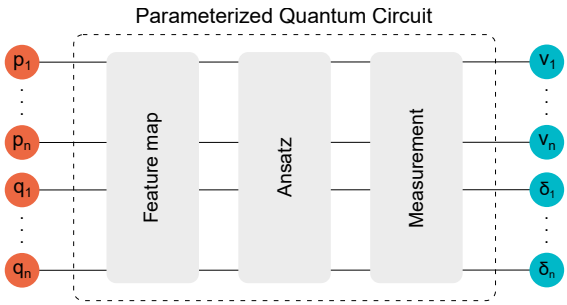



Figure 4.1: Schematic depiction of the QNN designed for PF analysis, including a feature map, an ansatz, and the measurement. $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ and $\mathbf{y} \in \{(\mathbf{V}_i, \delta_i) : i = 1, 2, \ldots, n\}$ respectively represent the input features and output labels, where $n$ is the number of buses.

For the QCNN, one or more of the hidden layers can be replaced by PQC(s), as shown in Figure 4.2. The data flow begins from a hidden layer, propagates through the feature map and ansatz, and then proceeds to the next hidden layer after the measurement. Note that the QCNN architecture can potentially resemble classical auto-encoders, where the first classical component encodes $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ to a lower dimensional latent space at which the QNN operates. Subsequently, the second classical

component decodes the measurement to a higher dimensional space to derive $\hat{\mathbf{y}} \in \{(\hat{\mathbf{V}}_i, \hat{\delta}_i) : i = 1, 2, \ldots, n\}$. This architecture has therefore the flexibility to accommodate larger input features and output labels for large-scale power systems, as it is not constrained by the number of available qubits.

In this study, two QCNNs are developed for the 4-bus and 33-bus test systems, each having a PQC as the quantum component sandwiched with classical hidden layers, as depicted in Figure 4.2. The number of classical hidden layers is the same as those of the corresponding NNs for the 4-bus and 33-bus test systems. The immediate hidden layer before and after the quantum component has $n \times 2$ neurons, where $n$ is the number of buses. Note that in this setting, the total number of calls to the quantum simulator matches that of the QNN as the qubit count, and the number of shots remains consistent.

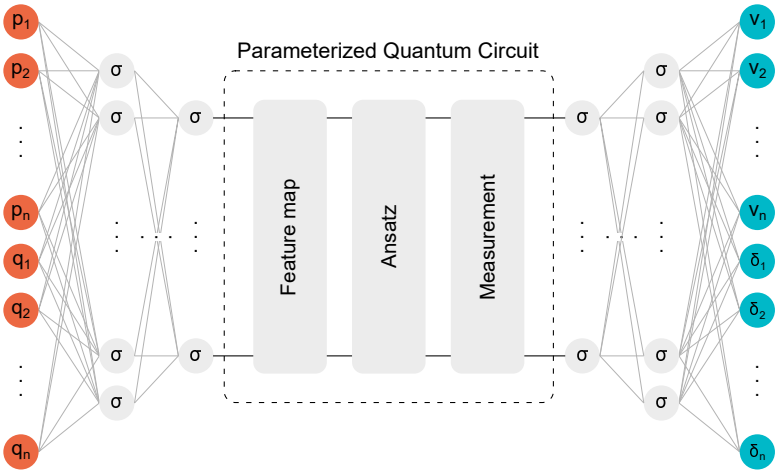



Figure 4.2: Schematic depiction of the QCNN designed for PF analysis with a PQC replacing one hidden layer. $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ and $\mathbf{y} \in \{(\mathbf{V}_i, \delta_i) : i = 1, 2, \ldots, n\}$ respectively represent the input features and output labels, where $n$ is the number of buses. The data flow starts from a hidden layer, passes through the PQC, and continues to the subsequent hidden layer after the measurement.

## 4.3. Results

All quantum components are implemented in `Qiskit` (version 0.46.0) and executed using the `Aer statevector` simulator on a classical computing system, specifically Ubuntu 22.04, with 16 physical cores and 64 GB of RAM. The simulations are made more realistic by introducing statistical noise and hardware noise. The same computer was used to obtain the ground truth data with the NR method. The focus is on a 4-bus test system [24]. Supplementary experiments are also performed on a 33-bus test system [25] to show the scalability and the potential

benefits of the QCNN for the PF problem. The chosen systems contain one *slack* bus with known $V$ and $\delta$ and unknown $P$ and $Q$, and $PQ$ buses for which $P$ and $Q$ are known, while $V$ and $\delta$ are unknown. Note that $PV$ buses are not considered in this work to maintain computational simplicity.

For the two test systems, the dataset comprises a total of 512 data points, which are randomly chosen from a pool of 5,000 samples. These samples are systematically generated based on calculated apparent power $S$ and power factor $pf$ for each bus. Initially, $P$ and $Q$ are known for a specific scenario of the test systems, as presented in [24, 25]. Subsequently, $S = \sqrt{P^2 + Q^2}$, and $pf = P/S$ are calculated for each bus. The calculated $S$ is considered as the mean and a deviation of 30% from $S$ is considered as the standard deviation to obtain a normal distribution with 5,000 samples for each bus. Finally, $P = S \times pf$ and $Q = \sqrt{S^2 - P^2}$ are calculated for all buses and all samples. This approach ensures that the datasets reflect the inherent variability in $P$ and $Q$ across the power system.

The input features and output labels of the datasets are respectively $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ and $\mathbf{y} \in \{(\mathbf{V}_i, \delta_i) : i = 1, 2, \ldots, n\}$, obtained from the NR. The dataset for each test system is divided into three subsets, allocating 25% for training, 25% for validation, and the remaining 50% for testing. The training dataset intentionally includes only 128 training data points to highlight the enhanced performance of the QNNs. The training process concludes after 1,000 epochs for the NN, QCNN, and QNN. The utilized loss function is the mean squared error (MSE) (2.9), the activation function is ReLU, and the Adam optimization algorithm is employed. The performance of the NN, QCNN, and QNN is systematically evaluated as follows:

**Generalization ability:** The MSE obtained for the testing dataset for the QNN and QCNN is 41% and 52% lower, respectively, compared to that of the NN.

**Robustness:** The influence of noisy training dataset is investigated by systematically introducing controlled levels of noise, ranging from 1% to 10%, to both $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ and $\mathbf{y} \in \{(\mathbf{V}_i, \delta_i) : i = 1, 2, \ldots, n\}$ of the training points. A certain percentage of the input features and output labels of the training dataset is randomly selected. The corrupted vectors are $\mathbf{x}' = \mathbf{x} \pm \mathbf{r}_x$ and $\mathbf{y}' = \mathbf{y} \pm \mathbf{r}_y$, where $\mathbf{r}_x$ and $\mathbf{r}_y$ are vectors of random values between 0 and 1, and 0 and 0.1, respectively. The investigation is based on the testing MSE. The QNN and QCNN exhibit superior robustness against the noisy training dataset compared to the NN, as shown in Figure 4.3. The performance of the NN significantly decreases by up to two times as the noise level increases.

**Training dataset size:** The impact of training dataset size on the performance of the NN is investigated. Figure 4.4 shows the changes in the MSE obtained for the testing dataset with varying training dataset sizes. The comparison is made relative to a constant curve, which represents the MSE obtained for the testing dataset for the QCNN using 128 training data points. It is observed that the NN necessitates a training dataset four times larger than that of the QCNN to achieve a performance level that still remains inferior.

**Training error:** The MSE obtained for the training dataset for the QNN and QCNN after 1,000 epochs is 48% and 54%, respectively, less than that of the NN.

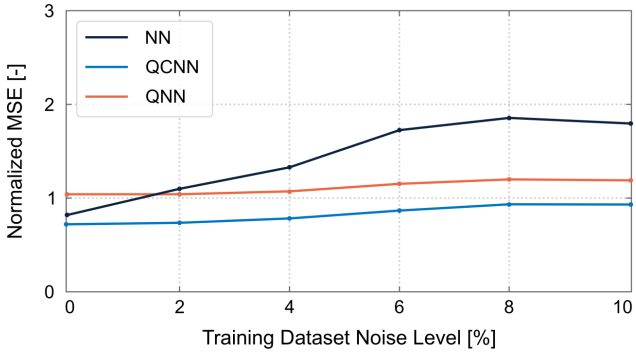



Figure 4.3: Illustration of the performance of the NN, QCNN, and QNN based on the MSE obtained for the testing dataset under varying levels of noise in the training dataset for the 4-bus test system. The MSE values are normalized relative to the testing MSE obtained for the QNN at the noise level of 10%.

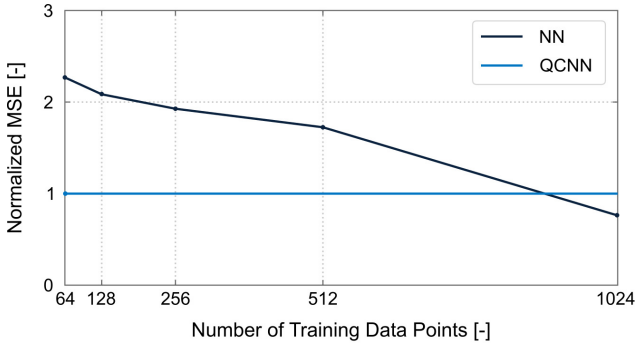



Figure 4.4: Illustration of the performance of the NN based on the MSE obtained for the testing dataset for the 4-bus test system under different training dataset sizes, compared to that of the QCNN with 128 training data points relative to which the MSE values are also normalized.

**Training process stability:** The mean and standard deviation of the MSE obtained over 1,000 epochs for the training dataset is evaluated. The QNN and QCNN show a respective reduction of 39% and 45% in the mean of the training MSE compared to the classical NN. Similarly, the standard deviation of the training MSE for the QNN and QCNN is lower by 25% and 34%, respectively, compared to the classical NN. Note that the differences in the training processes are due to the differences in the gradient calculation methods employed. For the classical NN and the classical component of the QCNN, the gradients are computed using backpropagation [26]. However, for the pure QNN and the quantum component of the QCNN, gradient calculation is achieved using the parameter shift rule [23].

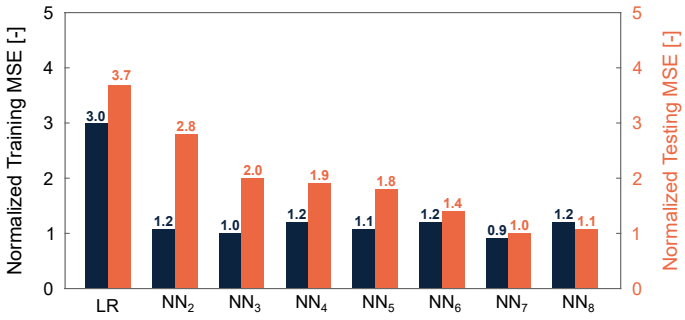



Figure 4.5: Illustration of the impact of the NN architecture depth on the training and testing MSE for the 4-bus test system. LR serves as a benchmark model. NN architectures with varying numbers of hidden layers, ranging from two ($NN_2$) to eight ($NN_8$), are assessed. The MSE values are normalized relative to the testing MSE obtained for the $NN_7$.

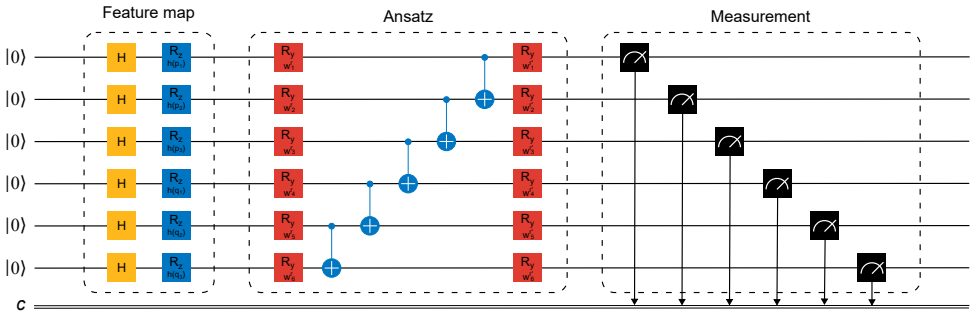



Figure 4.6: Detailed representation of the PQC shown in Figure 4.1 for the 4-bus test system. $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ is initially encoded into a quantum state of multiple qubits using the feature map. Then, the data is processed through the ansatz. The feature map utilizes the `Hadamard` gate $H$ and a rotation around the z-axis $R_z$ on individual qubits. The ansatz applies a y-rotation $R_y(\mathbf{w}_i^r)$, followed by a `CNOT` gate, and subsequently another y-rotation $R_y(\mathbf{w}_i^{r'})$ on each qubit. The expectation value for each qubit is then obtained within the interval of $[-1, 1]$ from the measurement in the Z-basis.

### 4.3.1. Sensitivity Analysis for NN

The architecture of the NN, i.e., the number of hidden layers and neurons per hidden layer, but also the hyper-parameters, i.e., the learning rate, the weight decay rate, and dropout percentage, are achieved through sensitivity analysis to ensure a

fair comparison between the NN, QCNN, and QNN[1]:

Seven different depths of NN architectures are explored to ascertain the architecture that enhances the generalization ability of the NN. Deep architectures with more than eight hidden layers are not considered due to the small size of both the test system and the training dataset. Across all architectures, the first and last hidden layers consist of $n \times 2$ neurons, while the intermediate hidden layers each contain $n \times 4$ neurons. A linear regression model (LR) is included in the evaluation as a benchmark. Figure 4.5 illustrates the MSE obtained for both the training and testing datasets. It is observed that as the number of hidden layers decreases, the generalization ability of the NN decreases.

An exploration of different learning rates, weight decay rates, and dropout percentages is conducted. The learning rates are explored across a range of values between $1 \times 10^{-1}$ and $1 \times 10^{-6}$. Wight decay rates are considered from $1 \times 10^{-1}$ to $1 \times 10^{-6}$. Dropout percentages are considered between 0% and 2%. The selection of the optimal values is based on the training and testing MSE, along with their relative proximity to ensure generalization ability.

The architecture with seven hidden layers is selected as the basis for evaluating the QCNN and QNN due to its lower MSE obtained for training and testing datasets. The proximity of the training and testing MSE also suggests a lack of overfitting in the proposed architecture. The NN is trained with a learning rate of $1.5 \times 10^{-4}$, a weight decay rate of $3 \times 10^{-3}$, and a dropout percentage of 0%, with a batch size of 16. The training and testing MSE obtained for the 4-bus test system and a noisy training dataset with 128 training data points are 2.41 and 2.83, respectively.

### 4.3.2. Convergence analysis of shots

A PQC serves as the core unit of the QNN architecture. It comprises a feature map, an ansatz, and the measurement, as illustrated in Figure 4.6. The feature map is responsible for translating the six input features into quantum states. The `ZFeatureMap` method from the `Qiskit` standard library is used, that is, the vacuum state $|000000\rangle$ passes through a layer of `Hadamard` gates $H$ before each qubit undergoes a z-rotation, i.e., $R_z(h(P, Q))$, where $h(\cdot)$ maps the input features $\mathbf{x} \in \{(\mathbf{P}_i, \mathbf{Q}_i) : i = 1, 2, \ldots, n\}$ to the interval $[-\frac{\pi}{2}, \frac{\pi}{2}]$ through the formula $\mathbf{x}' = \tan^{-1}(\mathbf{x})$. The `RealAmplitude` method from the `Qiskit` standard library is employed as the ansatz. This entails each qubit undergoing a y-rotation $R_y(\mathbf{w}_i^r) : i = 1, 2, \ldots, n \times 2$, followed by a `CNOT` gate, and another y-rotation $R_y(\mathbf{w}_i^{r'}) : i = 1, 2, \ldots, n \times 2$. Consequently, the ansatz has $n \times 4$ free parameters, where $n$ is the number of buses. From the measurement in the Z-basis, the expectation value for each qubit is obtained within the interval $[-1, 1]$. A classical post-processing conversion is then employed to map these values to problem-specific intervals. These intervals are determined based on the lower and upper bounds of the dataset with an additional margin to ensure coverage of extreme values. Accordingly, the intervals for $V$ and $\delta$ are determined as

[1] The `Optuna` Python package is used to conduct an exhaustive search across 2,000 unique hyperparameter combinations.

$[0.85, 1.15]$ and $[-8, 8]$, respectively. Details about the sensitivity analysis for the QNN are provided in the following subsections.

The error between the predictions obtained from the `Aer statevector` simulator and those obtained from the shot simulator is investigated, where each shot represents a repetition of the measurement. The results are shown in Figure 4.7. It can be observed that the error stabilizes when the shot count reaches 1,024 or higher. Therefore, a shot number of 1,024 is selected for the training of the QNN and the quantum component of the QCNN.

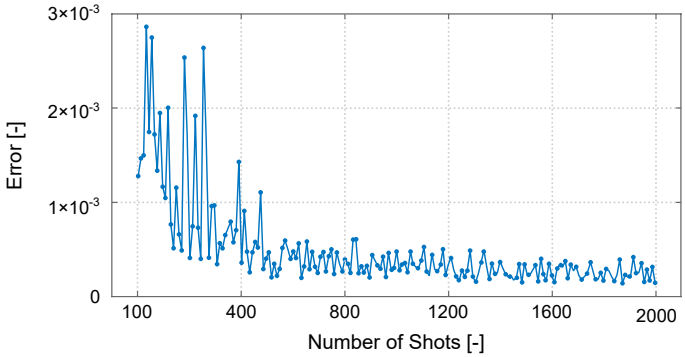



Figure 4.7: Illustration of the error between the predictions obtained from the `Aer statevector` simulator and those from the shot simulator as a function of the number of shots.

### 4.3.3. Impact of hardware noise

The impact of different sources of noise, including measurement error, gate imperfection, depolarizing error, and amplitude-damping error, is examined. This exploration simulates the real-world behavior of quantum hardware. The noise levels range from 0% to 10% and the analysis utilizes the dataset for the 4-bus test system. The results are depicted in Figure 4.8. Details about different error types can be found in Ref. [27].

### 4.3.4. Impact of the number of qubits

Increasing the number of qubits inherently complicates the training process, particularly for the QCNN which integrates both quantum and classical components. The impact of qubit count on the MSE obtained for the training and testing datasets is therefore studied to highlight the trade-offs between the model complexity and model performance. The analysis uses the dataset for the 4-bus test system. The results, shown in Figure 4.9, lead to the selection of six qubits for the QCNN.

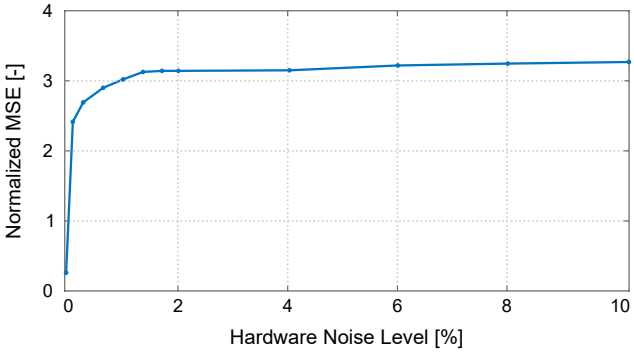



Figure 4.8: Illustration of the impact of various noise levels on the performance of the QNN for the 4-bus test system. Measurement error, gate imperfection, depolarizing error, and amplitude-damping error are considered.

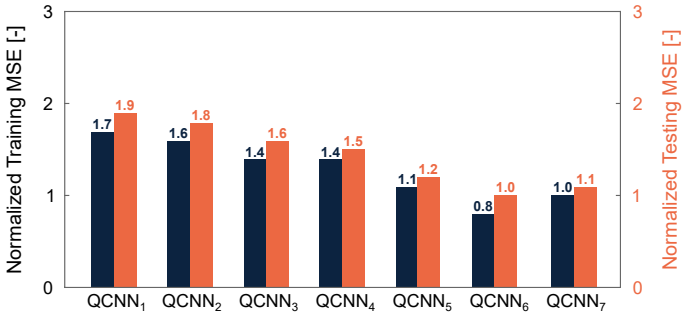



Figure 4.9: Illustration of the impact of the qubit count on the training and testing MSE for the 4-bus test system. Different QCNNs with varying numbers of qubits, ranging from one ($QCNN_1$) to seven ($QCNN_7$), are assessed. The MSE values are normalized relative to the testing MSE obtained for the $QCNN_6$.

### 4.3.5. PF Analysis

The mean and standard deviation of $\hat{\mathbf{y}} \in \{(\hat{\mathbf{V}}_i, \hat{\delta}_i) : i = 1, 2, \ldots, n\}$ approximated by the NN, QCNN, and QNN for the 4-bus test system are computed relative to the ground truth data $\mathbf{y} \in \{(\mathbf{V}_i, \delta_i) : i = 1, 2, \ldots, n\}$ obtained from the NR, see Table 4.2. A linear regression model (LR) is also included in the evaluation as a benchmark. The mean and standard deviation obtained for the QCNN exhibit the superior performance of the QCNN compared to the QNN, NN, and LR.

An investigation is also done to explore the performance of the NN and QCNN for the 33-bus test system under extreme conditions, where the power system experiences over-voltage or under-voltage situations. According to the results shown in Figure 4.10, the QCNN achieves a maximum MSE of $0.011$ pu for the testing dataset, while the NN obtains a maximum MSE of $0.032$ pu for the testing

Table 4.2: Performance comparison of the NN relative to the results obtained by the NR method for the 4-bus test system.

| **Approach** | $\overline{\lvert\Delta V_i\rvert}$ | $\sigma_{\lvert\Delta V_i\rvert}$ | $\overline{\lvert\Delta\delta_i\rvert}$ | $\sigma_{\lvert\Delta\delta_i\rvert}$ |
|---|---|---|---|---|
| LR | $8.85 \times 10^{-1}$ | $6.74 \times 10^{-2}$ | $9.53 \times 10^{-1}$ | $4.02 \times 10^{-2}$ |
| NN | $1.44 \times 10^{-1}$ | $4.02 \times 10^{-2}$ | $3.18 \times 10^{-1}$ | $3.24 \times 10^{-2}$ |
| QCNN | $2.67 \times 10^{-3}$ | $1.03 \times 10^{-2}$ | $1.23 \times 10^{-2}$ | $2.4 \times 10^{-2}$ |
| QNN | $6.1 \times 10^{-3}$ | $3.08 \times 10^{-2}$ | $2.26 \times 10^{-2}$ | $2.98 \times 10^{-2}$ |

dataset, compared to the ground truth data obtained by the NR. This improvement is significant for power system operations, where unexpected events can induce deviations in $v$ from the nominal value.

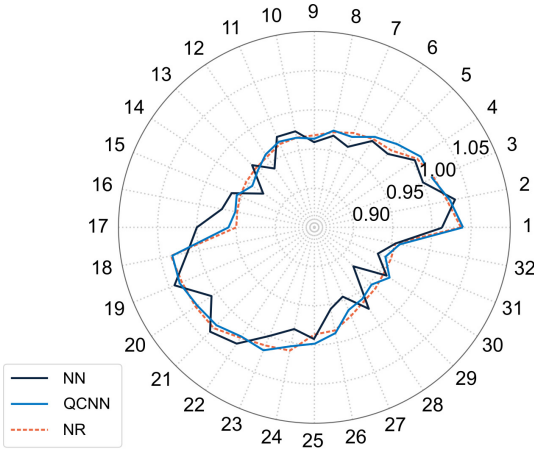



Figure 4.10: Comparison of $\mathbf{V}$ obtained from the NR and $\hat{\mathbf{V}}$ obtained from the NN and QCNN for the 33-bus test system under extreme conditions.

## 4.4. Discussion and Conclusions

The results position QNNs not as replacements for traditional solvers such as NR, but as surrogate models that learn the input–output mapping through linear and nonlinear transformations. Framed this way, the chapter is less about matching NR's exact computation than about whether a QNN recovers competitive accuracy from fewer and/or noisier samples. While the same inputs and outputs can be shared by traditional solvers, e.g., the NR, and surrogate models, e.g., NNs, their differences lie in the computational processes for deriving outputs from the provided inputs. Generally, the success of a surrogate model depends on the size and quality of historical data. In contrast, traditional solvers rely on mathematical models to obtain outputs from inputs. Note that the output of QNNs can serve as an initial estimate,

which can then be further refined by a few iterations of the NR method to converge to a solution sufficiently quickly.

Quantum simulations are conducted using the `Aer statevector` simulator of `Qiskit`. However, real quantum hardware, such as that provided by IBM, remains an available option for future research and experimentation. Note that the current high noise levels in today's NISQ hardware make it impractical to implement the proposed QNN approaches. However, it is expected that as NISQ hardware evolves and early FT computers with reduced noise levels and a moderate number of logical qubits emerge, the proposed approaches will become viable for successful execution.

Although this work demonstrates the potential of QNNs for the PF problem and their superior performance compared to classical NNs, further investigation is needed to fully understand the added value of these complex approaches. Note also that the superior performance of the QCNN in terms of generalization, robustness, training dataset size needed, training error, and training process stability compared to the classical NN can be attributed to the capacity of the QCNN to leverage the strengths of both classical and quantum paradigms, which makes it a more robust choice, particularly in the NISQ era. On the other hand, the limited performance of the classical NN can be attributed to the limited number of training data points and the noise introduced into the training dataset. An extension is to compare QNNs directly with the physics-informed neural network of Chapter 3, as both target the same limited, noisy data.

## References

[1] **Z. Kaseb**, M. Möller, G. T. Balducci, P. Palensky and P. P. Vergara. 'Quantum neural networks for power flow analysis'. In: *Electric Power Systems Research* 235 (2024), p. 110677. DOI: 10.1016/j.epsr.2024.110677.

[2] T. Pham and X. Li. 'Neural Network-based Power Flow Model'. In: *2022 IEEE Green Technologies Conference (GreenTech)*. ISBN: 978-1-6654-6768-1. DOI: 10.1109/GreenTech52845.2022.9772026.

[3] B. Donon, R. Clément, B. Donnot, A. Marot, I. Guyon and M. Schoenauer. 'Neural networks for power flow: Graph neural solver'. In: *Electric Power Systems Research* 189 (2020). ISSN: 03787796. DOI: 10.1016/j.epsr.2020.106547.

[4] S. M. Miraftabzadeh, F. Foiadelli, M. Longo and M. Pasetti. 'A Survey of Machine Learning Applications for Power System Analytics'. In: *2019 IEEE International Conference on Environment and Electrical Engineering (EEEIC)*. ISBN: 978-1-7281-0653-3. DOI: 10.1109/EEEIC.2019.8783340.

[5] F. Feng, Y. Zhou and P. Zhang. 'Quantum Power Flow'. In: *IEEE Transactions on Power Systems* 36 (2021). ISSN: 0885-8950. DOI: 10.1109/TPWRS.2021.3077382.

[6] L. von Rueden, S. Mayer, K. Beckh, B. Georgiev, S. Giesselbach, R. Heese, B. Kirsch, M. Walczak, J. Pfrommer, A. Pick, R. Ramamurthy, J. Garcke, C. Bauckhage and J. Schuecker. 'Informed Machine Learning - A Taxonomy and Survey of Integrating Prior Knowledge into Learning Systems'. In: *IEEE Transactions on Knowledge and Data Engineering* (2021). ISSN: 1041-4347. DOI: 10.1109/TKDE.2021.3079836.

[7] F. Feng, P. Zhang, Y. Zhou and Z. Tang. 'Quantum microgrid state estimation'. In: *Electric Power Systems Research* 212 (2022). ISSN: 03787796. DOI: 10.1016/j.epsr.2022.108386.

[8] S. Golestan, M. Habibi, S. Mousazadeh Mousavi, J. Guerrero and J. Vasquez. 'Quantum computation in power systems: An overview of recent advances'. In: *Energy Reports* 9 (2023). ISSN: 23524847. DOI: 10.1016/j.egyr.2022.11.185.

[9] F. Feng, Y. Zhou and P. Zhang. 'Noise-Resilient Quantum Power Flow'. In: *arXiv preprint* (2022). DOI: arXiv:2211.10555.

[10] T. Morstyn. 'Annealing-Based Quantum Computing for Combinatorial Optimal Power Flow'. In: *IEEE Transactions on Smart Grid* 14 (2023). ISSN: 1949-3053. DOI: 10.1109/TSG.2022.3200590.

[11] M. Schuld, I. Sinayskiy and F. Petruccione. 'The quest for a Quantum Neural Network'. In: *Quantum Information Processing* 13 (2014). ISSN: 1570-0755. DOI: 10.1007/s11128-014-0809-8.

[12] K. Beer, D. Bondarenko, T. Farrelly, T. J. Osborne, R. Salzmann, D. Scheiermann and R. Wolf. 'Training deep quantum neural networks'. In: *Nature Communications* 11 (2020). ISSN: 2041-1723. DOI: 10.1038/s41467-020-14454-2.

[13] K. Beer. 'Quantum neural networks'. In: *Institutionelles Repositorium der Leibniz Universitat Hannover* (2022). DOI: 10.15488/11896.

[14] R. Eskandarpour, K. Ghosh, A. Khodaei, L. Zhang, A. Paaso and S. Bahramirad. 'Quantum Computing Solution of DC Power Flow'. In: *arXiv preprint* (2020). DOI: arXiv:2010.02442.

[15] Y. Zhou and P. Zhang. 'Noise-Resilient Quantum Machine Learning for Stability Assessment of Power Systems'. In: *arXiv preprint* (2021). DOI: arXiv:2104.04855.

[16] B. Sævarsson, S. Chatzivasileiadis, H. Jóhannsson and J. Østergaard. 'Quantum Computing for Power Flow Algorithms: Testing on real Quantum Computers'. In: *arXiv preprint* (2022). DOI: arXiv:2204.14028.

[17] N. Nikmehr, P. Zhang and M. A. Bragin. 'Quantum Distributed Unit Commitment: An Application in Microgrids'. In: *IEEE Transactions on Power Systems* 37 (2022). ISSN: 0885-8950. DOI: 10.1109/TPWRS.2022.3141794.

[18] H. Jing, Y. Wang and Y. Li. 'Data-driven quantum approximate optimization algorithm for power systems'. In: *Communications Engineering* 2 (2023). ISSN: 2731-3395. DOI: 10.1038/s44172-023-00061-8.

[19] F. Amani, R. Mahroo and A. Kargarian. 'Quantum-Enhanced DC Optimal Power Flow'. In: *2023 IEEE Texas Power and Energy Conference (TPEC)*. ISBN: 978-1-6654-9071-9. DOI: 10.1109/TPEC56611.2023.10078612.

[20] A. Ajagekar and F. You. 'Quantum computing based hybrid deep learning for fault diagnosis in electrical power systems'. In: *Applied Energy* 303 (2021). ISSN: 03062619. DOI: 10.1016/j.apenergy.2021.117628.

[21] P. Halffmann, P. Holzer, K. Plociennik and M. Trebing. 'A Quantum Computing Approach for the Unit Commitment Problem'. In: *arXiv preprint* (2022). DOI: arXiv:2212.06480.

[22] P. W. Sauer, M. A. Pai and J. H. Chow. *Power System Dynamics and Stability with Synchrophasor Measurement and Power System Toolbox*. Wiley, 2017. ISBN: 978-1-119-35575-5.

[23] K. Mitarai, M. Negoro, M. Kitagawa and K. Fujii. 'Quantum circuit learning'. In: *Physical Review A* 98 (2018). ISSN: 2469-9926. DOI: 10.1103/PhysRevA.98.032309.

[24] J. J. Grainger and W. D. Stevenson Jr. *Power System Analysis*. McGraw-Hill, Inc., 1994.

[25] M. Baran and F. Wu. 'Network reconfiguration in distribution systems for loss reduction and load balancing'. In: *IEEE Transactions on Power Delivery* 4 (1989). ISSN: 0885-8977. DOI: 10.1109/61.25627.

[26] J. Fang, M. Xu, H. Chen, B. Shuai, Z. Tu and J. Tighe. 'An In-depth Study of Stochastic Backpropagation'. In: *arXiv preprint* (2022). DOI: arXiv:2210.00129.

[27] H.-L. Huang, X.-Y. Xu, C. Guo, G. Tian, S.-J. Wei, X. Sun, W.-S. Bao and G.-L. Long. 'Near-term quantum computing techniques: Variational quantum algorithms, error mitigation, circuit compilation, benchmarking and classical simulation'. In: *Science China Physics, Mechanics & Astronomy* 66 (2023). ISSN: 1674-7348. DOI: 10.1007/s11433-022-2057-y.

# 5

# Quantum Optimization Algorithms for (Optimal) Power Flow

*We present a combinatorial reformulation of the power flow (PF) and optimal power flow (OPF) problems to address scalability and convergence limitations associated with classical solvers. This reformulation enables the construction of both an Ising model and/or an equivalent quadratic unconstrained binary optimization (QUBO) model. These models are solved using quantum and quantum-inspired annealers. A quantum approximate optimization algorithm (QAOA) is also developed for execution on gate-based quantum hardware. The reader is referred to Sections 2.1 and 2.2 for background on PF, OPF, gate-based quantum computing (GQC), and adiabatic quantum computing (AQC), upon which this chapter builds.*

Parts of this chapter have been published in Nature Scientific Reports **14**, 1 (2024) with the title: "Power flow analysis using quantum and digital annealers: a discrete combinatorial optimization approach", doi: 10.1038/s41598-024-73512-7 [1], and as arXiv preprint **2505.15978** (2026) with the title: "Power flow and optimal power flow using quantum and digital annealers: a computational scalability analysis" [2].

## 5.1. Motivation

To address the unsolved challenges of Power Flow (PF) and Optimal Power Flow (OPF) problems, our research has investigated reformulating them as combinatorial optimization problems [1–3]. The reader is referred to Chapter 1 for detailed information on the convergence and scalability challenges of classical solvers. By discretizing complex bus voltages using spin or binary decision variables, the nonlinear equations are embedded into an Ising model or a quadratic unconstrained binary optimization (QUBO) model. This approach enables the use of discrete optimization solvers but comes at the cost of transforming the problems into an NP-hard problem [4]. As system size increases, the number of decision variables grows rapidly, thus making exhaustive search infeasible. Furthermore, the discretization process introduces higher-order polynomial terms that must be quadratized, increasing the dimensionality and complexity of the resulting optimization landscape. Therefore, while the combinatorial formulation offers an alternative viewpoint, it yields a large-scale, NP-hard optimization problem that is challenging to solve using classical solvers.

Quantum computing (QC) has emerged as a promising paradigm for tackling complex combinatorial optimization problems. Quantum algorithms can explore exponentially large search spaces more efficiently than classical heuristics [5]. QC paradigms can be broadly categorized into two main types: gate-based quantum computing (GQC) and adiabatic quantum computing (AQC). The reader is referred to Section 2.2 for a detailed description, as well as the similarities and differences between GQC and AQC. On GQC hardware, the Quantum Approximate Optimization Algorithm (QAOA) is the most widely used framework for encoding combinatorial costs into parameterized quantum circuits. Although theoretically capable of outperforming classical heuristics, QAOA has so far been demonstrated only on small-scale problems, often using simulators [6]. Comparative studies increasingly suggest that, for current hardware generations, Ising machines can exhibit better performance and scalability for certain classes of optimization tasks [7]. They include D-Wave's Advantage™ quantum annealer (QA)[1] with several thousand superconducting qubits, D-Wave's quantum-classical hybrid annealer (HA), Fujitsu's digital annealer version 3 (DAv3), and Fujitsu's Quantum-Inspired Integrated Optimization software (QIIO)[2] capable of simulating annealing on tens of thousands of fully connected logical variables. While these devices do not guarantee exact optimality, they often return high-quality low-energy states suitable for practical combinatorial optimization.

Motivated by these developments, this chapter investigates the application of quantum and quantum-inspired optimization algorithms to combinatorial PF and OPF problems. We discretize the equations into Ising and QUBO Models and solve them using the PennyLane `lightning.qubit` statevector simulator and multiple Ising machines. We extend the proposed algorithm to large-scale and ill-conditioned scenarios for PF and OPF, including systems up to 1354 buses.

[1] www.dwavequantum.com
[2] en-portal.research.global.fujitsu.com/kozuchi

## 5.2. Combinatorial PF

PF equations in rectangular coordinates, see (2.3), can be re-ordered, as:

$$P_i = \sum_{k=1}^{N} \mu_i G_{ik} \mu_k + \omega_i G_{ik} \omega_k + \omega_i B_{ik} \mu_k - \mu_i B_{ik} \omega_k, \tag{5.1a}$$

$$Q_i = \sum_{k=1}^{N} \omega_i G_{ik} \mu_k - \mu_i G_{ik} \omega_k - \mu_i B_{ik} \mu_k - \omega_i B_{ik} \omega_k, \tag{5.1b}$$

where $\mu_i$ and $\omega_i$ are continuous real numbers and must be discretized. One possible approach is to introduce multiple decision variables per bus $i$, each associated with a predefined increment. These variables determine whether the increment is added to or subtracted from a given base value for $\mu_i$ and $\omega_i$. While this method provides a fine-grained search space, it requires a large number of decision variables per bus $i$ for both $\mu_i$ and $\omega_i$, which significantly increases the dimensionality of the combinatorial problem.

### 5.2.1. QUBO Model

An alternative approach for discretization is to assign two binary decision variables to each $\mu_i$ and each $\omega_i$, while iteratively refining their base values $\mu_i^0$ and $\omega_i^0$, leading to a higher-order unconstrained binary optimization (HUBO) model, which is ultimately a QUBO model after order reduction. A QUBO model is defined by its symmetric, real-valued matrix $Q \in \mathbb{R}^{n \times n}$ and the binary minimization problem:

$$\min_{\mathbf{x} \in \{0,1\}^n} \sum_{i=1}^{n} \sum_{j=1}^{n} Q_{ij} x_i x_j. \tag{5.2}$$

$\mu_i$ and $\omega_i$ are encoded as a base value offset by scaled binary variables:

$$\mu_i = \mu_i^0 + \Delta\mu_i (x_{i,0}^{\mu} - x_{i,1}^{\mu}), \tag{5.3a}$$

$$\omega_i = \omega_i^0 + \Delta\omega_i (x_{i,0}^{\omega} - x_{i,1}^{\omega}), \tag{5.3b}$$

where the variables $x_{i,\{0,1\}}^{\{\mu,\omega\}} \in \{0, 1\}^{4N}$ are *binary* indicators that control deviations from the reference values $\mu_i^0$ and $\omega_i^0$. An increase occurs when $x_{i,0} = 1$ and $x_{i,1} = 0$, a decreased when $x_{i,0} = 0$ and $x_{i,1} = 1$, and no change if both bits are equal. While the case $\{00\}$ and $\{11\}$ introduce redundancy in representing no change, allowing all four bit combinations $\{00, 01, 10, 11\}$ maintains a complete and unconstrained binary encoding space.

By inserting (5.3) into (5.1a), we derive an expression for the net active power

injection $P_i$:

$$\begin{aligned}
P_i = &\sum_{k=0}^{N}\Big[\mu_i^0 G_{ik}\mu_k^0 + \omega_i^0 G_{ik}\omega_k^0 + \omega_i^0 B_{ik}\mu_k^0 - \mu_i^0 B_{ik}\omega_k^0\Big] \\
&+\sum_{k=0}^{N}\sum_{l=0}^{1}\Big[(-1)^l\mu_i^0 G_{ik}x_{k,l}^{\mu}\Delta\mu_k + (-1)^l x_{i,l}^{\mu}\Delta\mu_i G_{ik}\mu_k^0 \\
&\quad +(-1)^l\omega_i^0 G_{ik}x_{k,l}^{\omega}\Delta\omega_k + (-1)^l x_{i,l}^{\omega}\Delta\omega_i G_{ik}\omega_k^0 \\
&\quad +(-1)^l\omega_i^0 B_{ik}x_{k,l}^{\mu}\Delta\mu_k + (-1)^l x_{i,l}^{\omega}\Delta\omega_i B_{ik}\mu_k^0 \\
&\quad -(-1)^l\mu_i^0 B_{ik}x_{k,l}^{\omega}\Delta\omega_k - (-1)^l x_{i,l}^{\mu}\Delta\mu_i B_{ik}\omega_k^0\Big] \\
&+\sum_{k=0}^{N}\sum_{l=0}^{1}\sum_{m=0}^{1}\Big[(-1)^{l+m}x_{i,l}^{\mu}\Delta\mu_i G_{ik}x_{k,m}^{\mu}\Delta\mu_k \\
&\quad +(-1)^{l+m}x_{i,l}^{\omega}\Delta\omega_i G_{ik}x_{k,m}^{\omega}\Delta\omega_k \\
&\quad +(-1)^{l+m}x_{i,l}^{\omega}\Delta\omega_i B_{ik}x_{k,m}^{\mu}\Delta\mu_k \\
&\quad -(-1)^{l+m}x_{i,l}^{\mu}\Delta\mu_i B_{ik}x_{k,m}^{\omega}\Delta\omega_k\Big].
\end{aligned} \tag{5.4}$$

A similar expression for the net reactive power $Q_i$ is obtained by inserting (5.3) into (5.1b):

$$\begin{aligned}
Q_i = &\sum_{k=0}^{N}\Big[\omega_i^0 G_{ik}\mu_k^0 - \mu_i^0 G_{ik}\omega_k^0 - \mu_i^0 B_{ik}\mu_k^0 - \omega_i^0 B_{ik}\omega_k^0\Big] \\
&+\sum_{k=0}^{N}\sum_{l=0}^{1}\Big[(-1)^l\omega_i^0 G_{ik}x_{k,l}^{\mu}\Delta\mu_k + (-1)^l x_{i,l}^{\omega}\Delta\omega_i G_{ik}\mu_k^0 \\
&\quad -(-1)^l\mu_i^0 G_{ik}x_{k,l}^{\omega}\Delta\omega_k - (-1)^l x_{i,l}^{\mu}\Delta\mu_i G_{ik}\omega_k^0 \\
&\quad -(-1)^l\mu_i^0 B_{ik}x_{k,l}^{\mu}\Delta\mu_k - (-1)^l x_{i,l}^{\mu}\Delta\mu_i B_{ik}\mu_k^0 \\
&\quad -(-1)^l\omega_i^0 B_{ik}x_{k,l}^{\omega}\Delta\omega_k - (-1)^l x_{i,l}^{\omega}\Delta\omega_i B_{ik}\omega_k^0\Big] \\
&+\sum_{k=0}^{N}\sum_{l=0}^{1}\sum_{m=0}^{1}\Big[(-1)^{l+m}x_{i,l}^{\omega}\Delta\omega_i G_{ik}x_{k,m}^{\mu}\Delta\mu_k \\
&\quad -(-1)^{l+m}x_{i,l}^{\mu}\Delta\mu_i G_{ik}x_{k,m}^{\omega}\Delta\omega_k \\
&\quad -(-1)^{l+m}x_{i,l}^{\mu}\Delta\mu_i B_{ik}x_{k,m}^{\mu}\Delta\mu_k \\
&\quad -(-1)^{l+m}x_{i,l}^{\omega}\Delta\omega_i B_{ik}x_{k,m}^{\omega}\Delta\omega_k\Big].
\end{aligned} \tag{5.5}$$

The ansatz presented in (5.4) and (5.5) yields a HUBO model enabling the

minimization of the squared mismatch terms in (2.5), as:

$$\min_{\mathbf{x}\in\{0,1\}^{4N}} \sum_{i=1}^{N} \left(P_i - P_i^{\mathrm{G}} + P_i^{\mathrm{D}}\right)^2 + \left(Q_i - Q_i^{\mathrm{G}} + Q_i^{\mathrm{D}}\right)^2, \tag{5.6}$$

where $\mathbf{x}$ is the vector of binary decision variables.

### 5.2.2. Ising Model

An alternative, more efficient approach for discretization is to assign a single spin decision variable to each $\mu_i$ and each $\omega_i$, while iteratively refining their base values $\mu_i^0$ and $\omega_i^0$, leading to an Ising model [8], which is formulated as the problem of finding the spin decision variable vector that minimizes:

$$\min_{\mathbf{s}\in\{\pm 1\}^n} \sum_{i=1}^{n} h_i s_i + \sum_{\langle i,j\rangle} J_{ij} s_i s_j, \tag{5.7}$$

where $h_i$ represents an external field that acts as a linear bias, influencing the tendency of $s_i$ toward $+1$ or $-1$, and $J_{ij}$ is the interaction coefficient between spins $i$ and $j$. $\langle i,j\rangle$ denotes all unique pairs of spins with $i<j$.

$\mu_i$ and $\omega_i$ are encoded as a base value offset by scaled spin variables:

$$\mu_i := \mu_i^0 + s_i^{\mu}\,\Delta\mu_i, \tag{5.8a}$$

$$\omega_i := \omega_i^0 + s_i^{\omega}\,\Delta\omega_i, \tag{5.8b}$$

where $s_i^{\{\mu,\omega\}} \in \{\pm 1\}^{2N}$ determine whether the base values, $\mu_i^0$ and $\omega_i^0$, are increased or decreased per iteration. In doing so, $\mu_i$ and $\omega_i$ are then iteratively updated.

Substituting (5.8) into (5.1a) yields:

$$\begin{aligned}
P_i = \sum_{k=1}^{N} \Big[ & \mu_i^0 G_{ik} \mu_k^0 + \omega_i^0 G_{ik} \omega_k^0 + \omega_i^0 B_{ik} \mu_k^0 - \mu_i^0 B_{ik} \omega_k^0 \Big] \\
& + \Big[ \mu_i^0 G_{ik} s_k^{\mu} \Delta\mu_k + s_i^{\mu} \Delta\mu_i G_{ik} \mu_k^0 + \omega_i^0 G_{ik} s_k^{\omega} \Delta\omega_k \\
& + s_i^{\omega} \Delta\omega_i G_{ik} \omega_k^0 + \omega_i^0 B_{ik} s_k^{\mu} \Delta\mu_k + s_i^{\omega} \Delta\omega_i B_{ik} \mu_k^0 \\
& - \mu_i^0 B_{ik} s_k^{\omega} \Delta\omega_k - s_i^{\mu} \Delta\mu_i B_{ik} \omega_k^0 \Big] \\
& + \Big[ s_i^{\mu} \Delta\mu_i G_{ik} s_k^{\mu} \Delta\mu_k + s_i^{\omega} \Delta\omega_i G_{ik} s_k^{\omega} \Delta\omega_k \\
& + s_i^{\omega} \Delta\omega_i B_{ik} s_k^{\mu} \Delta\mu_k - s_i^{\mu} \Delta\mu_i B_{ik} s_k^{\omega} \Delta\omega_k \Big].
\end{aligned} \tag{5.9}$$

Similarly, substituting (5.8) into (5.1b) yields:

$$\begin{aligned} Q_i = &\sum_{k=1}^{N}\Big[\omega_i^0 G_{ik}\mu_k^0 - \mu_i^0 G_{ik}\omega_k^0 - \mu_i^0 B_{ik}\mu_k^0 - \omega_i^0 B_{ik}\omega_k^0\Big] \\ &+\Big[\omega_i^0 G_{ik}s_k^\mu\Delta\mu_k + s_i^\omega\Delta\omega_i G_{ik}\mu_k^0 - \mu_i^0 G_{ik}s_k^\omega\Delta\omega_k \\ &-s_i^\mu\Delta\mu_i G_{ik}\omega_k^0 - \mu_i^0 B_{ik}s_k^\mu\Delta\mu_k - s_i^\mu\Delta\mu_i B_{ik}\mu_k^0 \\ &-\omega_i^0 B_{ik}s_k^\omega\Delta\omega_k - s_i^\omega\Delta\omega_i B_{ik}\omega_k^0\Big] \\ &+\Big[s_i^\omega\Delta\omega_i G_{ik}s_k^\mu\Delta\mu_k - s_i^\mu\Delta\mu_i G_{ik}s_k^\omega\Delta\omega_k \\ &-s_i^\mu\Delta\mu_i B_{ik}s_k^\mu\Delta\mu_k - s_i^\omega\Delta\omega_i B_{ik}s_k^\omega\Delta\omega_k\Big]. \end{aligned} \tag{5.10}$$

To further obtain a combinatorial optimization problem suitable for an Ising model, (2.5) is recast as the minimization of the sum of squared residuals for all terms:

$$\min_{\mathbf{s}\in\{\pm1\}^{2N}} \sum_{i=1}^{N}\left(P_i - P_i^{\mathrm{G}} + P_i^{\mathrm{D}}\right)^2 + \left(Q_i - Q_i^{\mathrm{G}} + Q_i^{\mathrm{D}}\right)^2, \tag{5.11}$$

where $\mathbf{s}$ is a vector of spin decision variables. It should be noted that (5.11) is not an Ising model until order reduction is performed.

## 5.3. Combinatorial OPF

Power balance equations (2.5) become equality constraints in the OPF problem, see (2.8a) and (2.8b), and, consequently, (5.6) becomes a penalty term in the QUBO model of the OPF problem, $\lambda_{\mathrm{PF}}H(\mathbf{x})$, where $\lambda_{\mathrm{PF}}$ is the corresponding penalty parameter. Another penalty term is needed to implement the inequality constraint terms, $H_{\mathrm{const}}(\mathbf{x})$, as well as the objective function of the OPF problem as an equality constraint term, $H_{\mathrm{cost}}(\mathbf{x})$.

Let us start by describing the Lagrange multiplier approach, which is used to impose equality constraints in a weak sense in a QUBO model. To this end, consider the following minimization problem with a single equality constraint:

$$\min_x \quad f(x) \tag{5.12a}$$

$$s.t. \quad g(x) = c. \tag{5.12b}$$

An equivalent formulation of the problem reads as follows:

$$\min_x \quad f(x) + \lambda\left(g(x) - c\right)^2, \quad \lambda > 0, \tag{5.13}$$

where $\lambda$ is a penalty parameter that ensures that any violation of the equality constraint incurs a cost in the QUBO model. A set of $m$ equality constraints on $n$

primary variables can be written in matrix form as follows:

$$\mathbf{Cx} = \mathbf{c}, \quad \mathbf{C} \in \mathbb{R}^{m \times n}, \quad \mathbf{c} \in \mathbb{R}^{m}, \tag{5.14}$$

and imposed via the Lagrange multiplier approach as:

$$P(\mathbf{x}) = \sum_{i=1}^{m} \lambda_i \left( \sum_{j=1}^{n} C_{ij} x_j - c_i \right)^2, \quad \lambda_i > 0. \tag{5.15}$$

Next, we describe the procedure for converting inequality constraints into equality constraints by adopting slack variables. To this end, consider the general minimization problem with a single inequality constraint:

$$\min_{x} \quad f(x) \tag{5.16a}$$

$$s.t. \quad h(x) \leq 0, \tag{5.16b}$$

and introduce the non-negative slack variable $s \in \mathbb{R} \geq 0$ to obtain an optimization problem with equality constraint:

$$\min_{x} \quad f(x) \tag{5.17a}$$

$$s.t. \quad h(x) + s = 0 \quad s \geq 0. \tag{5.17b}$$

To enforce $m$ inequality constraints on $n$ binary variables:

$$\mathbf{Dx} \leq \mathbf{d}, \quad \mathbf{D} \in \mathbb{R}^{m \times n}, \quad \mathbf{d} \in \mathbb{R}^{m}, \tag{5.18}$$

we introduce a vector of slack variables $\mathbf{s} \in \mathbb{R}^{m}_{\geq 0}$ such that:

$$\mathbf{Dx} + \mathbf{s} = \mathbf{d}, \quad \mathbf{s} \geq \mathbf{0}. \tag{5.19}$$

Since QUBO models require binary variables, we represent each slack variable using binary encoding:

$$s_i = \sum_{l=0}^{K} 2^l a_{i,l}, \quad a_{i,l} \in \{0, 1\}, \tag{5.20}$$

where $a_{i,l}$ are binary variables representing the discretized values of the slack variable $s_i$. By substituting expression (5.20) into (5.19), the original inequality constraints can be transformed into an equivalent QUBO formulation:

$$P(\mathbf{x}, \mathbf{a}) = \sum_{i=1}^{m} \lambda_i \left( \sum_{j=1}^{n} D_{ij} x_j + s_i - d_i \right)^2, \quad \lambda_i > 0. \tag{5.21}$$

Utilizing the slack variable approach, the inequality constraints (2.8c)–(2.8f) can be converted into equality constraints:

$$P_i^G = \underline{P_i^G} + \Delta P \cdot s_{P_i^+}, \quad s_{P_i^+} \geq 0, \tag{5.22a}$$

$$P_i^G = \overline{P_i^G} - \Delta P \cdot s_{P_i^-}, \quad s_{P_i^-} \geq 0, \tag{5.22b}$$

$$Q_i^G = \underline{Q_i^G} + \Delta Q \cdot s_{Q_i^+}, \quad s_{Q_i^+} \geq 0, \tag{5.22c}$$

$$Q_i^G = \overline{Q_i^G} - \Delta Q \cdot s_{Q_i^-}, \quad s_{Q_i^-} \geq 0, \tag{5.22d}$$

$$V_i = \underline{V_i} + \Delta V \cdot s_{V_i^+}, \quad s_{V_i^+} \geq 0, \tag{5.22e}$$

$$V_i = \overline{V_i} - \Delta V \cdot s_{V_i^-}, \quad s_{V_i^-} \geq 0, \tag{5.22f}$$

$$\delta_i = \underline{\delta_i} + \Delta\delta \cdot s_{\delta_i^+}, \quad s_{\delta_i^+} \geq 0, \tag{5.22g}$$

$$\delta_i = \overline{\delta_i} - \Delta\delta \cdot s_{\delta_i^-}, \quad s_{\delta_i^-} \geq 0, \tag{5.22h}$$

which can be imposed as penalty terms to ensure that the active and reactive power generation limits, voltage limits, and angle limits are respected:

$$H_{\text{const}}(\mathbf{x}) = \sum_i \lambda_{P_i^+} \left( P_i^G - \underline{P_i^G} - \Delta P \cdot s_{P_i^+} \right)^2 \tag{5.23a}$$

$$+\lambda_{P_i^-} \left( \overline{P_i^G} - P_i^G - \Delta P \cdot s_{P_i^-} \right)^2 \tag{5.23b}$$

$$+\lambda_{P_i} \left( \underline{P_i^G} - \overline{P_i^G} + \Delta P \cdot s_{P_i^+} - \Delta P \cdot s_{P_i^-} \right)^2 \tag{5.23c}$$

$$\lambda_{Q_i^+} \left( Q_i^G - \underline{Q_i^G} - \Delta Q \cdot s_{Q_i^+} \right)^2 \tag{5.23d}$$

$$+\lambda_{Q_i^-} \left( \overline{Q_i^G} - Q_i^G - \Delta Q \cdot s_{Q_i^-} \right)^2 \tag{5.23e}$$

$$+\lambda_{Q_i} \left( \underline{Q_i^G} - \overline{Q_i^G} + \Delta Q \cdot s_{Q_i^+} - \Delta Q \cdot s_{QP_i^-} \right)^2 \tag{5.23f}$$

$$\lambda_{V_i^+} \left( V_i - \underline{V_i} - \Delta V \cdot s_{V_i^+} \right)^2 \tag{5.23g}$$

$$+\lambda_{V_i^-} \left( \overline{V_i} - V_i - \Delta V \cdot s_{V_i^-} \right)^2 \tag{5.23h}$$

$$+\lambda_{V_i} \left( \underline{V_i} - \overline{V_i} + \Delta V \cdot s_{V_i^+} - \Delta V \cdot s_{V_i^-} \right)^2 \tag{5.23i}$$

$$\lambda_{\delta_i^+} \left( \delta_i - \underline{\delta_i} - \Delta\delta \cdot s_{\delta_i^+} \right)^2 \tag{5.23j}$$

$$+\lambda_{\delta_i^-} \left( \overline{\delta_i} - \delta_i - \Delta\delta \cdot s_{\delta_i^-} \right)^2 \tag{5.23k}$$

$$+\lambda_{\delta_i} \left( \underline{\delta_i} - \overline{\delta_i} + \Delta\delta \cdot s_{\delta_i^+} - \Delta\delta \cdot s_{\delta_i^-} \right)^2. \tag{5.23l}$$

The actual cost function of the Hamiltonian is defined as:

$$H_{\text{cost}}(\mathbf{x}) = \lambda_{P_k^G} \sum_{k \in \mathbb{G}}^{N_G} f_k \left(P_k^G\right)^2 . \tag{5.24}$$

Combining all three components yields the following problem Hamiltonian:

$$H_{\text{OPF}}(\mathbf{x}) = \lambda_{\text{PF}} H(\mathbf{x}) + H_{\text{const}}(\mathbf{x}) + H_{\text{cost}}(\mathbf{x}). \tag{5.25}$$

## 5.4. Algorithm Implementation

The iterative scheme used for the combinatorial PF problem is outlined in Algorithm 2. First, the generation and demand power vectors, $\mathbf{P}^{\text{G}}$, $\mathbf{P}^{\text{D}}$, and $\mathbf{Q}^{\text{D}}$, along with the admittance matrix $\mathbf{Y}$, are initialized according to the given power system data (lines 1-3). The increment vectors, $\Delta\mu$ and $\Delta\omega$, and the initial real and imaginary voltage vectors, $\mu^0$ and $\omega^0$, are then assigned by user-defined values (lines 4-5). Based on these initializations, the corresponding active and reactive power vectors, $\mathbf{P}$ and $\mathbf{Q}$, are computed (line 6), excluding the *slack* bus entries, $P_1$ and $Q_1$, to remain consistent with the PF problem.

Next, the problem Hamiltonian (5.6) or (5.11) is evaluated for the initial $\mu^0$ and $\omega^0$ (line 7), providing a first estimate of the solution. A convergence threshold $\epsilon$ is set and the iteration counter $it$ is initialized to zero (line 8). During each iteration, the problem Hamiltonian is minimized with a given solver (line 10). In this study, QAOA and Ising machines are employed as solvers. The resulting binary/spin variable vector $\mathbf{x} \in \{0,1\}^{4N}$ or $\mathbf{s} \in \{\pm 1\}^{2N}$ is used to update the voltage components $\mu_i$ and $\omega_i$ according to (5.3) or (5.8) (line 11). If (5.6) or (5.11) falls below $\epsilon$, the corresponding complex voltages, $\mu + j\omega$, are accepted as the PF solution. Otherwise, the base voltage values are reset, $\mu^0 := \mu$ and $\omega^0 := \omega$ (line 14), and the increments are adjusted using (5.26). The loop continues with the updated Hamiltonian until the algorithm converges to a solution.

For OPF, Algorithm 2 should be modified to align with the OPF problem principle. First of all, line 1 is not needed, as the generator set points are obtained by solving the OPF problem. Moreover, the active and reactive power vectors, $\mathbf{P}$ and $\mathbf{Q}$, are computed only for load buses in lines 6 and 12. Finally, the problem Hamiltonian in line 10 should be replaced with (5.25).

### 5.4.1. Delta Update

The increments, $\Delta\mu_i$ and $\Delta\omega_i$, are iteration-dependent and gradually decrease over time, thus allowing the optimization to transition from coarse exploration to fine refinement of the solution space, as:

$$\Delta\mu_i = \exp\left(\ln(0.1) + \frac{it \cdot \left(\ln(1 \times 10^{-4}) - \ln(0.1)\right)}{it_{\max}}\right), \tag{5.26a}$$

$$\Delta\omega_i = \exp\left(\ln(0.05) + \frac{it \cdot \left(\ln(1 \times 10^{-5}) - \ln(0.05)\right)}{it_{\max}}\right), \tag{5.26b}$$

**Algorithm 2** Iterative scheme for combinatorial PF.

Initialize generation vector $\mathbf{P}^{\mathrm{G}} = [P_1^{\mathrm{G}}, P_2^{\mathrm{G}}, \ldots, P_{N_{\mathrm{G}}}^{\mathrm{G}}]$
Initialize demand vectors $\mathbf{P}^{\mathrm{D}} = [P_1^{\mathrm{D}}, \ldots, P_{N-N_{\mathrm{G}}-1}^{\mathrm{D}}]$ and $\mathbf{Q}^{\mathrm{D}} = [Q_1^{\mathrm{D}}, \ldots, Q_{N-N_{\mathrm{G}}-1}^{\mathrm{D}}]$
Initialize the admittance matrix $\mathbf{Y} = \{G_{ik} + jB_{ik} : i, k = 1, \ldots, N\}$
Initialize increment vectors $\Delta\mu \leftarrow 0.1$, $\Delta\omega \leftarrow 0.05$
Initialize voltage vectors $\mu^0 = [1, \ldots, 1]$, $\omega^0 = [0, \ldots, 0]$
Compute initial active and reactive power vectors $\mathbf{P} = [P_2, \ldots, P_N]$ and $\mathbf{Q} = [Q_2, \ldots, Q_N]$ using (5.1)
Evaluate (5.6) or (5.11) with $\mu^0$ and $\omega^0$
Set convergence threshold $\epsilon \leftarrow 1 \times 10^{-3}$ and iteration counter $it \leftarrow 0$
**while** (5.6) or (5.11)$> \epsilon$ and $it < it_{\max}$ **do**
    Minimize (5.6) or (5.11) with a given solver
    Update voltage vectors $\mu$ and $\omega$ using (5.3) or (5.8)
    Calculate $\mathbf{P}$ and $\mathbf{Q}$ with updated $\mu$ and $\omega$ using (5.1)
    Evaluate (5.6) or (5.11) updated $\mu$ and $\omega$
    Reset base voltage vectors $\mu^0 := \mu$ and $\omega^0 := \omega$
    Update increment vectors $\Delta\mu$ and $\Delta\omega$ using (5.26)
    Increment iteration counter: $it \leftarrow it + 1$
**end while**
Return complex voltage solution: $\mathbf{V} = \mu + j\omega$

where $it$ is the iteration counter and $it_{\max}$ is the maximum number of iterations. Since $\mu$ and $\omega$ are expressed in (p.u.), the maximum and minimum values for $\Delta\mu$ and $\Delta\omega$ are chosen accordingly to remain within numerically stable ranges.

In each iteration, $\Delta\mu$ and $\Delta\omega$ are updated individually. For example, we save two sets of binary variables $\mathbf{x}^{(it-1)}$ and $\mathbf{x}^{(it-2)} \in \{0,1\}^{4N}$, which represent the bitstrings for the previous and second previous iterations, respectively. We then use these bitstrings to update $\Delta\mu_i$ and $\Delta\omega_i$ for each bus $i$. For example, $x_{i,0}^{\mu(it-2)} = 1 \wedge x_{i,1}^{\mu(it-2)} = 0$, $x_{i,0}^{\mu(it-1)} = 1 \wedge x_{i,1}^{\mu(it-1)} = 0$, and $x_{i,0}^{\mu} = 0 \wedge x_{i,1}^{\mu} = 1$ indicates that at iteration $it-2$, the value of $\mu_i$ increased, followed by an increase at iteration $it-1$ and then a decrease at the current iteration $it$. This pattern suggests that to prevent $\mu_i$ from fluctuations and promote convergence, we should reduce $\Delta\mu_i$.

### 5.4.2. Higher-Order Reduction

(5.4), (5.5), (5.9) and (5.10) already contain quadratic terms resulting from the interactions between pairs of binary/spin decision variables, e.g., $s_i^{\mu}\Delta\mu_i G_{ik} s_k^{\mu}\Delta\mu_k$ in (5.9). Therefore, substituting (5.4) and (5.5) into (5.6) or (5.9) and (5.10) into (5.11) yields a fourth-order polynomial in the binary/spin variables. For GQC, we employ QAOA to solve this minimization problem, which utilizes a parameterized quantum circuit (PQC) and variational optimization, thereby allowing direct encoding

and optimization of problems with higher-order interactions. For AQC, we use two Ising machines provided by D-Wave and Fujitsu. Since current D-Wave hardware can only handle quadratic interactions, we use the Python package PyQUBO[3] to reduce higher-order interactions to quadratic ones. For DAv3, we use the DADK Python package provided by Fujitsu Technology Solutions to handle higher-order terms. QIIO natively supports higher-order reduction, thus allowing for the direct implementation of the fourth-order polynomial.

## 5.5. Solver Implementation

Two sorts of solvers are used to solve minimization problems, QUBO or Ising models. This implementation directly corresponds to line 10 in Algorithm 2. In the case of GQC hardware, the variational quantum subroutine is called inside the iterative scheme. For the Ising machines, the quantum/digital annealer is called instead.

### 5.5.1. QAOA

QAOA is implemented using PennyLane's `lightning.qubit` statevector simulator, a high-performance statevector backend that supports efficient simulation of mid-scale quantum circuits. The QAOA ansatz with $p$ alternating layers of cost and mixer Hamiltonians/unitaries is defined as:

$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = \prod_{k=1}^{p} e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes 2N}, \tag{5.27}$$

where $H_M = \sum_{i=1}^{2N} X_i$ is the mixer Hamiltonian with Pauli-$X$ operators acting on each qubit, and $H_C$ is the problem-specific cost Hamiltonian defined in (5.11). The system size is $2N$, where $N$ is the number of buses in the PF equations (5.1), since both $\mu$ and $\omega$ variables are discretized. The parameters $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_p)$ and $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_p)$ are randomly initialized within $[0, 2\pi]$, and optimized iteratively to minimize the expected energy $\langle H_C \rangle$. Each expectation value is estimated from projective measurements in the computational basis, with 1,000 shots per evaluation. The optimization loop is classical. The `Adam` optimizer is used to update parameters over 100 steps, with a learning rate of 0.1. For each step, the ansatz $|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle$ is prepared, measured, and $\langle H_C \rangle$ is computed. The optimizer then updates the parameters in order to converge towards an approximate ground state of $H_C$.

### 5.5.2. Ising Machines

Four annealers are employed: QA, HA, DAv3, and QIIO. The number of readout samples per execution is fixed for all solvers, with values ranging from 2,000 to 100,000, determined by the test system size. A 10-second time limit is assigned

[3] https://pyqubo.readthedocs.io

to HA. Minor embedding is employed for QA, with the chain strength tuned to balance between preventing chain breaks and preserving the relative importance of the QUBO coefficients. For DAv3 and QIIO, the runtime configuration includes a 10-second time limit, 64-bit scaling precision, and a total optimization timeout of 3,600 seconds.

## 5.6. Results

The key findings from our research studies [1–3] are presented in this section, which progressively advancing the application of quantum optimization algorithms to PF and OPF. First, we compare the performance of GQC and AQC hardware for solving PF equations using the developed Ising model on a 4-bus test system consisting of one *slack* bus and three *PQ* buses in Section 5.6.1 [9]. We extend the algorithm to large-scale systems using the QUBO model. The performance of QIIO is then compared with the NR solver provided by Pandapower [10] in Section 5.6.2 for the full and partitioned formulation of PF. These comparisons are performed based on the 118-bus test system[4], which includes 118 buses, 173 branches, 13 transformers, 53 generators, and 54 loads. In Section 5.6.3, the performance of QIIO under ill-conditioned scenarios is evaluated for the 118-bus test system. In Section 5.6.4, a scalability analysis is presented for QIIO, demonstrating its ability to handle increasingly larger test systems from the Power System Test Cases library[5].

### 5.6.1. Performance Comparison of GQC and AQC

An Ising model is developed based on the combinatorial PF formulation. The problem Hamiltonian is solved using both the GQC and AQC platforms. Experiments are conducted on the 4-bus test system. For a given load scenario, Table 5.1 summarizes the parameters used for QA and QIIO, and Table 5.2 summarizes the QAOA hyperparameters, where the specifications are selected based on preliminary runs to balance convergence accuracy and computational cost.

Table 5.3 summarizes the results. The active and reactive power demands for *PQ* buses, $\mathbf{P}^{\mathrm{D}}$ and $\mathbf{Q}^{\mathrm{D}}$, and $\mu_0$ and $\omega_0$ for the *slack* bus are specified, while $\mu_i$ and $\omega_i$ are unknown for all *PQ* buses $\forall i \in \{1, 2, 3\}$.

The Ising model implementation results in 26 spin variables for QA, where higher-order interactions are reduced to quadratic terms using PyQUBO, and 20 decision variables for QIIO. The discrepancy in the number of variables reflects the differing strategies employed by the underlying software frameworks in handling higher-order interactions. For QAOA, the variable count corresponds to the number of qubits. With two variables per bus $i$ (one for $\mu_i$ and one for $\omega_i$), the 4-bus test system requires 8 qubits. Compilation times for QA and QIIO are comparable, as both involve classical preprocessing and reduction of higher-order terms. QAOA, in contrast, exhibits a compile time that is one order of magnitude higher, primarily due to circuit transpilation and parameter initialization, as shown in Table 5.3.

[4] https://icseg.iti.illinois.edu/ieee-118-bus-system/
[5] https://pandapower.readthedocs.io/en/v2.2.2

Table 5.1: Ising machine implementation parameters for the combinatorial PF analysis based on the 4-bus test system.

| Parameter | Specification |
|---|---|
| QA Number of qubits $q$ | 26 (–) |
| QIIO Number of qubits $q$ | 20 (–) |
| Number of readouts $r$ | 1000 (–) |
| Convergence threshold $\epsilon$ | $1 \times 10^{-3}$ (–) |
| QIIO Time limit | 10 (seconds) |
| QIIO Precision | 64 – bits |
| QIIO Overall timeout | 3600 (seconds) |
| QA Chip ID | `Advantage2_system1.5` |
| QA Minor embedding | `EmbeddingComposite` |

Table 5.2: QAOA implementation parameters for the combinatorial PF analysis based on the 4-bus test system.

| Parameter | Specification |
|---|---|
| Number of qubits $q$ | 8 (–) |
| Circuit depth $p$ | 2 (–) |
| Optimization steps $st$ | 100 (–) |
| Learning rate $lr$ | 0.1 (–) |
| Shots per expectation evaluation $e$ | 1000 (–) |
| Convergence threshold $\epsilon$ | $1 \times 10^{-3}$ (–) |
| Optimizer | `Adam` |
| Simulator backend | `lightning.qubit` |

Table 5.3: Computational detail for the combinatorial PF analysis solved with QA, QIIO, and QAOA based on the 4-bus test system.

| Solver | No. of **Variables** (–) | Compile **Time** (seconds) | No. of **Iterations** (–) | Time per **Iteration** (seconds) | Residual ($\frac{MW^2+MVAR^2}{2}$) |
|---|---|---|---|---|---|
| QA | 26 | 0.003 | 222 | 0.015 | $5.18 \times 10^{-4}$ |
| QIIO | 20 | 0.025 | 63 | 0.06 | $3.31 \times 10^{-4}$ |
| QAOA | 8 | 0.03 | 300 | 15.6 | $2.49 \times 10^{-3}$ |

Convergence behavior differs considerably across solvers. QA requires 222 iterations to satisfy the tolerance $\epsilon = 1 \times 10^{-3}$, with an average iteration time of 0.015 seconds (QPU access time). Note that the wall-clock time per iteration is, however, 1.25 seconds, reflecting overheads from minor embedding and QPU communication (programming, annealing, readout, and sampling). QIIO achieves convergence in 63 iterations with the average iteration time of 0.06 seconds, excluding communication overhead. QAOA, executed on a simulator, is dominated

by repeated circuit evaluations and optimizer updates (100 steps per run). Despite 300 iterations, it does not reach the predefined threshold.

Table 5.4 summarizes the bus complex voltages ($\mu_i + j\omega_i$) obtained from the combinatorial PF analysis using QA, QIIO, and QAOA, in comparison with the NR solver for the 4-bus test system. The *slack* bus ($i = 0$) with fixed values $\mu_0 = 1$ and $\omega_0 = 0$ is not shown. Both QA and QIIO reproduce the NR solution with high accuracy, with deviations on the order of $10^{-3}$ for both $\vec{\mu}$ and $\vec{\omega}$. QAOA achieves comparable accuracy for $\vec{\mu}$, but shows slightly larger deviations for $\vec{\omega}$ and does not converge to the predefined threshold within 300 iterations.

Table 5.4: Performance comparison of QA, QIIO, and QAOA with the NR solver for the 4-bus test system. The *slack* bus $i = 0$ with known $\mu_0 = 1$ and $\omega_0 = 0$ is not shown.

| | $\mu_1$ (p.u.) | $\mu_2$ (p.u.) | $\mu_3$ (p.u.) | $\omega_1$ (p.u.) | $\omega_2$ (p.u.) | $\omega_3$ (p.u.) |
|---|---|---|---|---|---|---|
| NR | 0.902 | 0.916 | 0.890 | -0.092 | -0.080 | -0.104 |
| QA | 0.901 | 0.915 | 0.889 | -0.093 | -0.080 | -0.105 |
| QIIO | 0.901 | 0.915 | 0.889 | -0.092 | -0.080 | -0.105 |
| QAOA | 0.902 | 0.916 | 0.890 | -0.089 | -0.078 | -0.099 |

Figure 5.1 shows the evolution of bus complex voltage $\mu = [\mu_1, \mu_2, \mu_3]$ and $\omega = [\omega_1, \omega_2, \omega_3]$ obtained with QA, QIIO, and QAOA for the 4-bus test system over iterations. The *slack* bus values $\mu_0 = 1$ (p.u.) and $\omega_0 = 0$ (p.u.) are not shown. Results from the NR are included as references. Figure 5.1 (a–c) show the convergence of $\mu_{1-3}$, where $\mu_2$ reaches its NR value faster than $\mu_1$ and $\mu_3$. Figure 5.1 (d–f) display the corresponding $\omega_{1-3}$, with $\omega_2$ converging more rapidly than $\omega_1$ and $\omega_3$. QA oscillates around the NR values for $\omega_{1-3}$ before stabilizing. Its longer overall convergence rate is mainly due to the slower convergence of $\mu_1$ and $\mu_3$. Among the methods, QIIO converges the fastest, while QAOA is the slowest. Nevertheless, all three approaches ultimately yield solutions consistent with the NR benchmark.

### 5.6.2. Full and Partitioned Formulations

A QUBO model is developed based on the combinatorial PF and OPF formulations. The problem Hamiltonian is solved using Ising machines and the solver is QIIO. The results, both in the full and partitioned formulations, are compared with those of NR for the 118-bus test system. The partitioned formulation is derived by excluding a subset of buses, denoted as $\mathbb{P}$, from the problem Hamiltonian in each iteration. Specifically, in each iteration, a set of 24 buses (approximately 20% of the total buses) is randomly selected and removed from the computation. For the PF problem, this modification is applied in Algorithm 2 by excluding the subset $\mathbb{P}$ from

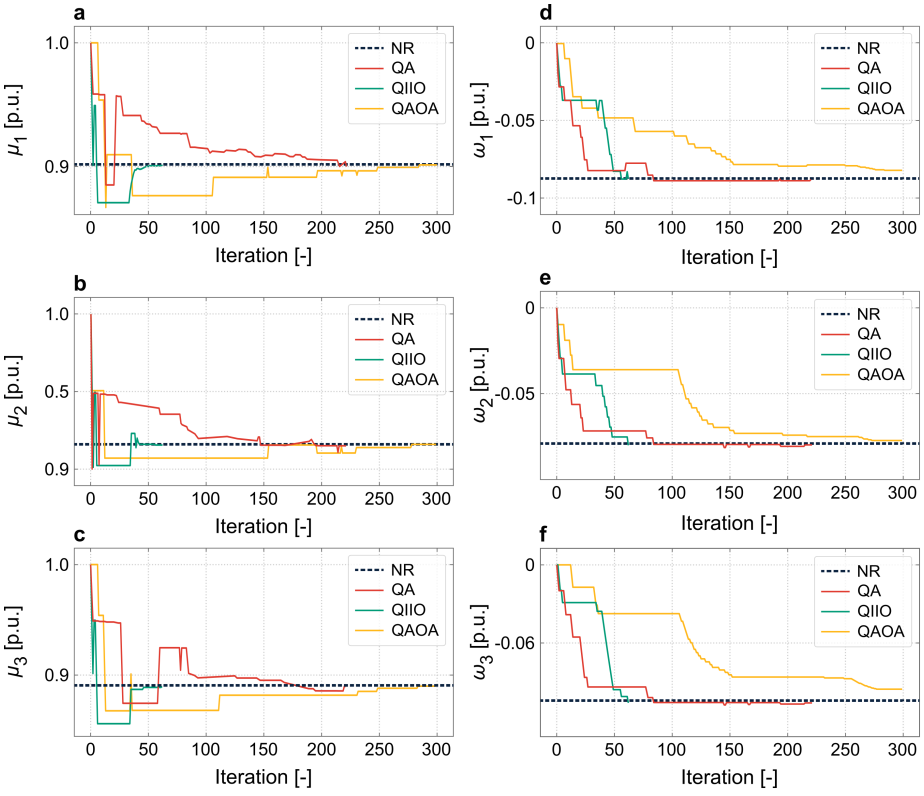



Figure 5.1: Representation of $\mu = [\mu_1, \mu_2, \mu_3]$ and $\omega = [\omega_1, \omega_2, \omega_3]$ obtained by QA, QIIO, and QAOA for the 4-bus test system. The *slack* bus $i = 0$ is not shown. The graphs include $\mu_i$ and $\omega_i$ obtained from the NR solver.

the summation term in (5.6) at line 19, yielding:

$$\min_{\mathbf{x}\in\{0,1\}^{4N}} \sum_{\substack{i=1 \\ i\notin\mathbb{P}}}^{N=118} \left(P_i - P_i^{\mathrm{G}} + P_i^{\mathrm{D}}\right)^2 + \left(Q_i - Q_i^{\mathrm{G}} + Q_i^{\mathrm{D}}\right)^2. \tag{5.28}$$

For the OPF problem, the same subset $\mathbb{P}$ is excluded from the objective function in (5.25), which allows for a reduced problem size per iteration.

The partitioned formulation in (5.28) results in a total of 7105 binary variables (including 472 base variables and auxiliary variables, which are introduced to reduce higher-order terms) for the PF problem, reducing the variable count by 1775 compared to the full formulation. With a residual threshold of $1 \times 10^{-2}$ ($\frac{\mathrm{MW}^2+\mathrm{MVAR}^2}{2}$), the mean square error (MSE) for the net active power $P_i$ between QIIO and NR is $4.28 \times 10^{-4}$ ($\mathrm{MW}^2$), while for partitioned formulation, it is $8.06 \times 10^{-4}$ ($\mathrm{MW}^2$). Similarly, the MSE for the net reactive power $Q_i$ between

QIIO and NR is $1.65 \times 10^{-2}$ (MVAR$^2$), while for partitioned formulation, it is $1.8 \times 10^{-2}$ (MVAR$^2$). These results, illustrated in Figure 5.2, indicate that QIIO closely matches NR, and the partitioned formulation does not significantly degrade solution quality. Further improvements can be achieved by reducing the residual threshold.

In terms of computational performance, QIIO requires 272.98 s for compilation, compared to 208.34 s for the partitioned formulation. The average time per iteration is 251.24 s for the full and 217.61 s for the partitioned formulation. Note that the total computational time includes communication overhead between the classical and quantum-inspired hardware.

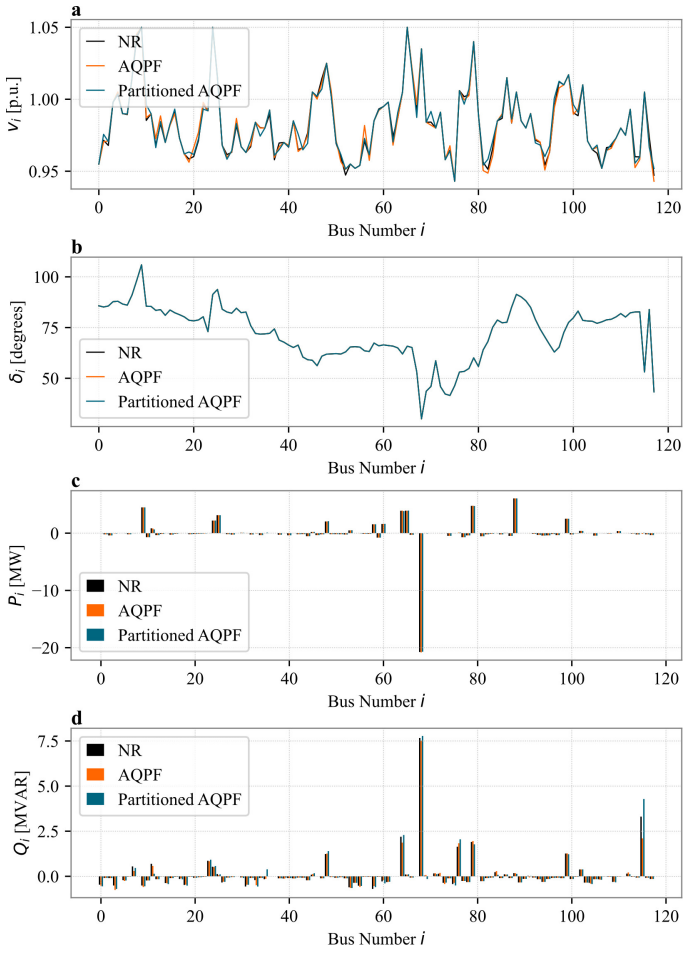



Figure 5.2: Performance comparison of NR and QIIO for the full and partitioned PF formulations for the 118-bus test system. The results are shown for (a) $V_i$ (p.u.), (b) $\delta_i$ (degrees), (c) $P_i$ (MW), and (d) $Q_i$ (MVAR).

For the OPF problem, the partitioned formulation reduces the number of binary variables (including 472 base variables, auxiliary variables, which are introduced to reduce higher-order terms, and slack variables, which are introduced to integrate the constraints) to 8459, which represents a reduction of 2189 binary variables compared to the full formulation in (5.25). With a residual of $1 \times 10^{-2}$ ($\frac{\text{MW}^2+\text{MVAR}^2}{2}$), the MSE for the net active power $P_i$ between QIIO and NR is $8.89 \times 10^{-4}$ ($\text{MW}^2$), while for the partitioned formulation, it is $1.57 \times 10^{-3}$ ($\text{MW}^2$). Similarly, the MSE for the net reactive power $Q_i$ between QIIO and NR is $1.68 \times 10^{-2}$ ($\text{MVAR}^2$), while for the partitioned formulation, it is $1.69 \times 10^{-2}$ ($\text{MVAR}^2$). The results are presented in Figure 5.3.

The compilation times for the full and partitioned formulation are 408.35 s and 318.67 s, respectively. The time per iteration is 384.15 s for full and 302.82 s for partitioned formulation. These results indicate that the partitioned formulation can reduce computational time by up to 20% while maintaining solution accuracy.

### 5.6.3. Well- and Ill-Conditioned Scenarios

The QUBO model developed based on the combinatorial PF and OPF formulations is used. The problem Hamiltonian is solved using Ising machines and the solver is QIIO. To simulate challenging operating conditions, two ill-conditioned scenarios are introduced for the 118-bus test system by modifying system parameters. Load demands at a subset of *PQ* buses are increased to create stress conditions that exceed the system's typical operating limits, which introduces the ill-conditioned scenario 1. The resistance ($R$) values of transmission lines connecting a subset of buses are increased, while the reactance ($X$) values remain unchanged. This results in significantly higher $R/X$ ratios and increasing system losses, which introduces ill-conditioned scenario 2. Under these ill-conditioned scenarios, the NR solver in Pandapower diverges for PF, and for OPF, numerical instability leads to infeasible solutions or failure to converge with optimization constraints remaining unmet. On the other hand, QIIO successfully solves both PF and OPF problems under the same ill-conditioned scenarios. The results are summarized in Tables 5.5 and 5.6. Note that the performance of QIIO for PF and OPF under a well-conditioned scenario is benchmarked against NR in Section 5.6.2.

Table 5.5: QIIO Performance for PF under well and ill-conditioned scenarios for the 118-bus test system.

| **Scenario** | **Residual** ($\frac{\text{MW}^2+\text{MVAR}^2}{2}$) | **Mismatch** ($\text{MW}^2$) | **Mismatch** ($\text{MVAR}^2$) |
|---|---|---|---|
| Well-conditioned case | $8.47 \times 10^{-3}$ | $4.28 \times 10^{-4}$ | $1.65 \times 10^{-2}$ |
| Ill-conditioned case 1 | $8.73 \times 10^{-3}$ | $5.75 \times 10^{-4}$ | $1.68 \times 10^{-2}$ |
| Ill-conditioned case 2 | $8.92 \times 10^{-3}$ | $5.24 \times 10^{-4}$ | $1.73 \times 10^{-2}$ |

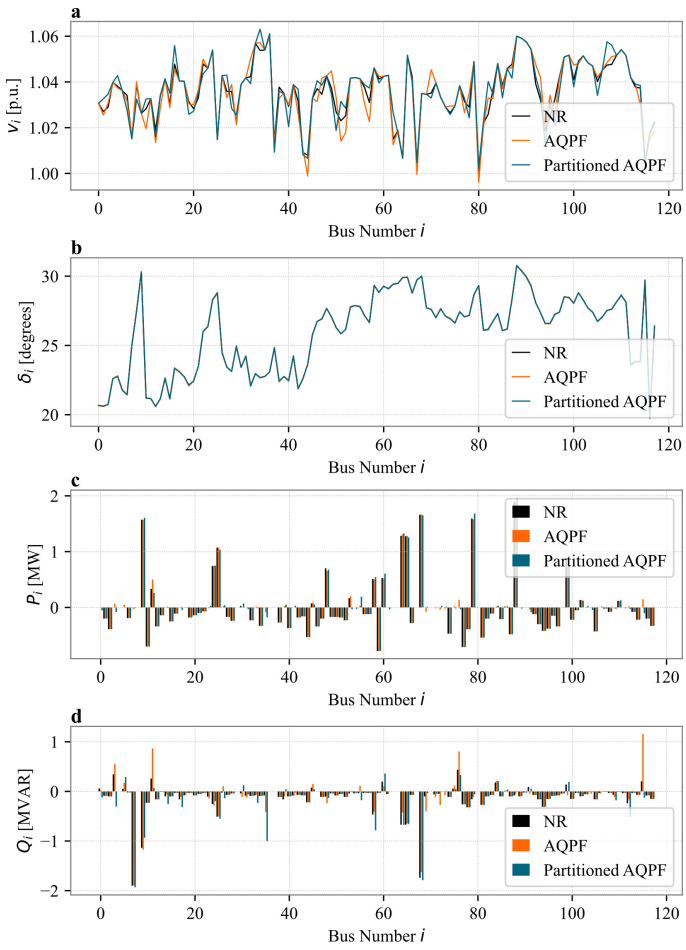



Figure 5.3: Performance comparison of NR and QIIO for the full and partitioned OPF formulations for the 118-bus test system. The results are shown for (a) $V_i$ (p.u.), (b) $\delta_i$ (degrees), (c) $P_i$ (MW), and (d) $Q_i$ (MVAR).

Table 5.6: QIIO Performance for OPF under well and ill-conditioned scenarios for the 118-bus test system.

| **Scenario** | **Residual** $(\frac{\text{MW}^2+\text{MVAR}^2}{2})$ | **Mismatch** $(\text{MW}^2)$ | **Mismatch** $(\text{MVAR}^2)$ |
|---|---|---|---|
| Well-conditioned case | $8.85 \times 10^{-3}$ | $8.89 \times 10^{-4}$ | $1.68 \times 10^{-2}$ |
| Ill-conditioned case 1 | $8.59 \times 10^{-3}$ | $8.67 \times 10^{-4}$ | $1.63 \times 10^{-2}$ |
| Ill-conditioned case 2 | $8.87 \times 10^{-3}$ | $7.23 \times 10^{-4}$ | $1.71 \times 10^{-2}$ |

### 5.6.4. Scalability

The QUBO model developed based on the combinatorial PF and OPF formulations is used. The problem Hamiltonian is solved using Ising machines and the solvers are QA, HA, DAv3, and QIIO. Table 5.7 summarizes the capability of the solvers to converge to a solution within the residual tolerance of $1 \times 10^{-2}$ ($\frac{MW^2+MVAR^2}{2}$). For QA and HA, convergence indicates that an embedding is successfully found, and the solution is obtained within the accepted residual tolerance. For DAv3 and QIIO, convergence implies that the solution is obtained within the same tolerance.

According to Table 5.7, QIIO is capable of handling all test case sizes, including the 1354-bus test case. DAv3, on the other hand, is limited to test cases up to the 89-bus test case due to the constraints on the number of available binary variables. QA and HA are even more restricted and are applicable only for small test cases. The difference between DAv3 and QIIO lies in their hardware configurations: DAv3 provides a fully connected block of 8,192 binary variables, while QIIO offers multiple fully connected blocks capable of managing up to 100,000 binary variables. This expanded capacity makes QIIO suitable for large-scale problems. The experiments include test systems ranging from 4- to 1354-bus from the Power System Test Cases library[6]. The results for the 4-, 9-, and 14-bus test systems are presented in [1], and the results for 5-, 6-, and 15-bus test systems are presented in [11].

The scalability of QIIO is demonstrated by its ability to manage larger sets of binary variables for PF, as confirmed through experiments conducted on test systems of varying sizes. The results are summarized in Table 5.8, presenting the number of binary variables (—), the time needed to compile the QUBO formulation (seconds), the time elapsed per iteration (seconds), and the residual ($\frac{MW^2+MVAR^2}{2}$). Accordingly, QIIO is capable of handling the number of binary variables needed to solve PF up to the 1354-bus test system. However, due to the high computational complexity associated with the larger test systems, only a limited number of iterations are performed for them with a residual threshold of $1 \times 10^{0}$ ($\frac{MW^2+MVAR^2}{2}$), serving as a proof of concept.

## 5.7. Discussion and Conclusions

The proposed algorithm in this chapter can be generalized as a “combinatorial model of models” approach that is executable on Ising machines but also on fault-tolerant GQC hardware. This capability opens avenues for tackling a wide range of problems beyond power systems, particularly those that are computationally challenging for classical solvers. The capability of the proposed algorithm in handling ill-conditioned scenarios is attributed to its enhanced numerical stability. This is achieved through a combinatorial formulation that avoids reliance on traditional linearization techniques, thereby mitigating issues caused by near-singular Jacobians. On the other hand, by iteratively refining increments/decrements, the algorithms ensure steady progress toward feasible solutions.

[6] https://pandapower.readthedocs.io/en/v2.2.2

Table 5.7: Performance comparison of different solvers under a user-defined residual of $1 \times 10^{-2}$ ($\frac{\text{MW}^2+\text{MVAR}^2}{2}$).

| Test System | QA | HA | DAv3 | QIIO |
|---|---|---|---|---|
| 4-bus | ✓ | ✓ | ✓ | ✓ |
| 5-bus | ✓ | ✓ | ✓ | ✓ |
| 6-bus | ✓ | ✓ | ✓ | ✓ |
| 9-bus | ✓ | ✓ | ✓ | ✓ |
| 14-bus | | ✓ | ✓ | ✓ |
| 15-bus | | | ✓ | ✓ |
| 30-bus | | | ✓ | ✓ |
| 39-bus | | | ✓ | ✓ |
| 57-bus | | | ✓ | ✓ |
| 89-bus | | | ✓ | ✓ |
| 118-bus | | | | ✓ |
| 145-bus | | | | ✓* |
| 200-bus | | | | ✓* |
| 300-bus | | | | ✓* |
| 1354-bus | | | | ✓* |

* Experiment for the 145-, 200-, 300-, and 1354-bus test case is performed for a residual of $1 \times 10^{0}$ ($\frac{\text{MW}^2+\text{MVAR}^2}{2}$) due to the limited computational time.

Table 5.8: Scalability of QIIO for PF based on different test systems.

| Test System | No. of Variables (—) | Compile Time (seconds) | Time per Iteration (seconds) | Residual ($\frac{\text{MW}^2+\text{MVAR}^2}{2}$) |
|---|---|---|---|---|
| 9-bus | 249 | 0.27 | 0.93 | $8.47 \times 10^{-3}$ |
| 14-bus | 494 | 39.62 | 41.81 | $8.11 \times 10^{-3}$ |
| 30-bus | 1,183 | 40.35 | 40.91 | $8.52 \times 10^{-3}$ |
| 57-bus | 3,873 | 90.05 | 65.81 | $8.57 \times 10^{-3}$ |
| 118-bus | 8,881 | 272.98 | 251.24 | $8.47 \times 10^{-3}$ |
| 200-bus | 15,098 | 238.22 | 231.07 | $9.38 \times 10^{-1}$ |
| 300-bus | 23,577 | 896.38 | 926.64 | $8.72 \times 10^{-1}$ |
| 1354-bus | 99,509 | 43,151.54 | 41,745.91 | $9.67 \times 10^{-1}$ |

Ongoing hardware advancements, particularly those enabling many-body interactions, higher qubit counts, and improved connectivity, are expected to alleviate problem size issues. Yet, we have explored several strategies to mitigate these

challenges within current hardware limitations. We demonstrate that reformulating the problem using spin variables $\{\pm 1\}$ instead of binary variables $\{0, 1\}$ can reduce the number of decision variables and couplers needed. Furthermore, a partitioned formulation is proposed that solves subsets of the problem, thus significantly decreasing the problem size per iteration without compromising accuracy. Nevertheless, developing problem-specific schemes for more efficient reduction of higher-order terms remains an important direction for future study.

The ability of Ising machines to handle a large number of binary decision variables also enables efficient searching of the solution space. Combinatorial PF and OPF formulations enable a discrete modeling approach that ensures convergence. Nevertheless, discretization introduces approximations that can reduce numerical accuracy. The quadratic nature of current hardware necessitates the reduction of higher-order terms through auxiliary variables, while slack binary variables are also needed to encode hard constraints, both of which increase the dimensionality. In proposed algorithm, the step sizes are not fixed but are adaptively updated. Specifically, larger step sizes are applied during the initial iterations to accelerate convergence, while smaller step sizes are used as the algorithm approaches the solution. Furthermore, the step sizes are updated per bus, which helps maintain numerical stability, mitigate cumulative discretization errors, and reduce the number of hardware calls.

## References

[1] **Z. Kaseb**, M. Möller, P. P. Vergara and P. Palensky. 'Power flow analysis using quantum and digital annealers: a discrete combinatorial optimization approach'. In: *Nature Scientific Reports* 14.1 (2024), p. 23216. DOI: 10.1038/s41598-024-73512-7.

[2] **Z. Kaseb**, M. Möller, P. Palensky and P. P. Vergara. 'Power flow and optimal power flow using quantum and digital annealers: a computational scalability analysis'. In: *arXiv preprint* (2026). DOI: arXiv:2505.15978.

[3] **Z. Kaseb**, M. Möller, P. Palensky and P. P. Vergara. 'Performance comparison of gate-based and adiabatic quantum computing for power flow analysis'. In: *arXiv preprint* (2026). DOI: arXiv:2510.13378.

[4] D. Bienstock and A. Verma. 'Strong NP-hardness of AC power flows feasibility'. In: *Operations Research Letters* 47.6 (Nov. 2019), pp. 494–501. ISSN: 01676377. DOI: 10.1016/j.orl.2019.08.009.

[5] R. Au-Yeung, N. Chancellor and P. Halffmann. 'NP-hard but no longer hard to solve? Using quantum computing to tackle optimization problems'. In: *Frontiers in Quantum Science and Technology* 2 (Feb. 2023). ISSN: 2813-2181. DOI: 10.3389/frqst.2023.1128576.

[6] H. Jing, Y. Wang and Y. Li. 'Data-driven quantum approximate optimization algorithm for power systems'. In: *Communications Engineering* 2 (2023). ISSN: 2731-3395. DOI: 10.1038/s44172-023-00061-8.

[7] E. Pelofske, A. Bärtschi and S. Eidenbenz. 'Short-depth QAOA circuits and quantum annealing on higher-order ising models'. In: *npj Quantum Information* 10 (1 Mar. 2024), p. 30. ISSN: 2056-6387. DOI: 10.1038/s41534-024-00825-w.

[8] H. Goto, K. Tatsumura and A. R. Dixon. 'Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems'. In: *Science Advances* 5 (4 Apr. 2019). DOI: 10.1126/sciadv.aav2372.

[9] J. J. Grainger and W. D. Stevenson Jr. *Power System Analysis*. McGraw-Hill, Inc., 1994.

[10] L. Thurner, A. Scheidler, F. Schäfer, J.-H. Menke, J. Dollichon, F. Meier, S. Meinecke and M. Braun. 'pandapower—an open-source python tool for convenient modeling, analysis, and optimization of electric power systems'. In: *IEEE Transactions on Power Systems* 33.6 (2018), pp. 6510–6521. DOI: 10.1109/TPWRS.2018.2829021.

[11] **Z. Kaseb**, M. Möller, M. Kirsch, P. Palensky and P. P. Vergara. 'Combinatorial power flow analysis using adiabatic quantum algorithms'. In: *IEEE PowerTech*. 2025.

# 6

# Quantum Hardware-in-the-Loop

*We propose a quantum hardware-in-the-loop (QHIL) framework that integrates a real-time digital simulator (RTDS®) with Ising machines to solve the combinatorial power flow (PF) and optimal power flow (OPF) problems presented in Chapter 5 under dynamic operating conditions. The reader is referred to Sections 2.1 and 2.2 for background on PF, OPF, and adiabatic quantum computing (AQC), upon which this chapter builds.*

Parts of this chapter have been published in IEEE Transactions on Power Systems (2026) with the title: "Quantum hardware-in-the-loop for optimal power flow in renewable-integrated power systems" [1].

## 6.1. Motivation

The global transition toward sustainable energy has driven a rapid expansion of renewable energy sources (RES), particularly photovoltaic solar panels and wind turbines. However, the inherent intermittency and stochastic behavior of these resources pose significant challenges to power system operation, thus requiring advanced methodologies to ensure stability and reliability [2]. On the other hand, the rising costs and environmental impact of fossil fuels have underscored the urgency of minimizing their use and integrating RES more effectively [3].

To maintain secure and efficient grid operation and assess system conditions, transmission and distribution system operators rely on power flow (PF) analysis [4]. When economic, stability, or operational objectives must be optimized, this extends to optimal power flow (OPF). Therefore, OPF plays a crucial role in power system operation and planning by determining the most efficient system configuration through the adjustment of control variables, including active and reactive power dispatch, transformer tap positions, and reactive power compensation [5]. Solving OPF problems enables system operators to minimize generation costs, enhance voltage stability, and ensure compliance with operational constraints, thereby improving the overall efficiency and cost-effectiveness of modern power systems [6].

Classical OPF solvers, including linear programming (e.g., [7]), nonlinear programming (e.g., [8]), quadratic programming (e.g., [9]), and Newton-based approaches (e.g., [10]), have been extensively studied and widely adopted in academia and industry [11]. While these solvers have proven effective for traditional power systems, their application to modern RES-integrated systems presents significant challenges. The inherent uncertainty of RES introduces complex, large-scale, and highly nonlinear and non-convex optimization problems, often leading to slow convergence and excessive computational burden [12]. As a result, advanced OPF algorithms are needed to ensure real-time applications, system stability, and optimal operation [13].

While recent theoretical advances in PF and OPF problems with Ising machines indicate great potential [14], their validation in real-time simulations remains essential to uncover practical limitations and implementation challenges. Hardware-in-the-loop (HIL) simulation is an effective approach for such validations, as it couples physical hardware, such as quantum or quantum-inspired processors, with real-time power system simulators. This setup offers a controlled yet realistic environment to systematically evaluate algorithmic performance, stability, and responsiveness under dynamic grid conditions, including variable renewable generation and fluctuating loads (e.g., [15, 16]). For example, a quantum-in-the-loop (QIL) framework that integrates GQC with real-time power system simulations is introduced in [17], showing the potential to address complex optimization challenges in modern power systems and to enhance real-time decision-making by processing inputs from numerous distributed sensors and controllers.

This chapter is motivated by the increasing complexity of RES-integrated power systems and the emerging potential of AQC to address large-scale nonlinear optimization problems. We develop a novel quantum hardware-in-the-loop (QHIL) framework to bridge the gap between real-time power system modelling and

control and Ising machines. We also develop a middleware that ensures efficient real-time communication between a real-time digital simulator (RTDS®) and Ising machines, including D-Wave's Advantage™ quantum annealer (QA)[1] and Fujitsu's Quantum-Inspired Integrated Optimization software (QIIO)[2]. The algorithm developed in Chapter 5, is used to solve the combinatorial PF and OPF problems. Based on the observations, lessons learned, and results, we demonstrate the feasibility of the proposed QHIL framework and identify practical challenges associated with current and near- to mid-future hardware capabilities.

## 6.2. Quantum Hardware-in-the-Loop Framework

The framework is built upon the proposed algorithm for the combinatorial PF and OPF problems, formulated using the quadratic unconstrained binary optimization (QUBO) model, as described in Chapter 5.

### 6.2.1. Computational and Information Flow

Figure 6.1 depicts the computational and information flow within the QHIL framework for OPF. Different colors and line styles are used to distinguish components and improve visual clarity. In particular, the yellow box represents the RTDS® hardware, while the dashed gray box corresponds to the Python environment. Solid box outlines denote the overall system architecture, whereas dashed boxes indicate internal processes, with arrows passing through them to show step-by-step operations and data flows within the system. The problem formulation is initially given essential power system parameters, such as the number of buses $N$, system connectivity, and transmission line resistance, inductance, and capacitance. In addition, real-time data are obtained from the simulator, including demand power values $P_i^D$ and $Q_i^D$, as well as RES power generation outputs $P_i^{PVF}$ and $P_i^{WF}$. If solar and/or wind farms are integrated into the power system, the corresponding parameters are also added.

For OPF, relevant constraints, including bus voltage limits $\underline{V_i}$ and $\overline{V_i}$, as well as generation limits $\underline{P_i^G}$, $\overline{P_i^G}$, $\underline{Q_i^G}$, and $\overline{Q_i^G}$, are provided to ensure that the power system functions within acceptable operating limits. The OPF problem is then solved using the proposed algorithm in Chapter 5, executed on QA and QIIO. The results are subsequently collected from the annealers and can be used for modeling and control purposes, e.g., to calculate the required generator voltage $V_{i,ref}^G$ and active power $P_{i,ref}^G$ set points based on the approach used for modeling the generator's excitor and governor in the real-time power system simulation.

[1] www.dwavequantum.com
[2] en-portal.research.global.fujitsu.com/kozuchi

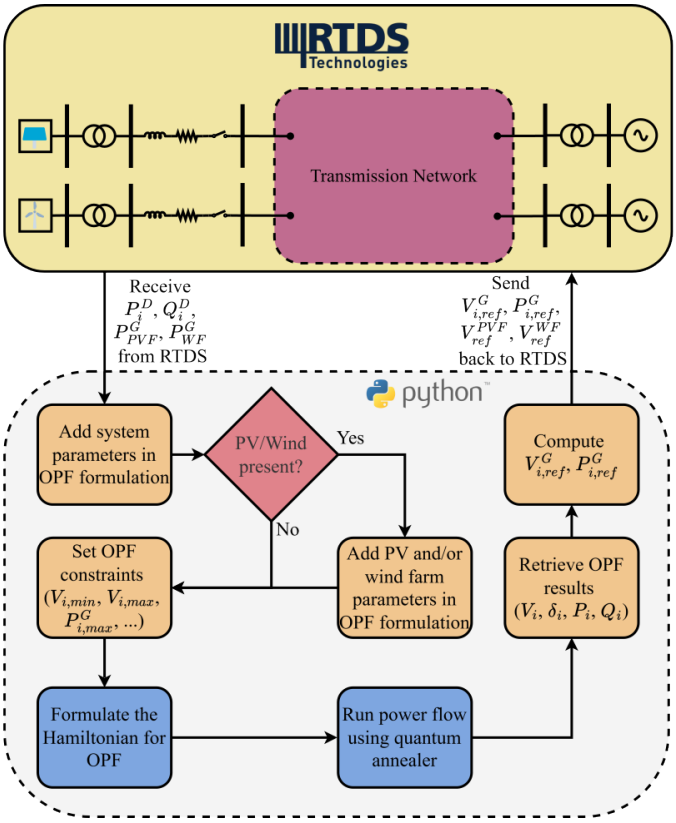



Figure 6.1: Computational and information flow within the QHIL framework for the OPF problem. It receives system parameters, real-time data, $P_i^D$, $Q_i^D$, $P_i^{PVF}$, $P_i^{WF}$, and operational constraints. The problem is then solved using Ising machines. The results are compared to the NR solver in Pandapower. The computed generator set points $V_{i,ref}^G$ and $P_{i,ref}^G$ are then employed in the real-time power system simulation.

The required generator voltage $V_{i,ref}^G$ set point is given by:

$$V_{i,ref}^G = \frac{E_{f,i,pu}^G}{K_a} + V_{i,pu}^G \tag{6.1a}$$

$$\bar{E}_{f,i,pu}^G = \bar{V}_{i,pu}^G + jx_d\bar{I}_{i,pu}^G \tag{6.1b}$$

$$\bar{I}_{i,pu}^G = \frac{S_{i,pu}^*}{\bar{V}_{i,pu}^{G*}} \tag{6.1c}$$

$$S_{i,pu} = \frac{P_i^G + jQ_i^G}{S_{base}} \tag{6.1d}$$

where $K_a$ is the excitor gain; $x_d$ the $d$-axis reactance of the generator; $\bar{E}^G_{f,i,pu}$ the field voltage of the generator in per-unit; $\bar{I}^G_{i,pu}$ the current flowing through the generator in per-unit; $\bar{V}^G_{i,pu}$ the generator terminal voltage; $S_{i,pu}$ the apparent power supplied by the generator in per-unit; $S_{base}$ the base MVA value.

Similarly, the generator's active power $P^G_{i,ref}$ set point can be computed as:

$$P^G_{i,ref} = R\frac{P^G_i}{S_{base}} \tag{6.2}$$

where $R$ is the governor droop coefficient. Finally, the estimated generator and RES converter set points $V^G_{i,ref}$, $P^G_{i,ref}$, $V^{PVF}_{i,ref}$, and $V^{WF}_{i,ref}$ are returned to the simulator as input for system operation.

### 6.2.2. Hardware Configuration

Figure 6.2 shows the hardware configuration of the QHIL framework, where the dashed gray box highlights all equipment modeled in RTDS® and the associated connections, showing the internal operations of the real-time simulation framework. Power system simulations are carried out on the RTDS® NovaCor racks, and the response can be monitored using the RSCAD interface[3]. To provide real-time connectivity with the simulator, an external communication GTNETx2 card is used. When configured with the card's SKT protocol, it allows the simulator and external devices to communicate via the transmission control protocol (TCP). This TCP communication further allows the simulator to export system parameters, such as power demand values $P^D_i$ and $Q^D_i$, as well as RES power generation $P^{PVF}_i$ and $P^{WF}_i$ for OPF. Here, $P^{PVF}_i$ and $P^{WF}_i$ are the active power generated by the solar and wind farms, respectively. Subsequently, the computed generator and RES converter set points $V^G_{i,ref}$, $P^G_{i,ref}$, $V^{PVF}_{i,ref}$, and $V^{WF}_{i,ref}$ are sent back to the simulator in real-time.

RTDS® can be directly interfaced with QA. However, a direct connection with QIIO is currently not feasible, primarily due to the lack of proximity between the hardware and the simulation environment, and secondly, because vendor-specific VPN network access is required. Therefore, a Windows-based middleware is developed to facilitate communication between QIIO and RTDS®.

The inputs required the OPF problem are sent via Python scripts to the proposed algorithms in Chapter 5. The Python script is executed within a Windows Subsystem for Linux (WSL) environment, which runs a virtual Ubuntu instance, where the required Python packages (e.g., D-Wave SDK, PyQUBO, DADK) are installed. Azure VPN Client is used to establish a connection with QIIO. However, this step is not necessary for QA. After obtaining solutions for OPF, the corresponding generator set points are then transmitted back to the simulation loop.

[3] https://www.rtds.com/technology/graphical-user-interface

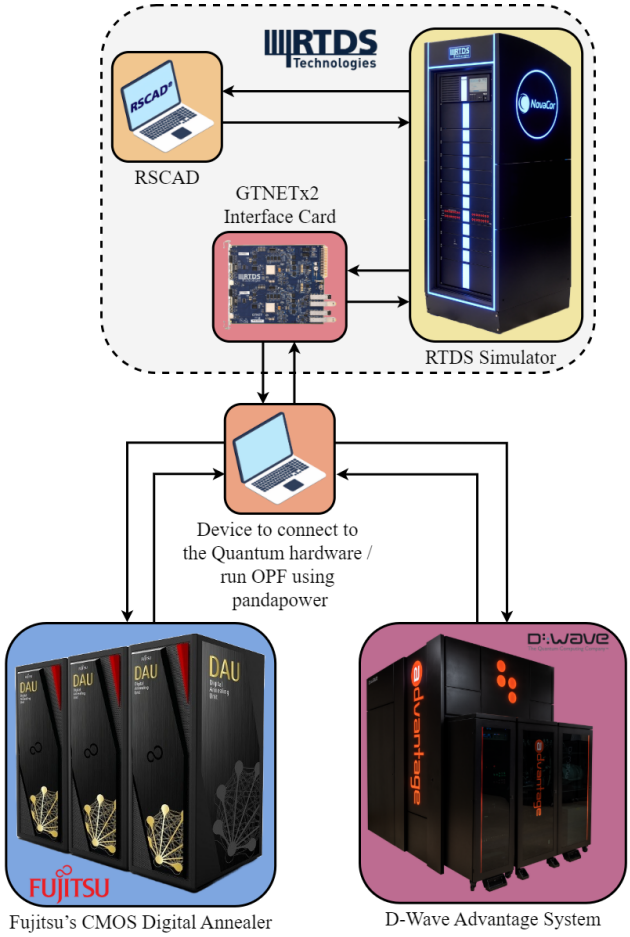



Figure 6.2: Hardware configuration of the proposed QHIL framework for real-time power system simulations using RSCAD/RTDS®. A GTNETx2 card with SKT protocol enables TCP communication to exchange system parameters and RES generation data with the OPF algorithm running on the quantum hardware or local device, which then returns generator and converter set points to the simulator in real time.

## 6.3. Results

The proposed QHIL framework is validated by performing PF analysis and OPF for the IEEE 9-bus test system and a modified version that incorporates solar and wind farms, resulting in a 13-bus test system. Power system simulations are performed on the RTDS® NovaCor rack, and Ising machine experiments are performed on QA and QIIO. The compiler used for the former is PyQUBO, and for the latter is DADK. The tolerance threshold of $1 \times 10^{-2}$ ($\frac{\mathrm{MW}^2+\mathrm{MVAR}^2}{2}$) is chosen for the error bound of

PF and OPF due to our limited hardware access and the computational expenses associated with QA and QIIO.

### 6.3.1. RES-Integrated Test System

The IEEE 9-bus test system is a simplified representation of a power grid, with nine buses, three generators, three transformers, and six transmission lines, see Figure 6.3. Three of the buses are categorized as *PV* buses, indicating generators that provide active power and regulate the bus voltage by controlling the reactive power. These generators are connected to buses 1, 2, and 3. The remaining six buses are *PQ* buses, which may or may not be connected to a load. Specifically, loads are connected to buses 5, 7, and 9. Three step-up transformers connect the generators to the transmission network, and six transmission lines interconnect the buses.

A 1 (MW) solar farm and a 2.5 (MW) wind farm have been modeled and connected to bus 7 to integrate RES, see Figure 6.3. These farms, Generators 4 and 5 in Table 6.1, are then scaled up to 20 (MW) each using the sub-step scaling option available when modeling in RTDS®. Each RES is connected to the power system via a step-up transformer and a series *RL* branch, ensuring that the farms' short-circuit ratio remains constant as the power output varies. A switch is also included to allow for the connection and disconnection of the RES when simulating various operational conditions. Since these RES units are equipped with voltage controllers in the grid-side converters at their respective connecting buses, they can be effectively modeled as *PV* buses in the PF and OPF formulations.

The power output of each RES is measured at the grid-side converter, corresponding to the point of common coupling (PCC), such that the measured active power represents the net power injected into the grid. Therefore, converter losses are inherently excluded from the measured values. Consequently, the maximum active power reported in Table 6.1 (19.8 MW) corresponds to the actual power delivered to the grid, which is slightly lower than the nominal 20 (MW) rating. The cost parameters of these RES units are set lower than those of conventional generators, reflecting their reduced operational and maintenance costs while providing a realistic economic representation in the OPF simulations. Tables 6.1 to 6.4 respectively summarize the test system's generator, load, line, and cost parameters, considering base values of 345 (kV) and 100 (MVA).

Table 6.1: Generator parameters for the IEEE 9-bus test system with integrated RES, including generator locations, capacities, and operational limits.

| **Sr. No.** | **Bus** $i$ | $P_i^G$ (MW) | $V_i$ (p.u.) | $\delta_i$ (deg.) | $\overline{P_i^G}$ (MW) | $\underline{P_i^G}$ (MW) | $\overline{Q_i^G}$ (MVAR) | $\underline{Q_i^G}$ (MVAR) |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | - | 1 | 0 | 250 | 10 | 300 | -300 |
| 2 | 2 | 163 | 1 | - | 300 | 10 | 300 | -300 |
| 3 | 3 | 85 | 1 | - | 270 | 10 | 300 | -300 |
| 4 | 11 | 19.8 | 1 | - | 19.8 | 19.8 | 20 | -20 |
| 5 | 13 | 19.8 | 1 | - | 19.8 | 19.8 | 20 | -20 |

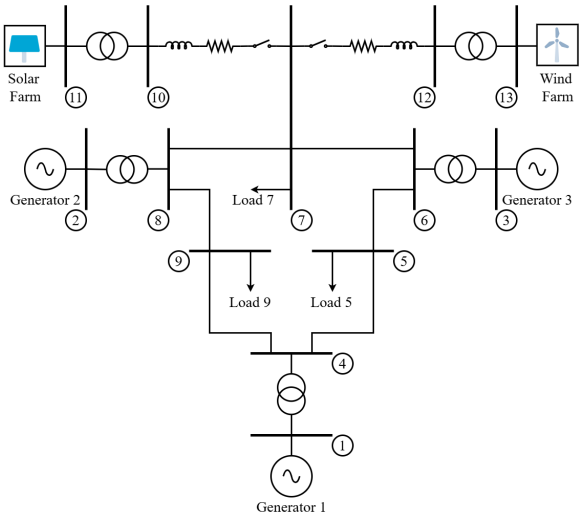



Figure 6.3: The IEEE 9-bus test system representation with integrated RES. The system includes nine buses, three generators at buses 1, 2, and 3 (*PV* buses), and loads at buses 5, 7, and 9 (*PQ* buses). A 1 (MW) solar farm and a 2.5 (MW) wind farm are connected to bus 7 which are scaled to 20 (MW) each. Both RES units are connected using step-up transformers and *RL* branches, maintaining a constant short circuit ratio. Voltage controllers at the RES buses allow them to function as *PV* buses in PF and OPF formulations.

Table 6.2: Load parameters for the IEEE 9-bus test system with integrated RES, containing the active and reactive power demands at each load-connected bus.

| **Sr. No.** | **Bus** $i$ | $P_i^D$ (MW) | $Q_i^D$ (MVAR) |
|---|---|---|---|
| 1 | 5 | 90 | 30 |
| 2 | 7 | 100 | 35 |
| 3 | 9 | 125 | 50 |

### 6.3.2. Computational Performance Assessment

The computational setup for solving PF and OPF combinatorial formulations using QA and QIIO is summarized in Table 6.5. Key computational performance metrics include the number of variables in the QUBO formulation, compilation time, iteration time, and the total number of iterations required for convergence. Overall, QIIO demonstrates significantly faster compilation and iteration times compared to QA. For example, in the PF problem for the IEEE 9-bus test system, the compilation time drops from 4.35 (seconds) with QA to 0.27 (seconds) with QIIO, while the

Table 6.3: Transmission line parameters for the IEEE 9-bus test system with integrated RES, specifying the resistance ($r_{ij}$), inductance ($x_{ij}$), and susceptance ($b_{ij}$) values of each line connecting the buses based on a 345 (kV), 100 (MVA) base.

| **Sr. No.** | **From Bus** $i$ | **To Bus** $j$ | $r_{ij}$ (p.u.) | $x_{ij}$ (p.u.) | $b_{ij}$ (p.u.) |
|---|---|---|---|---|---|
| 1 | 1 | 4 | 0 | 0.0576 | 0 |
| 2 | 4 | 5 | 0.017 | 0.092 | 0.158 |
| 3 | 5 | 6 | 0.039 | 0.17 | 0.358 |
| 4 | 3 | 6 | 0 | 0.0586 | 0 |
| 5 | 6 | 7 | 0.0119 | 0.1008 | 0.209 |
| 6 | 7 | 8 | 0.0085 | 0.072 | 0.149 |
| 7 | 8 | 2 | 0 | 0.0625 | 0 |
| 8 | 8 | 9 | 0.032 | 0.161 | 0.306 |
| 9 | 9 | 4 | 0.01 | 0.085 | 0.176 |
| 10 | 7 | 10 | 0.2379 | 1.6189 | 0 |
| 11 | 10 | 11 | 0.01 | 1 | 0 |
| 12 | 7 | 12 | 0.2583 | 1.6552 | 0 |
| 13 | 12 | 13 | 0.01 | 1 | 0 |

Table 6.4: Generator cost parameters for the IEEE 9-bus test system with integrated RES, specifying the cost coefficients used in the OPF calculations. The cost function is modeled as a polynomial $C_0 + P_i^G C_1 + {P_i^G}^2 C_2$.

| **Generator** | $C_0$ (euros) | $C_1$ (euros/MW) | $C_2$ (euros/MW$^2$) |
|---|---|---|---|
| 1 | 150 | 5 | 0.11 |
| 2 | 600 | 1.2 | 0.085 |
| 3 | 335 | 1 | 0.1225 |
| 4 | 8 | 0.3 | 0.0005 |
| 5 | 10 | 0.5 | 0.001 |

iteration time reduces from 238.48 (seconds) for QA to 0.93 (seconds) for QIIO. The OPF problem for the 13-bus test system exhibits higher computational complexity, involving 635 variables and a longer compilation time of 41.25 (seconds).

The computational times reported in Table 6.5 include the computation overhead associated with QA and QIIO, which is unavoidable with current hardware. The processing time of QA and QIIO is also compared with that of the NR solver. For the 9-bus test system, NR achieves an average processing time of 0.049 (seconds) for PF and 0.397 (seconds) for OPF, measured over 10,000 runs. For 2,000 readouts/samples per iteration, the average anneal time per iteration for PF is 0.04 (seconds) using QA and 0.671 (seconds) using QIIO. The corresponding time for OPF on the 9-bus test system using QIIO is 0.981 (seconds). Note that with NR, the OPF processing time is roughly eight times that of PF, while with QIIO, this ratio is less than two, which suggests improved scalability potential as hardware continues to advance. It should be emphasized, however, that the processing times reported for QA and QIIO are subject to variability due to several factors, such

as hardware noise, specialized interfaces, and VPN connections. Consequently, accurately isolating the pure processing time remains challenging at present.

Table 6.5: Performance comparison of QA and QIIO for the PF and OPF problems in terms of variable count, compilation time, and iteration time across various test systems.

| **Test System** | **Problem** | **Solver** | **Variable No.** (—) | **Compile Time** (seconds) | **Iteration Time** (seconds) | **Iteration No.** (—) |
|---|---|---|---|---|---|---|
| 9-bus | PF | QA | 666 | 4.35 | 238.48 | 152 |
| 9-bus | PF | QIIO | 249 | 0.27 | 0.93 | 119 |
| 9-bus | OPF | QIIO | 331 | 0.42 | 1.81 | 200 |
| 13-bus | OPF | QIIO | 635 | 41.25 | 42.37 | 200 |

### 6.3.3. Error Assessment

Table 6.6 shows the deviation of $V_i$, $\delta_i$, $P_i$, and $Q_i$ for PF and OPF using QA and QIIO compared to the results obtained by NR. The final deviations for $V_i$ and $\delta_i$ remain within the predefined residual threshold $1 \times 10^{-2}$ ($\frac{\text{MW}^2+\text{MVAR}^2}{2}$), which indicates the feasibility and reliability os the proposed algorithms.

Table 6.6: Deviation analysis of values for $V_i$ and $\delta_i$ obtained by QA and QIIO for the PF and OPF problems.

| **Test System** | **Problem** | **Annealer** | $\overline{\lvert\Delta V_i\rvert}$ (p.u.) | $\sigma_{\lvert\Delta V_i\rvert}$ (p.u.) | $\overline{\lvert\Delta\delta_i\rvert}$ (degrees) | $\sigma_{\lvert\Delta\delta_i\rvert}$ (degrees) |
|---|---|---|---|---|---|---|
| 9-bus | PF | QA | $2.53 \times 10^{-4}$ | $2.41 \times 10^{-4}$ | $2.13 \times 10^{-3}$ | $1.37 \times 10^{-3}$ |
| 9-bus | PF | QIIO | $1.29 \times 10^{-3}$ | $1.64 \times 10^{-3}$ | $1.88 \times 10^{-2}$ | $2.15 \times 10^{-2}$ |
| 9-bus | OPF | QIIO | $1.24 \times 10^{-3}$ | $1.75 \times 10^{-3}$ | $1.37 \times 10^{-2}$ | $9.95 \times 10^{-3}$ |
| 13-bus | OPF | QIIO | $1.57 \times 10^{-3}$ | $9.83 \times 10^{-4}$ | $1.52 \times 10^{-2}$ | $2.20 \times 10^{-2}$ |

To examine the achieved errors more closely, the PF result for the 9-bus test system obtained using QA and QIIO is compared against that of NR in Figures 6.4 and 6.5. The top row shows the computed values of $V_i$, $\delta_i$, $P_i$, and $Q_i$, while the bottom row shows the absolute errors of QA, QIIO, relative to NR for each corresponding value. As observed, QA accurately computes both $V_i$ and $\delta_i$, yielding $P_i$ and $Q_i$ values that closely match those of NR. QIIO also produces results within the predefined residual threshold for $V_i$ and $\delta_i$, as shown in Figures 6.4a and 6.4b. However, QA achieves higher accuracy in $P_i$ and $Q_i$ by up to 90%, as shown in Figures 6.5a and 6.5b. These results indicate that while both QA and QIIO satisfy the overall convergence and accuracy criteria based on average error across all buses, QA demonstrates superior accuracy per bus.

The experiments are extended to include OPF for the 9-bus and 13-bus test systems for which the results using QIIO are presented in Figures 6.6 to 6.9. QA is not included as the problem graph could not be embedded onto the available hardware graph. Furthermore, due to limited access to QIIO, simulations are

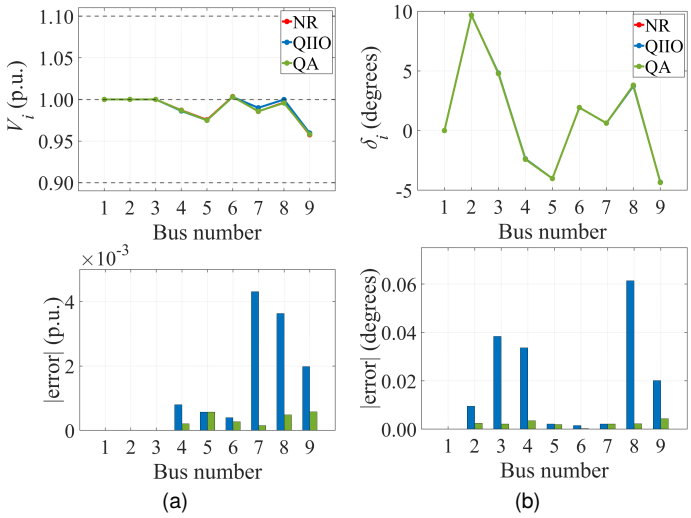



Figure 6.4: PF results for the IEEE 9-bus test system. The top row shows the computed values for (a) $V_i$ and (b) $\delta_i$. The bottom row shows the absolute errors between QA, QIIO, and NR for corresponding parameters.

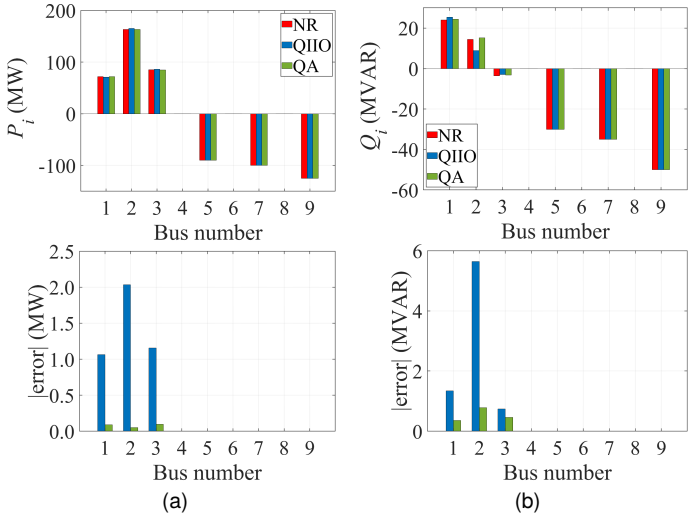



Figure 6.5: PF results for the IEEE 9-bus test system. The top row shows the computed values for (a) $P_i$ and (b) $Q_i$. The bottom row shows the absolute errors between QA, QIIO, and NR for corresponding parameters.

terminated after $it_{\max} = 200$ iterations, leading to residuals slightly more than the predefined residual threshold, see Tables 6.5 and 6.6. Nevertheless, the $V_i$ and $\delta_i$ values computed by QIIO exhibit agreement with NR. For the 9-bus test system, the mean deviation in $P_i$ between QIIO and NR is $2.20 \times 10^{-2}$ (MW), while in $Q_i$, the mean deviation is $1.13$ (MVAR), as shown in Figures 6.7a and 6.7b. For the 13-bus test system, the corresponding deviations are $6.92 \times 10^{-2}$ (MW) and $1.15$ (MVAR), respectively, as shown in Figures 6.9a and 6.9b.

The maximum error in **P** for PF is $0.1\%$ using QA and $2\%$ using QIIO for the 9-bus test system. For OPF, QA results are not included due to hardware embedding limitations. Using QIIO, the maximum error in **P** is $0.6\%$ for both the 9-bus and 13-bus test systems. The corresponding maximum errors in **Q** are $9.2\%$ and $9.7\%$ for the 9-bus and 13-bus test systems, respectively. As observed, the corresponding deviations in **Q** are comparatively higher. This behavior is consistent with established findings that voltages are generally more sensitive to variations in **Q** than in **P** (e.g., [18]). Consequently, small deviations in **V** can lead to amplified errors in computed **Q**.

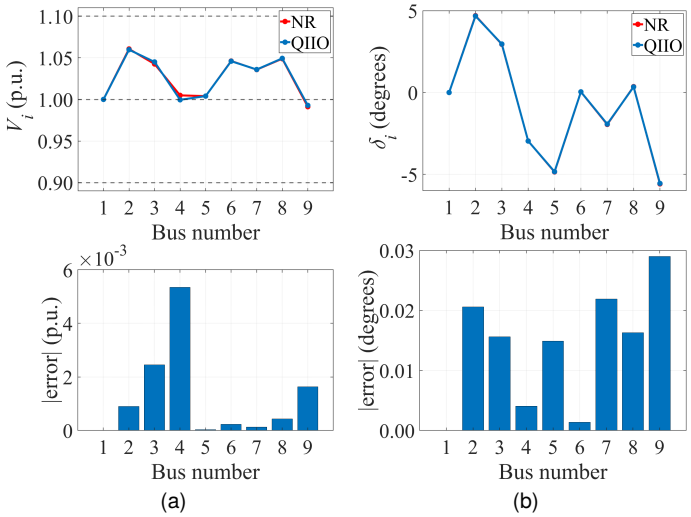



Figure 6.6: OPF results for the IEEE 9-bus test system. The top row shows the computed values for (a) $V_i$ and (b) $\delta_i$. The bottom row shows the absolute errors between QIIO and NR for corresponding parameters.

## 6.4. Discussion and Conclusions

The tolerance threshold of $1 \times 10^{-2}$ ($\frac{\text{MW}^2+\text{MVAR}^2}{2}$) and $it_{\max} = 200$ are selected as stopping criteria for PF and OPF due to limited hardware access and the computational cost associated with QA and QIIO. Within this relatively relaxed

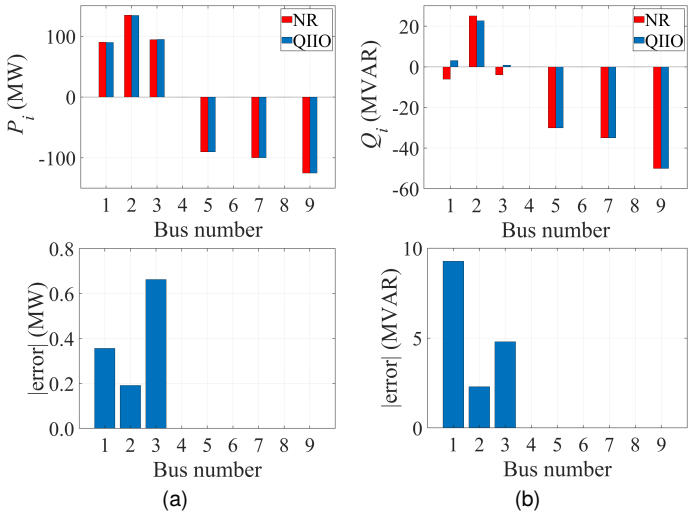



Figure 6.7: OPF results for the IEEE 9-bus test system. The top row shows the computed values for (a) $P_i$ and (b) $Q_i$. The bottom row shows the absolute errors between QIIO and NR for corresponding parameters.

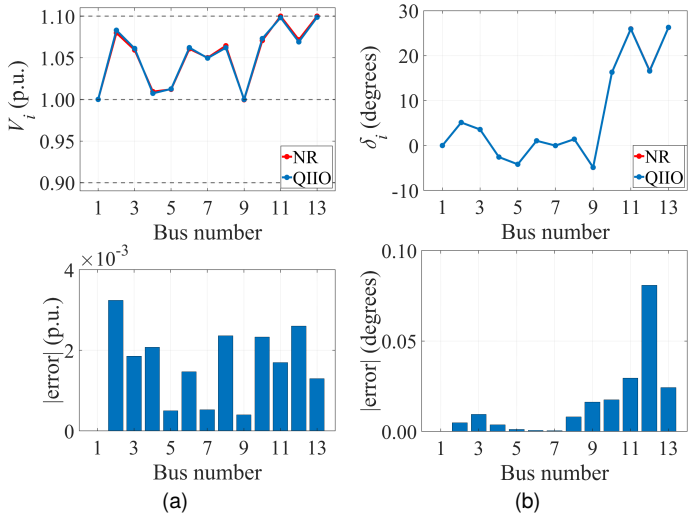



Figure 6.8: OPF results for the RES-integrated IEEE 9-bus test system. The top row shows the computed values for (a) $V_i$ and (b) $\delta_i$. The bottom row shows the absolute errors between QIIO and NR for corresponding parameters.

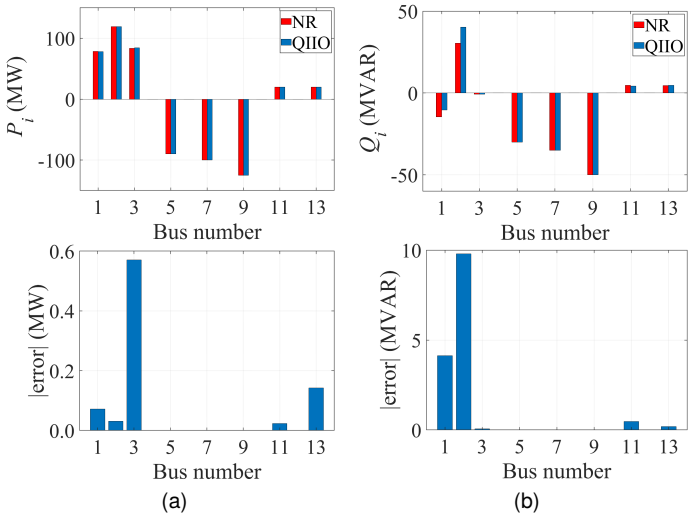



Figure 6.9: OPF results for the RES-integrated IEEE 9-bus test system. The top row shows the computed values for (a) $P_i$ and (b) $Q_i$. The bottom row shows the absolute errors between QIIO and NR for corresponding parameters.

tolerance, the results show good agreement with those obtained using the NR solver in Pandapower. Furthermore, we observe that QIIO exhibits better scalability in terms of computational processing time compared to the NR solver. As the problem size increases, the growth rate of the computational processing time of QIIO remains lower than that of NR.

While the average and per bus deviations in **P** remain below 1% for all experiments, the corresponding deviations in **Q** are comparatively higher. This behavior is consistent with established findings showing that voltages are generally more sensitive to variations in **Q** than in **P** (e.g., [18]). Consequently, small deviations in **V** can lead to amplified errors in computed **Q**. Furthermore, we observe that while QA and QIIO satisfy the overall accuracy criteria based on average error across all buses, QA demonstrates superior accuracy per bus. In this perspective, a bus-specific residual, rather than relying on average deviations, can potentially enhance accuracy per bus.

For the current AQC hardware, which is not yet as readily accessible or scalable as mature classical hardware, computational overhead remains a key limitation for real-time applications. Nevertheless, the information flow within the proposed QHIL framework operates effectively and demonstrates reliable communication between components. That is, once fault-tolerant quantum hardware becomes available, the framework is expected to respond efficiently to dynamic changes in system topology or operating conditions in real-time, as previously obtained solutions can be used as warm starts, further accelerating convergence.

Furthermore, current QA is constrained by limited qubit counts, sparse

connectivity, and restricted precision in coupling coefficients, which collectively hinder their application to large-scale power systems. These challenges are expected to diminish as quantum hardware matures. In contrast, QIIO can handle up to 100,000 decision variables and therefore does not currently face the same scalability limitations. Another key observation is that middleware design and data exchange efficiency are highly platform-specific, that is, different hardware platforms require specific middleware implementations even when solving the same PF and OPF formulations. This highlights the importance of developing standardized, adaptable middleware for real-time applications across various hardware platforms.

## References

[1] **Z. Kaseb**, R. Rane, A. Lekić, M. Möller, A. Khodaei, P. Palensky and P. P. Vergara. 'Quantum hardware-in-the-loop for optimal power flow in renewable-integrated power systems'. In: *IEEE Transactions on Power Systems* (May 2026). DOI: 10.1109/TPWRS.2026.3696764.

[2] M. S. Alam, F. S. Al-Ismail, A. Salem and M. A. Abido. 'High-level penetration of renewable energy sources into grid utility: Challenges and solutions'. In: *IEEE access* 8 (2020), pp. 190277–190299.

[3] H. van Asselt, P. Fragkos, L. Peterson and K. Fragkiadakis. 'The environmental and economic effects of international cooperation on restricting fossil fuel supply'. In: *International Environmental Agreements: Politics, Law and Economics* 24.1 (2024), pp. 141–166.

[4] **Z. Kaseb**, S. Orfanoudakis, P. P. Vergara and P. Palensky. 'Physics-informed neural network with adaptive activation for power flow'. In: *International Journal of Electrical Power & Energy Systems* 174 (2026), p. 111525. DOI: 10.1016/j.ijepes.2025.111525.

[5] M. Huneault and F. D. Galiana. 'A survey of the optimal power flow literature'. In: *IEEE Transactions on Power Systems* 6.2 (1991), pp. 762–770.

[6] N. Mbuli and W. Ngaha. 'A survey of big bang big crunch optimisation in power systems'. In: *Renewable and Sustainable Energy Reviews* 155 (2022), p. 111848.

[7] S. Mhanna and P. Mancarella. 'An exact sequential linear programming algorithm for the optimal power flow problem'. In: *IEEE Transactions on Power Systems* 37.1 (2021), pp. 666–679.

[8] J. A. Momoh, R. Adapa and M. El-Hawary. 'A review of selected optimal power flow literature to 1993. I. Nonlinear and quadratic programming approaches'. In: *IEEE Transactions on Power Systems* 14.1 (1999), pp. 96–104.

[9] P. Fortenbacher and T. Demiray. 'Linear/quadratic programming-based optimal power flow using linear power flow and absolute loss approximations'. In: *International Journal of Electrical Power & Energy Systems* 107 (2019), pp. 680–689.

[10] D. N. d. Silva, B. R. Pereira Júnior and G. R. M. d. Costa. 'A novel MILP-newton approach for power flow analysis'. In: *IET Generation, Transmission & Distribution* 15.3 (2021), pp. 518–532.

[11] A. F. Zobaa, S. A. Aleem and A. Y. Abdelaziz. *Classical and recent aspects of power system optimization*. Academic Press, 2018.

[12] A. Ali, A. Hassan, M. Keerio, N. H. Mugheri, G. Abbas, M. Hatatah, E. Touti and A. Yousef. 'A novel solution to optimal power flow problems using composite differential evolution integrating effective constrained handling techniques'. In: *Scientific Reports* 14.1 (2024), p. 6187.

[13] A. A. Z. Diab, A. M. Abdelhamid and H. M. Sultan. 'Comprehensive analysis of optimal power flow using recent metaheuristic algorithms'. In: *Scientific Reports* 14.1 (2024), p. 13422.

[14] **Z. Kaseb**, M. Möller, P. Palensky and P. P. Vergara. 'Power flow and optimal power flow using quantum and digital annealers: a computational scalability analysis'. In: *arXiv preprint* (2026). DOI: arXiv:2505.15978.

[15] R. Wagle, G. Tricarico, A. F. S. Melo, V. Rosero-Morillo, A. Shukla, F. Gonzalez-Longatt *et al.* 'Real-time cyber-physical power system testbed for optimal power flow study using co-simulation framework'. In: *IEEE Access* 12 (2024), pp. 150914–150929.

[16] R. Wagle, P. Sharma, C. Sharma, M. Amin, J. L. Rueda and F. Gonzalez-Longatt. 'Optimal power flow-based reactive power control in smart distribution network using real-time cyber-physical co-simulation framework'. In: *IET Generation, Transmission & Distribution* 17.20 (2023), pp. 4489–4502.

[17] S. Chanda, M. Mohanpurkar and R. Hovsapian. 'Architecture for Quantum-in-the Loop Real-Time Simulations for Designing Resilient Smart Grids'. In: *2023 13th International Symposium on Advanced Topics in Electrical Engineering (ATEE)*. IEEE. 2023, pp. 1–6.

[18] S. Talkington, D. Turizo, S. Grijalva, J. Fernandez and D. K. Molzahn. 'Conditions for Estimation of Sensitivities of Voltage Magnitudes to Complex Power Injections'. In: *IEEE Transactions on Power Systems* 39 (1 Jan. 2024), pp. 478–491. ISSN: 0885-8950. DOI: 10.1109/TPWRS.2023.3237505.

# 7

# Reinforcement Learning for Newton-Raphson Initialization

*We propose using reinforcement learning (RL) to optimize the initialization of the Newton-Raphson (NR) method, thereby improving its convergence under poor initialization or in extreme operating scenarios, e.g., high levels of renewable energy penetration. We also introduce a novel quantum-enhanced RL environment update mechanism to mitigate the high computational cost of evaluating power system states over a combinatorially large action space at each RL timestep by formulating the voltage adjustment task as a quadratic unconstrained binary optimization problem. Specifically, Ising machines are integrated into the RL environment update to evaluate state transitions using a problem Hamiltonian designed for the power flow (PF) problem in Chapter 5. The reader is referred to Sections 2.1 and 2.2 for background on PF and adiabatic quantum computing (AQC), upon which this chapter builds.*

Parts of this chapter have been published as arXiv preprint **2511.20237** (2026) with the title: "Reinforcement learning for Newton-Raphson initialization in stressed AC power flow with a quantum annealing update mechanism" [1].

## 7.1. Motivation

Several studies have explored techniques to improve the initialization of the Newton-Raphson (NR) method. For example, integrating the NR method with gradient descent and computational graphs has been proposed to enhance convergence and computational efficiency, as well as address challenges in large-scale grids [2]. Another example is the development of parallel PF solvers that employ high-performance computing to improve the scalability and efficiency of PF analysis across extensive grids [3]. Furthermore, holomorphic theory offers a non-iterative alternative that guarantees convergence to the correct operation solution, thereby mitigating issues related to ill-conditioned systems and divergence associated with the traditional NR method [4].

Machine learning (ML) approaches have also been used to predict high-quality initial conditions from historical data, thereby improving the convergence properties of the NR method [5]. For example, convolutional neural networks have been used to provide improved initial estimates of voltage magnitude and angle, which have been shown to significantly reduce solution iterations and time. Techniques, such as Semi-Definite Programming and Second-Order Cone relaxations, have also been used to convexify the nonconvex PF problem, providing feasible solutions that can serve as effective initializations for iterative methods, such as the NR method [6]. Adaptive convergence enhancement strategies have also been developed to improve the robustness of PF analysis in distribution networks [7, 8]. These strategies integrate mathematical techniques, such as the superposition theorem, graph theory, and the Kron reduction method, to enhance the convergence properties of the NR method in unbalanced distribution networks. By incorporating these methods, NR can better handle the complexities and asymmetries inherent in distribution networks, leading to more reliable PF analysis.

Despite these advancements, challenges persist in selecting initial values that ensure convergence across diverse operating conditions. For example, the effectiveness of adaptive strategies can be limited by the specific characteristics of the distribution network, such as the degree of imbalance, the presence of distributed energy resources, and varying load profiles [9]. Consequently, there is a pressing need for more adaptive, scalable approaches to selecting initial values to enhance convergence. Reinforcement Learning (RL) [10] enables an agent to learn through trial and error and make sequential decisions by interacting with an environment. In electricity grid applications, RL has emerged as a promising approach for addressing complex control problems, such as voltage regulation and PF optimization, especially in the presence of uncertainties and nonlinearities [11, 12]. In this regard, RL agents can also outperform traditional NR initialization techniques by dynamically adapting to real-time conditions using learned system-specific patterns and past experiences [13].

Motivated by this research gap and the combinatorial approach presented in Chapter 5, we propose an RL-based NR initialization that steers the NR method toward regions of the solution space with a higher likelihood of convergence. Experimental results indicate that even under challenging initial conditions, the RL agent can refine the initial guess, allowing NR to converge reliably within a limited

number of iterations. In doing so, the agent must determine an action at each RL timestep, specifically, choosing increments to adjust the voltage magnitudes and angles across all buses. A classical solver, e.g., NR, then evaluates the power system state resulting from a single combination of these adjustments per RL timestep. Nevertheless, due to the combinatorial nature of voltage modifications, classical solvers can become computationally inefficient when evaluating state transitions [14]. By formulating the voltage adjustment task as a quadratic unconstrained binary optimization (QUBO) problem, Ising machines can efficiently explore the high-dimensional action space to identify optimal or near-optimal voltage updates [15, 16], thereby improving scalability and accelerating convergence. Therefore, a novel quantum-enhanced RL environment update mechanism is proposed to strategically guide initialization.

## 7.2. Results

Experiments are conducted on 4- and 14-bus test systems. Classical experiments typically involve an ML framework that encompasses the implementation of the RL environment and deep learning algorithms for agent training. These classical algorithms are implemented in Python and executed on SURF's high-performance computing (HPC) Cloud. A dedicated workspace, configured with 16 CPU cores and 64 GB of RAM, running Ubuntu 22.04 Linux, is used. Quantum and digital experiments comprise the algorithm used to update the RL environment, implemented in Python and executed on D-Wave's Advantage™ system (QA) and Fujitsu's Quantum-Inspired Integrated Optimization software (QIIO), respectively.

### 7.2.1. RL for accelerating NR convergence

For a standard 4-bus test system [17], consisting of one *slack* bus and three *PQ* buses (see Figure 7.1), the basin of attraction and regions of initial voltage magnitudes $(p.u.)$ and voltage phase angles $(degrees)$ at *PQ* buses are illustrated in Figure 7.2. For each subplot, the initial values of the corresponding bus are varied while the initial values for the other buses are kept fixed at $1\angle 0$ $(p.u.)$. Accordingly, improper initialization of complex voltages in certain regions can lead to slow convergence of the NR method or even divergence. To mitigate this risk, we train an RL agent to adjust the initial complex voltage estimates, steering them toward regions that ensure NR convergence while minimizing the number of iterations required for PF analysis. Experiments for the 4-bus system are implemented in Python and executed on SURF's HPC Cloud.

Algorithm 3 represents an RL episode for selecting complex voltage adjustments following a policy. An episode in RL refers to one complete run of the agent in the environment, from the initial state until the agent reaches a terminal state or until the maximum episode length $T$ is reached. In the case of the latter, the episode is terminated without the agent reaching a terminal state. Each step refers to one interaction between the agent and the environment. At each step, the agent selects an action. Next, the environment processes that action, and transitions into the next

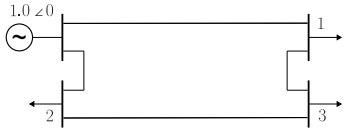



Figure 7.1: Schematic representation of the 4-bus test system. The system includes a *slack* bus and three *PQ* buses.

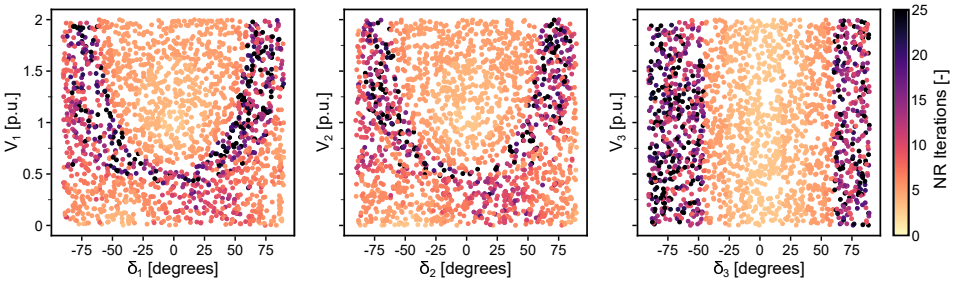



Figure 7.2: Basin of attraction for initial values at *PQ* buses for the 4-bus test system. Each subplot shows the number of NR iterations (with the heatmap) needed to perform PF analysis for the given voltage magnitude ($p.u.$) and phase angle ($degrees$).

state. Then, the agent receives a new state and reward. The timestep is simply the interval between two consecutive steps. Over the course of an episode, the agent takes multiple timesteps to reach a solution.

**Algorithm 3** RL episode for selecting complex voltage adjustments following a policy $\pi$.

**Initialize** Environment from test case data, policy $\pi$, $k_{max}$, $T$, $t \leftarrow 0$
Get initial state $s_0 = [\mathbf{V}^0, \theta^0, k_0]$
**while** $t < T$ **do**
    Based on the current state $s_t$, the agent selects action:
        $a_t \sim \pi = [\mathbf{\Delta}\mu^{\mathbf{t}}, \mathbf{\Delta}\omega^{\mathbf{t}}]$
    Perform environment update using action $a_t$
    Get new state $s_{t+1}$, reward $r_{t+1}$
    **if** $k_t \leq k_{max}$ **then**
        Episode done
        **break**
    **end if**
    $t \leftarrow t + 1$
    $s_t \leftarrow s_{t+1}$
**end while**
Episode terminated

The 4-bus test system is considered for the experiment, which is characterized by the following Markov decision process (MDP) parameters:

- The initial state distribution $d_0$ consists of initial voltage magnitudes and voltage phase angles at *PQ* buses $i \in \{1, 2, 3\}$, respectively denoted as $V_i^0$ and $\theta_i^0$, where $\mathbf{V}^0 \in (0, 2]$ $(p.u.)$ and $\theta^0 \in (-90, 90]$ $(degrees)$ are drawn randomly from a uniform distribution. Also, $d_0$ includes the number of NR iterations required to converge $k_0$ and is obtained by performing PF analysis using NR. The admittance matrix $\mathbf{Y} = \mathbf{G} + j\mathbf{B}$ is available in the Power System Test Cases library[1]. All other initial constant values further characterizing the initial state distribution are shown in Table 7.1.
- The state $s_t$ at timestep $t \in \{0, ..., T\}$ is represented as a vector $s_t = [\mathbf{V}^t, \theta^t, k_t]$ containing the voltage magnitude $\mathbf{V}^t \in (0, 2]$ $(p.u.)$, voltage phase angle $\theta^t \in (-90, 90]$ $(degrees)$ at *PQ* buses, and the number of NR iterations required to converge $k_t \in [0, 50]$ in the current timestep $t$. If NR requires more than 50 iterations or diverges, $k_t$ is set to 50.
- The actions consist of a vector $a_t = [\mathbf{\Delta\mu^t}, \mathbf{\Delta\omega^t}]$ for *PQ* buses, where $\Delta\mu_i^t \in (-0.5, 0.5]$ $(p.u.)$ and $\Delta\omega_i^t \in (-0.25, 0.25]$ $(p.u.)$ represent the complex voltage adjustments in rectangular coordinates at timestep $t$ and bus $i$, respectively.
- The reward $r_t$ is defined solely by a penalizing term based on the norm of the residual vector $[\mathbf{\Delta P}^t, \mathbf{\Delta Q}^t]$ for *PQ* buses, where $\Delta P_i^t$ and $\Delta Q_i^t$ are the active and reactive power mismatches for bus $i$ resulting from applying the updated initial values in PF analysis, $r_t = -||[\mathbf{\Delta P}^t, \mathbf{\Delta Q}^t]||$, where $r_t$ refers to $R(s_t, a_t)$ in (2.10).

Table 7.1: Initial constant values for the *slack* bus and *PQ* buses $i$ for the given scenario based on the 4-bus test system.

| Parameter | Value |
|---|---|
| Slack bus voltage | $1.0\angle 0.0$ |
| Active power demand ($\mathbf{P}^D$) | [1.7, 2.0, 0.8] |
| Reactive power demand ($\mathbf{Q}^D$) | [1.05, 1.24, 0.49] |

We design the RL agent to learn a policy $\pi$ that guides $V_i^0$ and $\theta_i^0$ at *PQ* buses, such that NR converges within $k_{max} = 3$ iterations, where $k_{max}$ is the maximum number of allowed iterations. The Pandapower Python package [18] is used to perform NR. During training, each episode consists of a maximum of $T = 20$ timesteps, after which the environment terminates even if convergence is not achieved. To optimize the policy, we employ PPO [19] using the implementation from Stable-Baselines3 [20], and train the RL agent for $1 \times 10^5$ timesteps. The

[1] https://pandapower.readthedocs.io/en/v2.2.2/networks/power_system_test_cases.html

hyperparameters used for training are summarized in Table 7.2. Note that these hyperparameters are selected based on a systematic sensitivity analysis[2], where:

- The learning rate is searched over the interval $[10^{-6}, 10^{-2}]$.
- The discount factor is selected from the range $[0.9, 1]$ with a step size of $0.01$.
- The entropy coefficient is varied within $[0, 1]$ with a step size of $0.1$.
- The policy and value networks are evaluated for architectures with hidden layer counts in the range $[1, 10]$.
- The number of neurons per hidden layer is selected from the set $\{16, 32, 64, 128\}$.

Table 7.2: Hyperparameter settings for training the PPO agent based on the 4-bus test system.

| **Hyperparameter** | **Value** |
|---|---|
| Learning rate ($\alpha$) | $0.7 \times 10^{-4}$ |
| Discount factor ($\gamma$) | 0.9 |
| Number of steps | 2048 |
| Batch size | 64 |
| Number of epochs | 10 |
| Clipping range | 0.2 |
| Entropy coefficient | 0.0 |
| Policy Network Architecture | [32, 32, 32] |
| Value Network Architecture | [32, 32, 32] |
| Activation Function | ReLU |

Figure 7.3 illustrates the evolution of episode reward and the number of NR iterations over RL timesteps during training for the 4-bus test system. As training progresses, the episode reward increases while the number of NR iterations decreases.

### 7.2.2. Scalability of RL for accelerating NR convergence

To assess the scalability of the proposed approach, we extend our investigation to a 14-bus test system, consisting of one *slack* bus, four *PV* buses, and nine *PQ* buses. Scalability here refers to the ability to maintain or improve performance as the size and complexity of the test system increase. Experiments for the 14-bus system are conducted on SURF's HPC Cloud within the same workspace as for the 4-bus, and the agent is trained for $2.5 \times 10^4$ timesteps. Similar to the 4-bus test system, $k_{max} = 3$ and each episode consists of a maximum of $T = 20$ timesteps. The initial constant values for the given scenario, based on which the experiments are

[2] An exhaustive search across 40 unique hyperparameter combinations is conducted using the Optuna Python package [21]. The hyperparameters are sampled from predefined ranges with fixed step sizes.

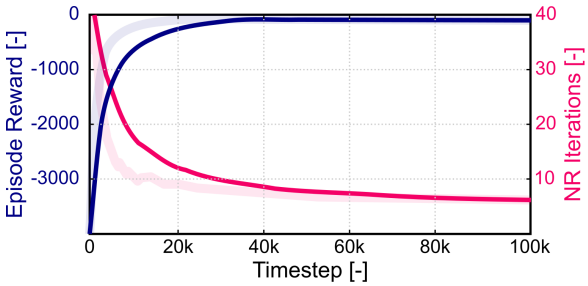



Figure 7.3: Training trajectory of the RL agent for the 4-bus test system. The graph shows the evolution of episode reward and the number of NR iterations over $1 \times 10^5$ timesteps.

performed, are summarized in Table 7.3. The hyperparameters used for training are summarized in Table 7.4. These hyperparameters are chosen based on a systematic sensitivity analysis, using the same ranges and approach as applied to the 4-bus test system.

Table 7.3: Initial constant values for the *slack* bus and *PV* and *PQ* buses *i* for the given scenario based on the 14-bus test system.

| **Parameter** | **Value** |
|---|---|
| Slack bus voltage | $1.0\angle 0.0$ |
| Active power generation ($\mathbf{P}^{\mathrm{G}}$) | [ 0, 0.4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ] |
| Active power demand ($\mathbf{P}^{\mathrm{D}}$) | [ 0, 0, 0, 0.478,0.076, 0, 0, 0, 0.295,0.09, 0.035,0.061,0.135,0.149 ] |
| Reactive power demand ($\mathbf{Q}^{\mathrm{D}}$) | [ 0, 0, 0, 0.039,0.016, 0, 0, 0, 0.166,0.058,0.018,0.016,0.058,0.05 ] |

Table 7.4: Hyperparameter settings for training the PPO agent based on the 14-bus test system.

| **Hyperparameter** | **Value** |
|---|---|
| Learning rate ($\alpha$) | $0.5 \times 10^{-4}$ |
| Discount factor ($\gamma$) | 0.9 |
| Number of steps | 2048 |
| Batch size | 64 |
| Number of epochs | 10 |
| Clipping range | 0.2 |
| Entropy coefficient | 0.0 |
| Policy Network Architecture | [64, 64, 64] |
| Value Network Architecture | [64, 64, 64] |
| Activation Function | ReLU |

Here, the RL agent is trained to learn voltage adjustments that accelerate NR convergence from a broader and more complex initial state space. Despite the increased number of buses and higher-dimensional state and action spaces, the agent successfully learns a policy that improves NR convergence, as shown in Figure 7.4. Specifically, the training trajectory demonstrates a steady increase in

cumulative reward $r_t$ alongside a reduction in the average number of NR iterations $k_t$ required for convergence. This finding suggests that the agent effectively generalizes the learned strategy from smaller to larger systems and adapts to the increased complexity without a significant loss in performance.

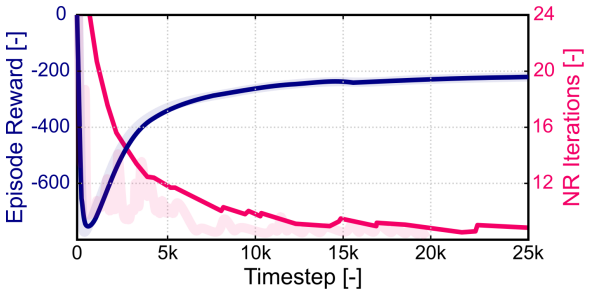



Figure 7.4: Training trajectory of the RL agent for the 14-bus test system. The graph shows the evolution of episode reward and (b) the number of NR iterations over $2.5 \times 10^4$ timesteps.

### 7.2.3. Quantum-enhanced RL environment update mechanism

The trained RL agent successfully shifts initial complex voltages into regions that require fewer NR iterations. However, one remaining challenge is to update the state $s$ more efficiently based on the agent's selected actions $a$. Given the combinatorial nature of possible complex voltage adjustments, we extend the experiment to further accelerate NR convergence by introducing a quantum-enhanced RL environment update mechanism. Specifically, the environment update is based on a problem Hamiltonian defined for a combinatorial reformulation of PF equations. We then use Ising machines to determine optimal voltage updates within the environment via quantum/digital annealing. Figure 7.5 shows the episode structure, state transitions, and quantum-enhanced RL environment updates. The agent interacts with the environment through key functions, including reset, action, state, and reward, while quantum/digital annealing refines complex voltage adjustments to further accelerate the convergence of NR.

The pseudo-code of the proposed update mechanism, i.e., the *Environment Update* block in Figure 7.5, can be found in Chapter 5. Accordingly, the power system data is first initialized, including load power demands, $\mathbf{P}^{\mathrm{D}}$ and $\mathbf{Q}^{\mathrm{D}}$, generation power outputs $\mathbf{P}^{\mathrm{G}}$, and the admittance matrix $\mathbf{Y}$. The RL agent then provides the action variables, which correspond to complex voltage adjustments in rectangular coordinates, $\mathbf{\Delta}\mu$ and $\mathbf{\Delta}\omega$. The voltage magnitudes $\mathbf{V}$ and phase angles $\theta$ at *PQ* buses are extracted from the state $s_t$ and converted to rectangular coordinates, yielding $\mu = V\cos(\theta)$ and $\omega = V\sin(\theta)$. The QUBO model presented in Chapter 5 for the combinatorial PF problem is used. The problem Hamiltonian (5.6) is solved using Ising machines, and the solvers are QA and QIIO with a predefined number of reads $n_{\mathrm{read}}$. After solving the minimization problem (5.6), the

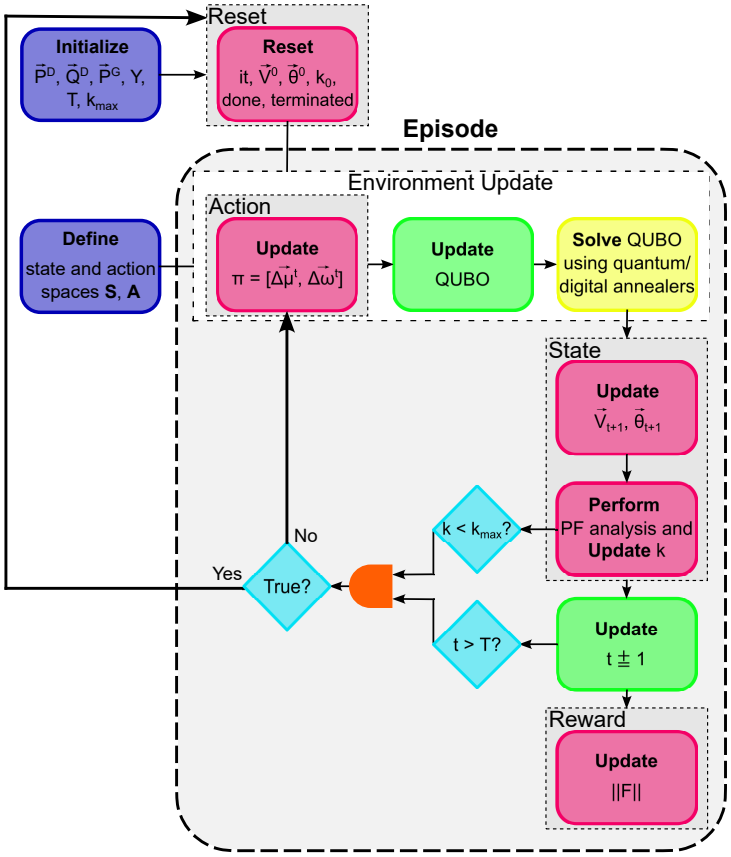



Figure 7.5: Schematic of the quantum-enhanced RL update mechanism for PF analysis based on the QUBO representation. The diagram depicts the episode structure, state transitions, and quantum-enhanced RL environment updates. Key functions, i.e., reset, action, state, and reward, govern the agent's interaction, while quantum annealing refines voltage adjustments to accelerate the convergence of NR.

obtained bitstring $\mathbf{x}$ is used to update $\mu$ and $\omega$ in accordance with (5.3), which ultimately yield the next state $s_{t+1}$.

The impact of the proposed update mechanism is evaluated by comparing the performance of the RL agent trained with the proposed quantum-enhanced update mechanism against one trained without it (see Figure 7.3) for the 4-bus test system for the given load scenario specified in Table 7.1. The update mechanism is performed using QIIO. The evolution of episode reward and the number of NR iterations over $1 \times 10^5$ timesteps are shown in Figure 7.6. It can be seen that the quantum-enhanced RL (QRL) agent achieves a higher maximum episode reward after $1 \times 10^5$ compared to that obtained by the RL agent (see Figure 7.3).

Figure 7.7 further visualizes a comparative analysis of the performance of classical RL and QRL agents in accelerating NR convergence for PF analysis based on the 4-bus test system across four difficult, challenging scenarios. These scenarios are selected from regions that require a large number of NR iterations to converge, as shown in Figure 7.2, and are excluded from the training. The update mechanism is performed using QIIO. While the classical RL agent still reduces the NR iterations compared to the initial conditions for the presented challenging scenarios, it still requires multiple RL timesteps to optimize the complex voltage adjustments, with the final NR iteration count ranging between 8 and 19. In contrast, QRL achieves convergence with as few as 3 to 7 NR iterations across all scenarios, often requiring only a single RL timestep for the complex voltage updates. This finding indicates that QRL is more effective in finding optimal complex voltage adjustments and facilitates faster convergence. Furthermore, the final complex voltages obtained by the QRL agent exhibit a more balanced and physically meaningful distribution than those obtained by the classical RL agent.

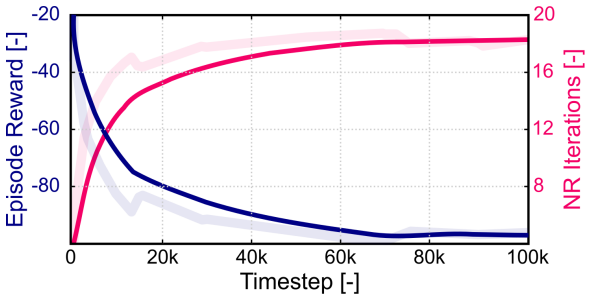



Figure 7.6: Training trajectory of the QRL agent for the 4-bus test system. The graph shows the evolution of episode reward and the number of NR iterations over $1 \times 10^5$ timesteps. The update mechanism is performed using QIIO.

### 7.2.4. Quantum Hardware Implementation

To further validate the applicability of the proposed quantum-enhanced RL environment update mechanism in practical settings, we extend our study by performing additional experiments using QA for the 4-bus test system. Due to the high computational cost and access limitations associated with real quantum hardware, we restrict these experiments to a maximum of $2.5 \times 10^4$ timesteps.

Figure 7.8 shows the evolution of episode reward and the number of NR iterations over $2.5 \times 10^4$ timesteps during the training of the QRL agent, where the quantum experiments are performed using QA. The maximum episode rewards obtained by the QRL agents trained using QIIO and QA are comparable, and both exceed that of the classical RL agent by more than a factor of five (see Figures 7.3 and 7.6). These findings suggest that the proposed quantum-enhanced update mechanism for RL environments improves the performance of RL agents. QA performs comparably to

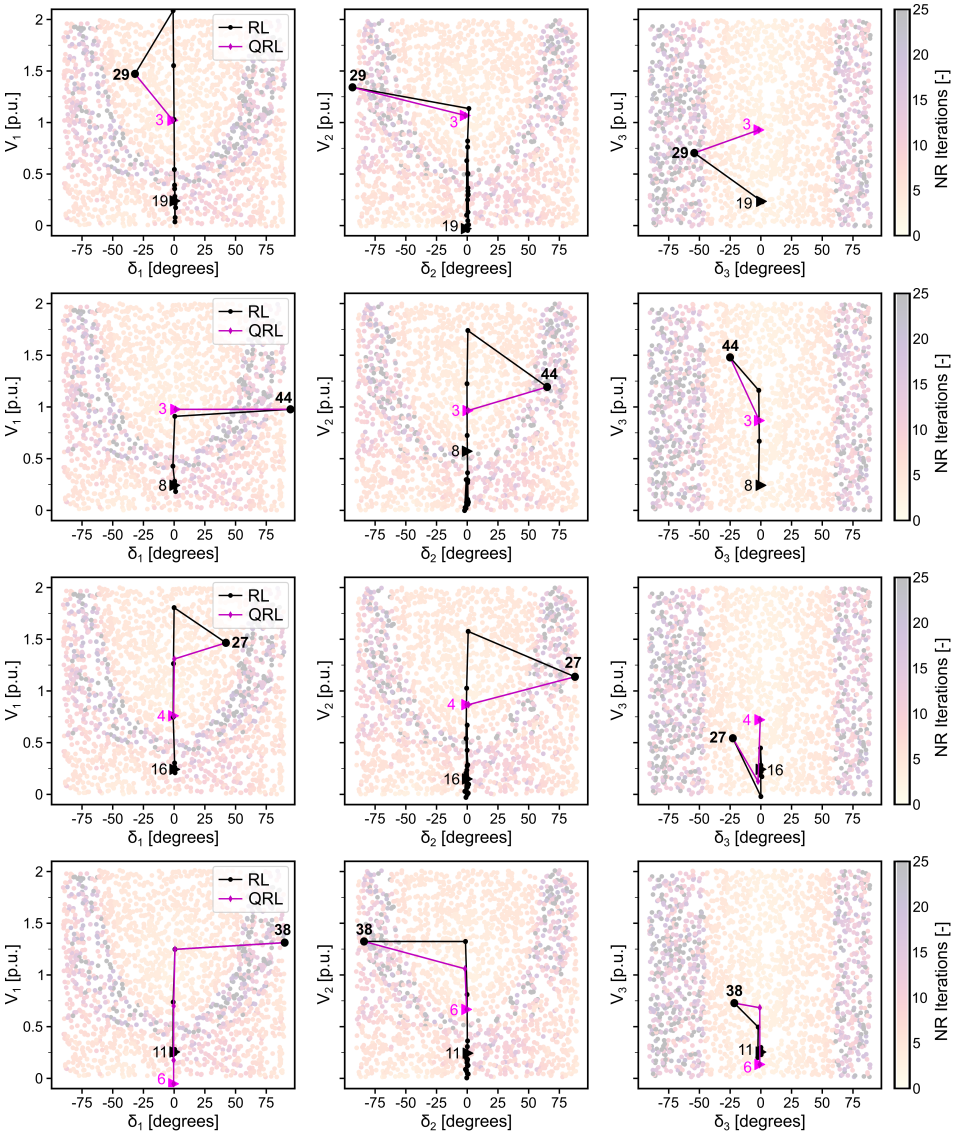



Figure 7.7: Comparison of classical RL and QRL for accelerating NR convergence in PF analysis for the 4-bus test system across four experiments. The initial and final states, at *PQ* buses, and the number of NR iterations, are highlighted. The RL and QRL agents are trained over $1 \times 10^5$ timesteps. The update mechanism is performed using QIIO.

QIIO in accelerating NR convergence, with QA slightly outperforming QIIO within the evaluated timesteps. Therefore, provided the quantum annealer can accommodate

the required number of binary variables and the problem graph can be successfully embedded into the hardware's connectivity graph, Ising machines can deliver competitive performance. This observation aligns with prior studies comparing current quantum and quantum-inspired hardware for electricity grid applications [22].

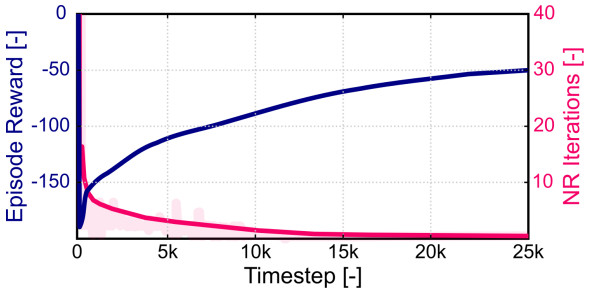



Figure 7.8: Training trajectory of the QRL agent for the 4-bus test system. The graph shows the evolution of episode reward and the number of NR iterations over $2.5 \times 10^4$ timesteps. The update mechanism is performed using QA.

## 7.3. Discussion and Conclusions

When it comes to advanced computational technologies, two distinct perspectives emerge. One group of researchers and practitioners advocates their transformative potential, while another remains skeptical of their reliability and real-world applicability. In this study, we present an example that integrates several emerging techniques, i.e., QC, AI, and optimization, to address a long-standing challenge in numerical analysis. Interestingly, the final solution is ultimately obtained by a well-established and trusted classical solver, thus bridging the gap between innovative and conservative approaches, but also making this study appealing to both communities.

Our findings demonstrate that deep RL and its quantum-enhanced equivalent can effectively optimize the iteration counts of the NR method, which is one of the most widely used numerical algorithms for problems lacking analytical solutions. It is well known that the NR method can fail to converge under certain conditions, even when the system of equations is not particularly large. A common cause of such failure is poor initialization of the solver's starting guesses. Yet, even at the time of writing, decision-makers in industry often exhibit resistance toward adopting advanced computational techniques, e.g., AI and QC, for real-world problems.

Among NR's many applications, PF analysis is a critical task in electricity grid planning and operation. A classical NR solver can diverge under poor initialization or in extreme operating scenarios, such as high demand or high levels of renewable energy penetration, even when a physical solution exists. In this context, we propose optimizing NR initialization to avoid divergence using a combination of QC,

AL, and optimization, with the final solution obtained from classical NR solvers. The proposed quantum-enhanced RL approach to accelerate NR convergence, however, is generalizable to other problems where NR or similar numerical solvers are employed. For example, our previous work has shown that the combinatorial way of modeling existing models can be applied to any system of equations with continuous variables, provided that the objective can be formulated as a root-finding problem [15].

The results highlight the potential of integrating quantum computing techniques with modern machine learning methods to address long-standing challenges in electricity grid applications. As quantum hardware continues to evolve, future work may explore extending the proposed approach to real-time PF operations, but also large-scale power systems, eventually unlocking new levels of computational efficiency and grid reliability.

The Python process for the quantum algorithm used approximately 64 GB of memory (RAM), exceeding the system's capacity and triggering the out-of-memory (OOM) killer. Upon investigation, we identified a memory leak potentially caused by repeatedly wrapping and recreating Gym environments without proper cleanup, failing to close open files (which prevents buffer release), and long-running deep RL training loops using PyTorch (2.6.0). To mitigate this, we introduced subprocess isolation and trained in defined chunks. This implementation ensures that all memory is released back to the operating system after each training segment. If any memory leaks occur within PyTorch, Gym, or the environment, they are confined and eliminated when the subprocess terminates.

As system size increases, training a single RL agent to handle all bus types becomes challenging. A promising direction for future work is to adopt a multi-agent RL approach, where separate agents are trained for each bus type. Centralized training with decentralized execution could enable coordination during training while maintaining scalability at execution time. Another potential avenue is the integration of graph neural networks, which explicitly exploit the topology of electricity grids to improve the generalization and scalability of policy/value networks. Finally, a systematic benchmark of the proposed initialization approach against standard heuristics, such as flat-start, would further highlight the convergence gains attributable to RL.

# References

[1] **Z. Kaseb**, M. Möller, L. Spoor, J. J. Guo, Y. Xiang, P. Palensky and P. P. Vergara. 'Reinforcement learning for Newton-Raphson initialization in stressed AC power flow with a quantum annealing update mechanism'. In: *arXiv preprint* (2026). DOI: arXiv:2511.20237.

[2] M. Barati. 'Enhancing ACPF Analysis: Integrating Newton-Raphson Method with Gradient Descent and Computational Graphs'. In: *2024 IEEE Texas Power and Energy Conference (TPEC)*. 2024, pp. 01–05. DOI: 10.1109/TPEC60005.2024.10472209.

[3] B. Wang, J. Bachan and C. Chan. 'ExaGridPF: A parallel power flow solver for transmission and unbalanced distribution systems'. In: *2018 IEEE Power & Energy Society Innovative Smart Grid Technologies Conference (ISGT)*. 2018, pp. 1–5. DOI: 10.1109/ISGT.2018.8403383.

[4] A. Trias. 'The Holomorphic Embedding Load Flow method'. In: *2012 IEEE Power and Energy Society General Meeting*. 2012, pp. 1–8. DOI: 10.1109/PESGM.2012.6344759.

[5] S. N. Okhuegbe, A. A. Ademola and Y. Liu. 'A Machine Learning Initializer for Newton-Raphson AC Power Flow Convergence'. In: *2024 IEEE Texas Power and Energy Conference (TPEC)*. 2024, pp. 1–6. DOI: 10.1109/TPEC60005.2024.10472261.

[6] D. K. Molzahn and I. A. Hiskens. 'Convex Relaxations of Optimal Power Flow Problems: An Illustrative Example'. In: *IEEE Transactions on Circuits and Systems I: Regular Papers* 63.5 (2016), pp. 650–660. DOI: 10.1109/TCSI.2016.2529281.

[7] L. A. Kraft and G. T. Heydt. 'Adaptive Acceleration Factors for the Newton-Raphson Power Flow Study'. In: *Electric Machines & Power Systems* 11 (4 Jan. 1986), pp. 337–346. DOI: 10.1080/07313568608909188.

[8] N.-C. Yang and C.-H. Tseng. 'Adaptive convergence enhancement strategies for Newton-Raphson power flow solutions in distribution networks'. In: *IET Generation, Transmission & Distribution* 18 (13 July 2024), pp. 2339–2352. DOI: 10.1049/gtd2.13197.

[9] J. Avilés, D. Guillen, L. Ibarra and J. D. Dávalos-Soto. 'Reconfiguration of active distribution networks as a means to address generation and consumption dynamic variability'. In: *IET Generation, Transmission & Distribution* 18 (19 Oct. 2024), pp. 3120–3137. DOI: 10.1049/gtd2.13264.

[10] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018.

[11] C. Ma, A. Li, Y. Du, H. Dong and Y. Yang. 'Efficient and scalable reinforcement learning for large-scale network control'. In: *Nature Machine Intelligence* (Sept. 2024), pp. 1006–1020. DOI: 10.1038/s42256-024-00879-7.

[12] Y. Wang, X. Yu and W. Zhang. 'An improved reinforcement learning-based differential evolution algorithm for combined economic and emission dispatch problems'. In: *Engineering Applications of Artificial Intelligence* 140 (Jan. 2025), p. 109709. DOI: 10.1016/j.engappai.2024.109709.

[13] Q. Yang, G. Wang, A. Sadeghi, G. B. Giannakis and J. Sun. 'Two-Timescale Voltage Control in Distribution Grids Using Deep Reinforcement Learning'. In: *IEEE Transactions on Smart Grid* 11.3 (2020), pp. 2313–2323. DOI: 10.1109/TSG.2019.2951769.

[14] F. Chicano, G. Luque, Z. A. Dahi and R. Gil-Merino. 'Combinatorial optimization with quantum computers'. In: *Engineering Optimization* 57 (1 Jan. 2025), pp. 208–233. DOI: 10.1080/0305215X.2024.2435538.

[15] **Z. Kaseb**, M. Möller, P. Palensky and P. P. Vergara. 'A framework for solving continuous energy and power system problems using adiabatic quantum computing'. In: *arXiv preprint* (2026). DOI: arXiv:2504.06458.

[16] T. Morstyn and X. Wang. 'Opportunities for quantum computing within net-zero power system optimization'. In: *Joule* 8 (6 June 2024), pp. 1619–1640. DOI: 10.1016/j.joule.2024.03.020.

[17] J. J. Grainger and W. D. Stevenson Jr. *Power System Analysis*. McGraw-Hill, Inc., 1994.

[18] L. Thurner, A. Scheidler, F. Schäfer, J.-H. Menke, J. Dollichon, F. Meier, S. Meinecke and M. Braun. 'pandapower—an open-source python tool for convenient modeling, analysis, and optimization of electric power systems'. In: *IEEE Transactions on Power Systems* 33.6 (2018), pp. 6510–6521. DOI: 10.1109/TPWRS.2018.2829021.

[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov. 'Proximal Policy Optimization Algorithms'. In: *arXiv preprint* (2017). DOI: arXiv:1707.06347.

[20] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus and N. Dormann. 'Stable-baselines3: reliable reinforcement learning implementations'. In: *The Journal of Machine Learning Research* 22.1 (Jan. 2021).

[21] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama. 'Optuna: A Next-Generation Hyperparameter Optimization Framework'. In: *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019.

[22] **Z. Kaseb**, M. Möller, P. P. Vergara and P. Palensky. 'Power flow analysis using quantum and digital annealers: a discrete combinatorial optimization approach'. In: *Nature Scientific Reports* 14.1 (2024), p. 23216. DOI: 10.1038/s41598-024-73512-7.

# 8

# Conclusions

## 8.1. Contributions to Research Questions

This PhD thesis aims to investigate algorithmic innovations that enhance power flow (PF) analysis and optimal power flow (OPF) under the emerging constraints of modern grids. The thesis addresses the research questions introduced in Chapter 1 by examining complementary methodological directions to target challenges associated with classical solvers, as summarized below:

### RQ1 - How can prior physical knowledge be efficiently embedded into neural networks to improve the accuracy, interpretability, and data efficiency for PF analysis?

An end-to-end architecture, called physics-informed neural network for power flow (PINN4PF), is introduced in Chapter 3. PINN4PF includes two important advancements in the training pipeline: (i) a double-head feed-forward NN that aligns with PF analysis, including an activation function that adjusts to the net active and reactive power injection patterns, and (ii) a physics-based loss function that partially incorporates power system topology information through a novel hidden function. PINN4PF therefore offers a straightforward yet practical approach to capturing the complexities of large-scale modern power systems. The results demonstrate that incorporating power system physics into the learning process leads to more physically consistent predictions not only for bus voltages, but also for derived quantities, such as line currents. Beyond improved numerical accuracy, a key insight is that physics-informed learning helps mitigate sensitivity to data and system sizes. The results further highlight essential trade-offs. While PINN4PF improves robustness and data efficiency, the design requires careful formulation of physical constraints, and the training complexity is higher than that of purely data-driven NNs. These findings suggest that physics-informed machine learning is not a drop-in replacement for classical solvers or black-box learning, but rather a complementary paradigm that balances interpretability and generalization. Consequently, this thesis motivates further research into scalable physics-informed architectures and hybrid approaches for PF analysis.

### RQ2 - How can quantum neural networks provide advantages over classical deep learning for PF analysis under data scarcity and noise?

Chapter 4 systematically investigates the application of quantum neural networks (QNNs) for PF analysis. A comprehensive comparison of a classical neural network (NN), a hybrid quantum-classical neural network (QCNN), and a QNN is performed. The comparison is based on (i) generalization ability, (ii) robustness, (iii) training dataset size needed, (iv) training error, and (v) training process stability. The findings clarify important limitations of current QNN approaches. Fully quantum models remain sensitive to circuit depth and noise, which constrain their practical usage on near-term hardware. By contrast, QCNNs demonstrate improved data efficiency and training stability, indicating that hybrid architectures are a more realistic pathway in the mid-term. These observations lead to the conclusion that QNNs should not be viewed as direct replacements for NNs, but as complementary

tools that are particularly advantageous when data availability is limited or when measurements are noisy. Consequently, this thesis motivates further research into scalable hybrid quantum-classical architectures, noise-aware training strategies, and principled criteria for selecting when quantum components provide a tangible benefit. Future work should also investigate the interaction between physical constraints and quantum learning models, as well as their integration with classical solvers. Collectively, these directions are essential for assessing the long-term scientific and practical value of quantum machine learning in PF analysis.

### RQ3 - How can PF and OPF be reformulated for solution on quantum and quantum-inspired hardware?

In Chapter 5, we present combinatorial formulations for both gate-based quantum computing (GQC) and adiabatic quantum computing (AQC) hardware to solve the PF and OPF problems. For GQC, Quantum Approximate Optimization Algorithm (QAOA) is implemented using PennyLane's `lightning.qubit` statevector simulator. For AQC, Ising machines, including D-Wave's Advantage™ system (QA) and Fujitsu's Quantum-Inspired Integrated Optimization software (QIIO), are used. The feasibility of the proposed algorithms is demonstrated for small- to large-scale power systems. The findings confirm that the proposed algorithm consistently converges across various test systems, including complex and ill-conditioned scenarios where conventional solvers often struggle. The results also suggest that QIIO offers a compelling alternative to classical solvers, particularly for problems with high-dimensional, combinatorial structures, whereas GQC implementation remains limited in accuracy under the tested configuration. The significance is that, for the ill-conditioned and combinatorial cases where NR is least reliable, a discrete reformulation solved on QIIO converges by reframing such cases not as numerical failures to be patched but as optimization problems to be solved on a different technique.

### RQ4 - What are the key system-level challenges in deploying quantum and quantum-inspired PF and OPF solvers within real-time power system simulators?

We develop a QHIL framework to bridge the gap between real-time power system modeling and control and AQC hardware in Chapter 6. We also develop a middleware that ensures efficient real-time communication between QIIO and RTDS®. The proposed algorithm in Chapter 5 solves the PF and OPF problems. Based on the observations, lessons learned, and results, we demonstrate the feasibility of the proposed QHIL framework and identify practical challenges associated with near- to mid-future hardware capabilities.

For the current AQC hardware, which is not yet as readily accessible or scalable as mature classical hardware, computational overhead remains a key limitation for real-time applications. Nevertheless, the information flow within the proposed QHIL framework operates effectively and demonstrates reliable communication between components. That is, once fault-tolerant quantum hardware becomes available, the

framework is expected to respond efficiently to dynamic changes in system topology or operating conditions in real time. Another key observation is that middleware design and data exchange efficiency are highly platform-specific; that is, different hardware platforms require specific middleware implementations even when solving the same PF and OPF formulations. This highlights the importance of developing standardized, adaptable middleware for future real-time applications across various hardware platforms.

#### RQ5 - How can reinforcement learning (RL) enhance the convergence reliability of classical PF solvers, and can quantum mechanisms improve RL learning efficiency?

In Chapter 7, we show that our proposed classical RL approach enhances NR initializations under poor initial conditions. The results confirm the approach's scalability, as performance gains are maintained when the classical RL agent is trained for a larger test system. A key innovation is integrating AQC into the RL environment update mechanism by implementing the algorithm developed in Chapter 5. This integration enables efficient exploration of the combinatorially large action space via a quadratic unconstrained binary optimization (QUBO) formulation, offloading the combinatorial optimization task to an Ising machine. The proposed QRL approach enables faster convergence, fewer NR iterations, and greater robustness across diverse operating scenarios compared to classical RL.

## 8.2. Discussion and Recommendations

The contribution of this thesis is methodological. That is, PF analysis and OPF, and more broadly, problems expressible as root-finding over continuous variables, can be re-cast so that emerging computational techniques augment rather than replace the trusted classical solver. The value lies less in any single solver than in this transferable reformulation, which is what makes the results relevant beyond power systems. As shown in Figure 8.1, the proposed approach extends naturally to various power and energy system applications, including 2D heat transfer [1]. With ongoing advancements in quantum hardware, scalable AI architectures, and real-time simulation environments, the methodologies established in this thesis provide a foundation for next-generation computational tools that support reliable and efficient modeling of power and energy systems.

A continuation of the work presented in this thesis is to investigate how emerging computational paradigms can be systematically integrated into grid operation workflows at scale. In quantum optimization, advancements in hardware-efficient embeddings, quantum-native problem formulations, and warm-start strategies could substantially improve performance for large-scale PF and OPF problems. From a systems perspective, real-time digital twins that couple high-fidelity simulators with quantum or AI solvers may support continuous grid monitoring and adaptive control in the near to mid future. Finally, extending problem reformulation techniques to broader classes of differential and algebraic equations, such as thermal networks,

computational fluid dynamics, building energy management, and transportation systems, will help identify opportunities for algorithmic acceleration.

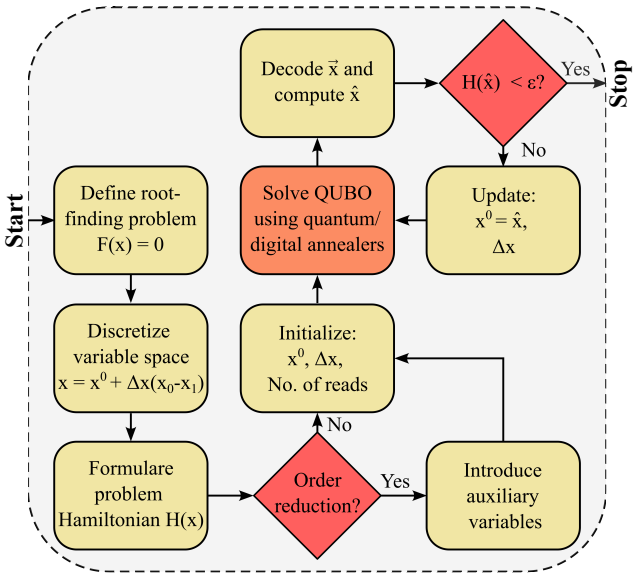



Figure 8.1: Schematic representation of the proposed framework for solving power system problems with Ising machines.

## References

[1] **Z. Kaseb**, M. Möller, P. Palensky and P. P. Vergara. 'A framework for solving continuous energy and power system problems using adiabatic quantum computing'. In: *arXiv preprint* (2026). DOI: arXiv:2504.06458.

# Acknowledgements

A PhD defense is the culmination of a long journey spanning years of academic study. My path as a student was far from straightforward, and this acknowledgment is where I pause to reflect on it. Over the past 14 years, countless events have shaped my journey and the person I am today. Along the way, people also left their mark in their own ways. I owe gratitude to all of them. For just as those who supported me helped me move forward, those who challenged me pushed me to think differently, grow stronger, and discover new paths I might never have found otherwise.

I love science. This passion has carried me through every turn of this journey. Even as a child, I would seize every moment to learn more of the world around me. I was never satisfied with what was taught in the classroom, so I also sought knowledge elsewhere through side activities like RoboCup soccer simulations and rescue training courses. As life does not always go as expected, I missed the national university entrance exam in 2011. So, I began my studies in architectural engineering at a university that did not require an admissions exam. After graduation, I spent a year preparing for the first change in my field, a shift toward building technology, where I also found space to explore programming, machine learning, and optimization techniques. I wanted to become someone people could turn to for computational solutions. After graduation, I ended up in China in pursuit of this goal, and suddenly the world began to change. COVID-19 had just emerged, and I became stranded for several days, without food and with no hope of returning home. In the end, I managed to return and did not stop. It took me another two years to shape my skills. During that time, I had the opportunity to work with a professor from Eindhoven University of Technology. Yet, as life does not always go as planned, after a fulfilling but ultimately unsuccessful attempt to join his team, I finally found my place at TU Delft, where I could further grow. When I received the offer, my sister told me, 'Don't go this far. Stay here, and we'll work together in our family business. Life is too short, you'll miss being with us, and the joy of watching the kids grow.' She was right. I left my heart behind, and replied, 'You're right, but I love science.'

Of course, many people have contributed to my growth and to this long journey, but I would like to mention those who have had the greatest impact and substantially changed my horizons. **Peter**, thank you for the trust you placed in me, and for the support I always felt in both big and small matters. Weijie recorded a film in which you announced the TUD ESE Apple Prize to me. I felt deeply proud when I heard your words. You might not remember, but I will never forget the day I asked for feedback on my research statement. You looked for me, office by office, to share your thoughts. That moment made me feel truly supported. I am happy that during our visit to Tsinguha University, I had the chance to spend time with you and learn more about your impressive and inspiring personality. As my promoter, I could not

have wished for someone more supportive and insightful. **Matthias**, I am grateful that my journey eventually crossed from your office and that I have had you as my second promoter since a few months after joining TU Delft. You were the one who gave me the courage to dive into the quantum world and move closer to who I had always wanted to become with your simple encouragement, 'you can learn', months before I started my PhD, and also through your great mentorship during my PhD. Even if it was a brief chat during your coffee break at an event in X, you always had something meaningful to share, something that reignited my curiosity and reminded me why I love science. You are a brilliant scientist, yet what I admire most is your honesty and integrity. **Pedro**, I still remember telling you that I wanted to explore quantum computing alongside artificial intelligence. You smiled and said, 'You should first convince me.' A few weeks later, I returned to you with some initial results. You seemed convinced, and from that moment, you never held me back. I cannot thank you enough for believing in me and for supporting my growth at every step. I am also deeply grateful for the respect you have always shown toward my space, and for ensuring that others do the same. I truly value the safe and open environment you have created within our Distribution Systems team, as well as your constant willingness to help in every way you can. You treat us not as students, but as friends, and that made all the difference, as I could only grow in such an atmosphere.

Beyond my PhD supervisors, I would like to thank Dr. Hamid Montazeri. You were missed on the supervisory side at my defense ceremony. Your seminar in 2019 at the University of Tehran and our collaborations over two years marked a true turning point in my career. I will never forget your contribution to shaping my identity as a researcher. I can easily trace your influence when I review, create slides, present, write, and even when I look for consistency or think critically. If life ever leads me to a place where I become a teacher, I hope to make the same positive impact that you have made on the paths of many students, including me. Furthermore, I would like to thank Dr. Morteza Rahbar. Your course on artificial intelligence in 2017, and our collaborations over the following three years, including organizing several national and international workshops, teaching, and supervising students, became an important chapter in my journey. I would also like to thank Hossein Ansari and Dr. Amin Milani Fard. It has been a very long time, but your contribution to shaping my skills remains significant. While my secondary school had offered an advanced curriculum, yet another unexpected transition to a standard high school left me with almost nothing new to learn during the first year. Then, your association 'PDP' announced a program to promote object-oriented programming and RoboCup soccer simulations in 2007, which made my high school years truly fulfilling. I still see that program as one of the defining chapters of my journey. Lastly, I would like to thank Prof. Mohammadreza Hafezi, Dr. Mohammad Tahsildoost, Dr. Hadi Safaeipour, and Dr. Ali Javid, among those who broadened my horizons and encouraged me to see the world in a different light. I would like to further thank all my friends and colleagues at TU Delft, including those in the sections Intelligent Electrical Power Grids, Numerical Analysis, and Building Design & Technology. I would also thank my friends outside the Netherlands for their kindness, presence, and support across the distance.

Last but not least, I want to thank my family; to my mother, my father, my sisters, and their husbands. Thank you for your ever-present love and support. My heartbeat is set to your rhythm, and no long distance or time difference can ever change this; to my dear nephews, Ali, Vahid, Taha, and Mehrdad, and to my sweet niece Noora. I always find myself searching for the sense of your laughter and purity in each and every child I meet. Finally, my appreciation goes to the man who, though not my father by blood, has been a father to me since childhood in every way that truly matters. Thank you for walking beside me as I grow into a better version of myself, with a compassion deeper than I can ever put into words.

If you have reached the end of this long acknowledgment, I would like to share a truly valuable experience during my PhD. Living alone far from my family has never been easy, especially for someone whose loved ones never wish even to leave the same neighborhood. It is more than missing someone or feeling alone. It is an encounter with yourself. It shapes you in solitude on cold days and through long, quiet nights. It is what it is, and unless you live it, you cannot truly imagine what it feels like. Yet, if you happen to survive it, you meet a version of yourself that amazes you. The moment you no longer run from it, stay, breathe, and embrace it, you are repaid in full. I received a gift I never thought I would find, in my 30s, far from home. I also learned, wholeheartedly, that a true sense of peace comes from within, and that my real and meaningful influence on the world begins with the small changes I make toward becoming a better version of myself. To whoever is reading this, I wish you a true sense of clarity and peace.

*Zeynab*
*Delft, February 2026*

# Curriculum Vitæ

## Zeynab Kaseb

| | |
|---|---|
| 11-10-1992 | Born in Mashhad, Iran. |

## Education

| | |
|---|---|
| 2022–2026 | PhD in Electrical Engineering<br>Delft University of Technology, Delft, Netherlands<br>*Thesis:* Quantum Computing and Artificial Intelligence for Power System Modeling and Control<br>*Promotors:* Prof. dr. P. Palensky & Dr. M. Möller & Dr. P.P. Vergara Barrios |
| 2016–2019 | MSc in Building Technology<br>Shahid Beheshti University, Tehran, Iran<br>*Thesis:* CFD-Based Optimization of Urban Microclimate<br>*Promotors:* Prof. dr. M.R. Hafezi & Dr. M. Tahbaz |
| 2011-2015 | BSc Architectural Engineering<br>Azad University, Mashhad, Iran<br>*Thesis:* Computational Design of Interactive Spaces<br>*Promotor:* Dr. L. Zamani Aghaei |

## Honors & Awards

| | |
|---|---|
| 2008 | Ranked $6^{th}$ in RoboCup Brazil Open |
| 2016 | Ranked $18^{th}$ in Nationwide University Entrance Exam |
| 2024 | TUD ESE Apple Prize for Outstanding Research Work |
| 2024 | TUD Best PowerWeb Paper Award |
| 2025 | INQA International Exchange Award |

# List of Publications

## First-Author Publication

1. **Z. Kaseb**, M. Möller, L. Spoor, J. J. Guo, Y. Xiang, P. Palensky and P. P. Vergara. 'Reinforcement learning for Newton-Raphson initialization in stressed AC power flow with a quantum annealing update mechanism'. In: *arXiv preprint* (2026). DOI: arXiv:2511.20237

2. **Z. Kaseb**, M. Möller, P. Palensky and P. P. Vergara. 'A framework for solving continuous energy and power system problems using adiabatic quantum computing'. In: *arXiv preprint* (2026). DOI: arXiv:2504.06458

3. **Z. Kaseb**, M. Möller, P. Palensky and P. P. Vergara. 'Power flow and optimal power flow using quantum and digital annealers: a computational scalability analysis'. In: *arXiv preprint* (2026). DOI: arXiv:2505.15978

4. **Z. Kaseb**, R. Rane, A. Lekić, M. Möller, A. Khodaei, P. Palensky and P. P. Vergara. 'Quantum hardware-in-the-loop for optimal power flow in renewable-integrated power systems'. In: *IEEE Transactions on Power Systems* (May 2026). DOI: 10.1109/TPWRS.2026.3696764

5. **Z. Kaseb**, S. Orfanoudakis, P. P. Vergara and P. Palensky. 'Physics-informed neural network with adaptive activation for power flow'. In: *International Journal of Electrical Power & Energy Systems* 174 (2026), p. 111525. DOI: 10.1016/j.ijepes.2025.111525

6. **Z. Kaseb**, M. Möller, P. P. Vergara and P. Palensky. 'Power flow analysis using quantum and digital annealers: a discrete combinatorial optimization approach'. In: *Nature Scientific Reports* 14.1 (2024), p. 23216. DOI: 10.1038/s41598-024-73512-7

7. **Z. Kaseb**, M. Möller, G. T. Balducci, P. Palensky and P. P. Vergara. 'Quantum neural networks for power flow analysis'. In: *Electric Power Systems Research* 235 (2024), p. 110677. DOI: 10.1016/j.epsr.2024.110677

## Book Chapter

1. M. Möller, **Z. Kaseb**, F. Key and M. Dukalski. *Quantum Optimization Applications*. 2026

## Conference Paper

1. **Z. Kaseb**, M. Möller, P. Palensky and P. P. Vergara. 'Can adiabatic quantum computing address unsolved challenges in power system problems?' In: *Applied Quantum Methods in Computational Science and Engineering (AQMCSE)*. 2025

2. **Z. Kaseb**, M. Möller, M. Kirsch, P. Palensky and P. P. Vergara. ‘Combinatorial power flow analysis using adiabatic quantum algorithms’. In: *IEEE PowerTech*. 2025

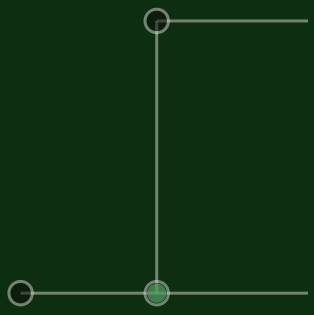
3. **Z. Kaseb**, M. Möller, P. Palensky and P. P. Vergara. ‘Solving complex systems of equations using adiabatic quantum computing’. In: *Adiabatic Quantum Computing (AQC)*. 2025

4. S. Yan, F. Vazinram, **Z. Kaseb**, L. Spoor, J. Stiasny, B. Mamudi, A. H. Ardakani, U. Orji, P. P. Vergara, Y. Xiang and J. Guo. ‘Data driven approach towards more efficient Newton-Raphson power flow calculation for distribution grids’. In: *ICT.OPEN*. 2025

5. **Z. Kaseb**, M. Möller, M. Kirsch, P. Palensky and P. P. Vergara. ‘Hybrid quantum physics-informed neural networks for power flow analysis’. In: *IEEE PES ISGT EUROPE*. 2024

6. **Z. Kaseb**, M. Möller, G. T. Balducci, P. Palensky and P. P. Vergara. ‘Quantum neural networks for power flow analysis’. In: *Power Systems Computation Conference (PSCC)*. 2024

7. **Z. Kaseb**, Y. Xiang, P. Palensky and P. P. Vergara. ‘Adaptive activation functions for deep learning-based power flow analysis’. In: *IEEE PES ISGT EUROPE*. 2023



ISBN 978-94-6518-375-6